ETL 7

In hoofdstuk 1 is het datawarehouse de '*single version of the truth*' genoemd. In hoofdstuk 3 is als een van de argumenten om een datawarehouse te bouwen de slechte kwaliteit aangegeven van de gegevens in operationele databases. Blijkbaar denken we in staat te zijn om die slechte kwaliteit te verbeteren tot 'de waarheid' door die gegevens uit de operationele systemen te halen en op te slaan in het datawarehouse. Absoluut geen kleine prestatie. Iedereen kent het cliché '*garbage in, garbage out*', maar hier willen we het omgekeerde doen: de waarheid maken van de troep die binnenkomt.
Bovenstaande uitdaging is het ETL-proces in een notendop.

7.1 Inleiding

ETL staat voor Extract, Transform en Load. De Extract staat voor het lezen van gegevens uit operationele systemen en andere mogelijke bronnen. De Load staat voor het vullen van het datawarehouse (of een datamart) met gegevens. De Transform in het midden moet de ingelezen gegevens zo manipuleren dat de organisatie uiteindelijk bereid is om de informatie die uit het datawarehouse komt, te accepteren als de waarheid. Dat is al een nuance ten opzichte van dat het de waarheid is, maar nog altijd geen kleine opgave. Als we het hier over het datawarehouse hebben, bedoelen we over het algemeen de datamarts waar eindgebruikers mee werken. In het echte datawarehouse en/of in het Data Lake zitten de gegevens vaak in hun originele vorm.
Op Wikipedia staat de volgende definitie van ETL: 'ETL is een groep technologieën die veelal gebruikt wordt bij de koppeling tussen systemen, waarbij er gestreefd wordt naar een minimale technische en semantische koppeling. Het is een batchproces dat regelmatig gebruikt wordt.'
In het kader van dit boek kan de koppeling tussen systemen op twee manieren worden opgevat. Enerzijds kennen de meeste Business Intelligence-oplossingen meer bronnen. Er worden dus meer bronnen gekoppeld. Anderzijds is het de koppeling tussen operationele systemen als CRM-systemen en het datawarehouse.



ETL is een groep technologieën die veelal gebruikt wordt bij de koppeling tussen systemen, waarbij er gestreefd wordt naar een minimale technische en semantische koppeling. Het is een batchproces dat regelmatig gebruikt wordt.

De minimale koppeling waar de definitie over spreekt is belangrijk voor de onderhoudbaarheid van het ETL-proces. De verschillende bronsystemen kunnen allemaal technisch verschillend zijn: relationele databases gemaakt in verschillende DMBS'en, tekstbestanden, RSS-feeds, REST API's en zo verder. Dit mag geen probleem opleveren op technisch vlak. De semantische koppeling zit meer richting de BI Life Cycle waar in hoofdstuk 2 over is gesproken. De organisatie mag anders tegen de wereld gaan aankijken, een kolom blijft een kolom. Verder moeten we er rekening mee houden dat bronsystemen kunnen veranderen bij de implementatie van nieuwere versies van de database. We moeten er naar streven dat deze veranderingen minimale impact hebben op de ETL en daarmee op het datawarehouse.
In dit hoofdstuk zullen we de verschillende aspecten van ETL op een rij zetten en enkele voorbeelden uitwerken. We beginnen echter met te kijken naar Master Data, een eerste stap in het omgaan met verschillende operationele databases.

7.2 Master Data Management

In de inleiding en in hoofdstuk 1 is gesproken over het bestaan van meer dan één operationeel systeem met ieder zijn eigen database(s) als een probleem dat (mede) leidt tot Business Intelligence-initiatieven. Om te zorgen voor eenduidige rapportages brengen we alle gegevens onder in één centraal datawarehouse. Zolang alle rapporten dit datawarehouse gebruiken als bron, is er eenduidigheid over alle rapportages. Er blijven echter twee problemen over:

- De operationele processen hebben nog steeds last van de niet-eenduidige gegevens.
- Tijdens de ETL moet gekozen worden 'welke waarheid' in het datawarehouse wordt opgenomen.

Op operationeel niveau kunnen er veel dingen fout gaan als verschillende processen en/of afdelingen werken met hun eigen gegevens.

Voorbeeld 7.1



Een huiseigenaar wordt door zijn bank gebeld met een aanbieding voor een nieuwe hypotheek, die gegarandeerd voordeliger is dan zijn huidige. Een paar maanden daarvoor heeft deze persoon echter zijn laatste hypotheekbetaling gedaan en zijn huis is volledig afgelost en vrij op naam. Blijkbaar is de database van de marketingafdeling een andere dan die van de hypotheekafdeling!

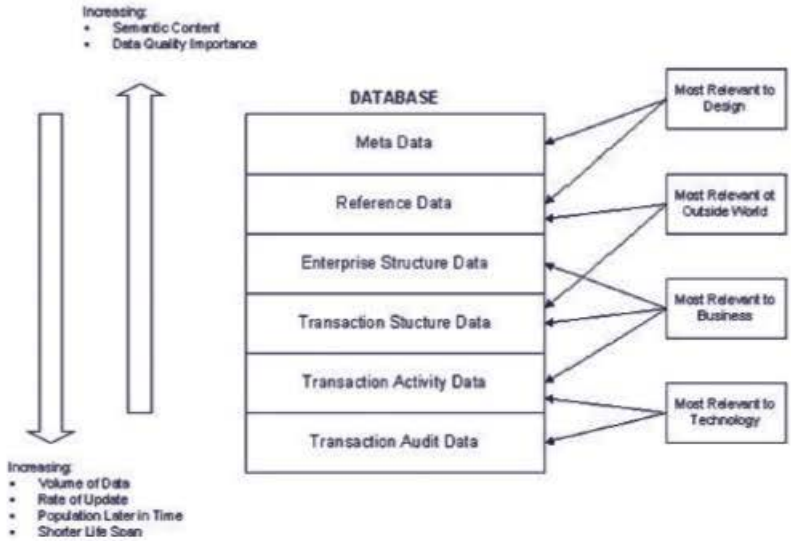
Voor het ETL-proces zijn er eigenlijk twee problemen:
- Twee verschillende systemen geven tegenstrijdige informatie.
- Voor twee verschillende systemen ziet de waarheid er anders uit.

Het eerste punt is hetzelfde als het voorbeeld. Uit één database komt naar voren dat een klant een hypotheek heeft, de andere database zegt van niet. Hoe verwerken we deze tegenstrijdige informatie in het datawarehouse?
Het tweede probleem ontstaat doordat andere processen met een andere bril naar hetzelfde kijken. Bij de afdeling inkoop worden producten gecategoriseerd naar leverancier. Bij de afdeling transport naar gewicht en volume. In het datawarehouse blijkt het soms moeilijk of onmogelijk om op beide manieren naar producten te kijken. Vaak wordt voor de ETL één database gekozen als 'leading' waardoor misschien informatie uit andere systemen verloren gaat. Erger is wellicht dat eindgebruikers het datawarehouse niet kunnen/willen gebruiken door de in hun ogen verkeerde weergave van zaken.

Master Data Management is een poging om dergelijke problemen aan te pakken. Voordat we Master Data en Master Data Management zullen definiëren, loont het om eens nader te kijken naar het soort en type gegevens dat we hebben en moeten beheren.

7.2.1 Verschillende soorten gegevens

Figuur 7.1 zet de verschillende soorten gegevens die in een database zitten op een rij. Verschillende soorten gegevens spelen een andere rol en moeten daarom anders beheerd worden. Van boven naar beneden zie je:
- metadata;
- reference data;
- enterprise structure data;
- transaction structure data;
- transaction activity data;
- transaction audit data.



Figuur 7.1 Soorten gegevens

Metadata

Metadata zijn gegevens over de gegevens. Met andere woorden, de metagegevens beschrijven de overige gegevens. Welke kolommen zijn er, wat is hun naam, in welke tabel zitten ze, wat is hun betekenis en welke mogelijke waarden (domein) kunnen voorkomen? Op meer technisch niveau is het van belang om te weten van welk datatype een kolom is. Voor een groot deel zitten deze gegevens in de systeemtabellen (de catalog) van de database. Vaak worden ze ook opgeslagen in een zogenaamde Data Dictionary.

Metadata zijn gegevens die de gegevens in een database beschrijven, zoals de naam van een kolom en het datatype van die kolom.

Reference data

Reference data zijn gegevens in look-up-tabellen. Je kunt hierbij denken aan een landentabel met ISO-landcodes, de officiële namen en de telefooncode van het land. Dit brengt externe informatie in de database. Maar ook een tabel met bijvoorbeeld klantcategorieën valt onder reference data. Het kan betekenis toevoegen, omdat een categorie iets zegt over bijvoorbeeld de toegestane kredietlimiet van een klant.

Enterprise structure data

Enterprise structure data zijn gegevens zoals grootboekrekeningen of journaalposten. Alle financiële handelingen zijn gecategoriseerd in kosten en baten en vervolgens door middel van een hele hiërarchie verder onderverdeeld. De hiërarchie is specifiek voor een bedrijf. Elk bedrijf zal zo zijn eigen grootboekrekeningen hebben.

Transaction structure data

Transaction structure data zijn de gegevens die nodig zijn om primaire gegevens over processen te kunnen opslaan. In een verkoopproces valt er niets te verkopen als er geen producten en geen klanten zijn. Product- en klantgegevens vallen onder transaction structure data. Dit zijn de dimensietabellen in een datamart.

Transaction activity data en transaction audit data

De laatste groep gegevens is de registratie van de feitelijke processen. Het registreren van een opname in een ziekenhuis, het opslaan van een melding in een incidentenapplicatie, een order in een verkooporganisatie: het zijn allemaal voorbeelden van transaction data. Transaction activity data zijn de records in een feitentabel.

7.2.2 Definities

Master Data

Master Data omvat de categorieën reference data, enterprise structure data en transaction structure data. Meestal wordt Master Data gedefinieerd in voorbeelden in plaats van een beschrijvende zin.

Een voorbeeld dat vaak genoemd wordt is klanten. Een organisatie bestaat uit meer processen uitgevoerd door meer afdelingen. Veelal worden de verschillende processen van de verschillende afdelingen ondersteund door eigen losstaande applicaties. Elke applicatie heeft zijn eigen database.

Dit alles leidt ertoe dat er meer klantenlijsten of klantentabellen bestaan binnen één organisatie. Alle klanten zitten in de financiële database omdat ze facturen hebben ontvangen en betalingen hebben gedaan. Ze zitten ook in de CRM-database omdat ze orders hebben besteld en contact hebben gehad met hun accountmanager. Dit brengt twee risico's met zich mee:

- Gegevens komen niet overeen.
- Definities zijn niet eenduidig.

In bovenstaande situatie kan (en zal) het voorkomen dat een verhuizing in de ene database wel verwerkt is en in een andere niet. Voor dezelfde klant bestaan nu twee adressen waardoor een deel van de post naar het verkeerde adres gaat en de klant nooit bereikt.

Naast dit probleem blijkt vaak dat de definitie van het begrip klant niet eenduidig is. In de marketingdatabase staan vooral kolommen die iets zeggen over demografie. Wat is de geboortedatum van de klant, welk opleidingsniveau heeft hij of zij, heeft de klant kinderen, et cetera? Op de afdeling financiën is een klant gedefinieerd met kolommen als bankrekeningnummer en creditcardnummer.

Los van de verschillende gegevens die van klanten bijgehouden worden, worden ze ook anders ingedeeld. Vaak zijn er hiërarchieën die op rapporten als

drill-downpaden gebruikt worden om van grove bij gedetailleerde informatie te komen. Elke tabel zal zijn eigen hiërarchieën kennen.
Master Data is eigenlijk het idee dat er voor een organisatie maar één lijst van klanten is. Logisch gezien doet een organisatie zaken met een lijst van mensen of organisaties. Dat zijn dé klanten. Als we Master Data betrekken op klanten, wordt het ook wel Customer Data Integration genoemd. Master Data is echter veel breder. Ook gegevens over producten, grootboekrekeningen, medewerkers en leveranciers zijn voorbeelden van Master Data. Wat hier wordt omschreven als Master Data komt sterk overeen met dimensies in een datawarehouse.



Master Data is de combinatie van reference data, enterprise structure data en transaction structure data. Een voorbeeld van Master Data zijn klantgegevens waarvan in het ideale geval in een organisatie één centrale set wordt bijgehouden die bepalend is voor alle processen.

Master Data Management

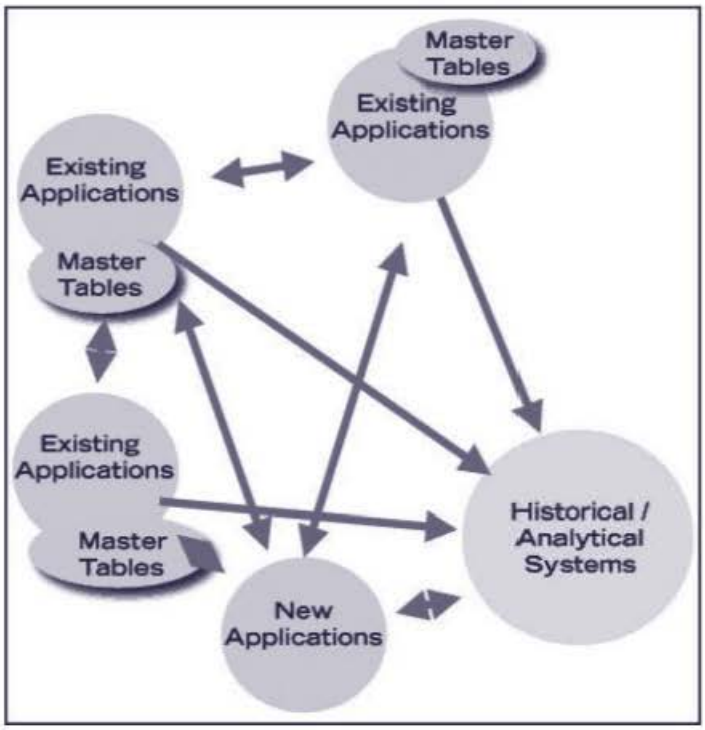
Op het moment dat je het belang van Master Data inziet en initiatieven gaat ontwikkelen om centrale opslag van deze gegevens te realiseren, ben je bezig met Master Data Management (MDM). De definitie van MDM is: '*Master Data Management (MDM) is a set of disciplines and processes for ensuring the accuracy, completeness, timeliness and consistency of the most important types (or domains) of enterprise data across different applications, systems and databases, multiple business processes, functional areas, organizations, geographies and channels.*' (Dan Power, z.d.).



Master Data Management is een verzameling disciplines en processen die zorgt voor accurate, complete, tijdige en consistente gegevens voor de belangrijke entiteiten binnen een organisatie over verschillende databases, afdelingen en landen heen.

Het tweede deel van deze definitie is eigenlijk een herhaling van wat Master Data is. Het eerst deel ('*a set of disciplines and processes*') geeft aan dat Master Data Management, net als Business Intelligence, geen eenmalig project is. Het is een continu proces om een centrale en correcte kopie te krijgen en te houden van de Master Data. Bovendien blijkt uit de definitie dat MDM niet los gezien kan worden van datakwaliteit.

7.2.3 Implementatie

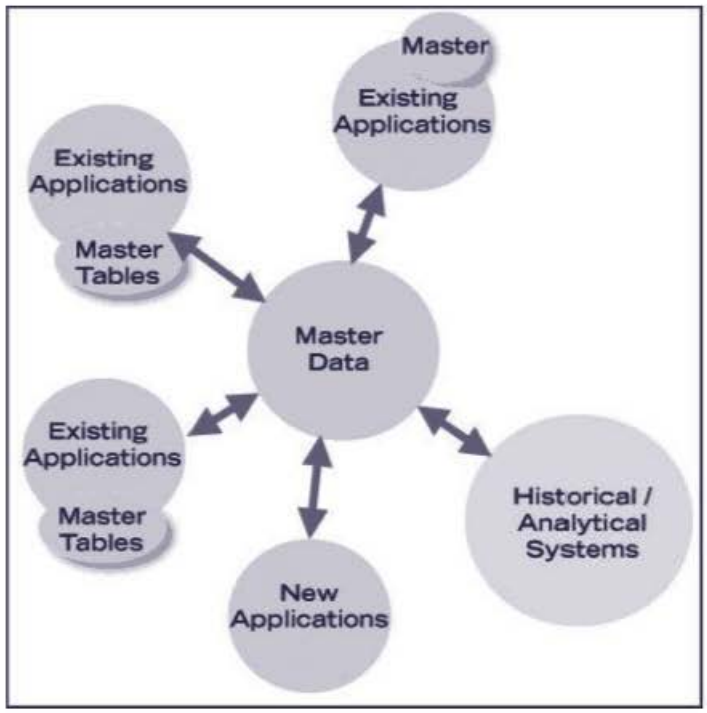


Figuur 7.2 Master Data many-to-many

Figuur 7.2 schetst het probleem dat Master Data Management moet oplossen. Een organisatie heeft meer applicaties die elk hun eigen set van Master Data bevatten. In het plaatje zijn dat drie verschillende systemen. Bij het vullen van het datawarehouse moet gekozen worden welk systeem van deze drie leidend is. Als je die keuze niet wilt maken, moet je tijdens het laden eventuele verschillen gladstrijken. In de praktijk is dat een onmogelijke taak.
Figuur 7.3 laat zien waar je heen wilt. Centraal staat de Master Data. De centrale Master Data is leidend bij het vullen van het datawarehouse. Zelf wordt de Master Data gevoed vanuit de applicaties. De applicaties worden echter weer gesynchroniseerd met de centrale Master Data. De centrale Master Data is leidend als er tijdens die synchronisatie verschillen optreden.

Vanwege de centrale rol van de Master Data wordt er vaak gesproken over een Master Data hub.
Let wel op dat MDM niet een project is en al helemaal geen ICT-project. Het is een multidisciplinair proces waarin data governance centraal staat. MDM is eigenlijk een onderdeel van het veel grotere data governance, waarbij 'governance' vertaald zou kunnen worden als 'beleid'.

Data governance staat voor het beleid dat een organisatie voert ten aanzien van gegevens en informatie.



Figuur 7.3 Master Data one-to-many

Als onderdeel van data governance moet een organisatie nadenken over hoe om te gaan met gegevens. Wetgevers en toezichthouders stellen daar regels voor op waar je je aan moet houden. Dat wordt compliancy genoemd. Een belangrijk deel van data governance is te zorgen dat je compliant bent. Zaken als beveiliging, verantwoordelijkheid en auditing horen daar ook bij. Master Data Management kan hier een belangrijke rol in spelen.

Compliancy is het voldoen aan (wettelijke) regels en eisen opgelegd door externe partijen zoals overheden en toezichthouders.

Data governance en MDM zijn relatief nieuw en veel projecten om MDM te implementeren mislukken (nog). Voor een deel is de reden hetzelfde als bij Business Intelligence: het wordt te veel gezien als een ICT-project. De technische implementatie is maar een klein deel van een MDM-implementatie. Omdat het multidisciplinair is en deels gaat om eigenaarschap en verantwoordelijkheid, is er een groot politiek aspect aan een MDM-implementatie.

Belangrijke zaken voor een goede introductie van MDM in een organisatie zijn:
- een projectleider die politieke gevoeligheden ziet en begrijpt en die oplossingen daarvoor kan aandragen;
- sponsorship van het management;
- een goede Business Case met een heldere doelstelling die gecontroleerd kan worden;

- duidelijke scope van het (initiële) project;
- inzicht in het businessmodel.

Business Case en doelstelling

Master Data Management kan op drie punten voordelen opleveren:

- Op operationeel niveau worden er minder fouten gemaakt en/of kan er efficiënter gewerkt worden, als de gegevens consistenter zijn.
- Op Business Intelligence-niveau kunnen eenduidiger analyses gemaakt worden van gegevens uit verschillende systemen.
- Het is makkelijker om te voldoen aan wet- en regelgeving als de gegevens centraal staan met duidelijke regels omtrent eigenaarschap en veranderingen.

Concreet zou dit kunnen leiden tot de volgende Business Case:

Voorbeeld 7.2



Een bedrijf dat hardware van verschillende merken verhuurt en verkoopt, levert computers aan klanten op basis van specificaties van de klant. Regelmatig treden daar problemen op omdat van processoren verschillende specificaties de ronde doen. Van een Intel Core i7-processor kun je de specificaties vinden bij Intel, maar ook bij leveranciers van notebooks en ook bij veel vergelijkingssites. Wat een processor levert kan ook nog eens verschillen per moederboard. Bij problemen moeten geleverde spullen omgeruild worden, wat tijd en geld kost. Door met één set van centrale specificaties van hardware te gaan werken, kan het aantal omgeruilde leveringen teruggedrongen worden. De doelstelling is een reductie van 25 procent in het eerste jaar.

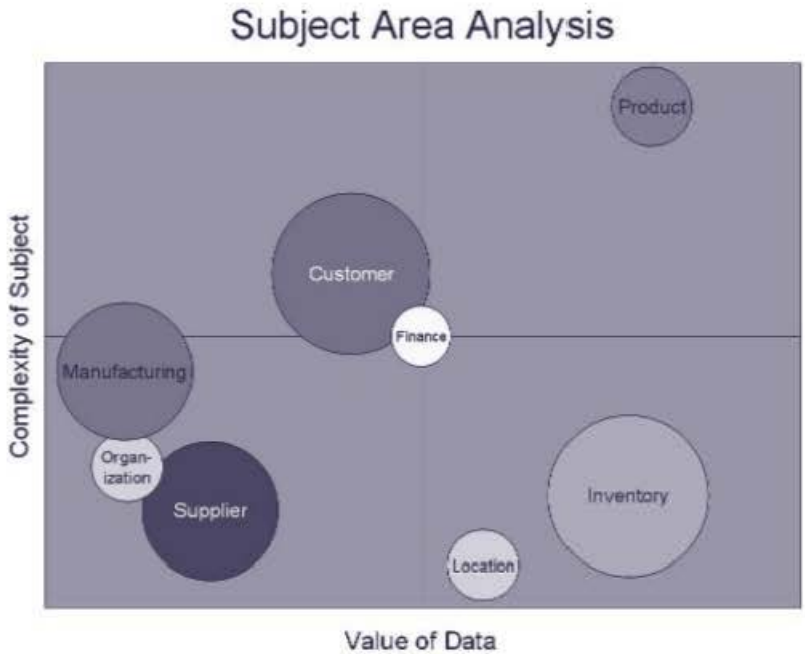
Scope

Net zoals bij een Business Intelligence-project is het van belang om de scope van een MDM-implementatie goed te definiëren. Big Bang-implementaties waarbij alle Master Data tegelijkertijd wordt aangepakt zullen mislukken. De hoeveelheid aan gegevens, maar vooral de hoeveelheid aan betrokken mensen en processen, is te groot. Kleine implementaties zijn overzichtelijk en hebben daarom een kans van slagen. Bovendien kan het nut van MDM daardoor worden bewezen, wat vanzelf leidt tot verdere implementaties.

De scope kan bijvoorbeeld zijn alleen een Master Data-set te maken van productgegevens.

Figuur 7.4 zou kunnen helpen bij het bepalen van de scope. De afbeelding komt overeen met plaatjes die je eerder in het boek hebt gezien. In een matrix zijn complexiteit uitgezet tegen meerwaarde waarbij de grootte van de bubble ook nog de hoeveelheid gegevens aangeeft. Het is verstandig om te beginnen bij de kleine bubbles en bij de gebieden die ingeschat worden als gemakkelijk en met veel toegevoegde waarde.

Gedeeltelijke MDM-implementaties kunnen ook overwogen worden. Stel dat afdeling A vijftien kolommen bijhoudt per product, terwijl afdeling B er dertig heeft. Tien van deze kolommen blijken hetzelfde te zijn. Er bestaat echter grote weerstand op de afdelingen om 'hun' informatie over te dragen aan een centraal MDM/data governance-team. Er zou (in eerste instantie) gekozen kunnen worden om de overeenkomstige kolommen op te nemen in een MDM-oplossing en de overige kolommen (voorlopig) te laten waar ze zijn.



Figuur 7.4 Mogelijke aandachtsgebieden bij MDM

Businessmodel

Van de gegevens die opgenomen gaan worden in een MDM-oplossing moet uitgezocht worden waar ze vandaan komen en welke regels ervoor gelden. Dat kunnen extern opgelegde eisen zijn, maar ook interne businessregels. Achterhaald moet worden wie deze regels kent en wie verantwoordelijk is, of gaat zijn, voor de naleving van deze regels.

Met bovenstaand voorbeeld zeg je eigenlijk dat er een data governance-proces opgezet moet worden. Misschien houdt dat wel in dat er een team samengesteld moet worden. Mensen in zo'n team worden vaak data stewards genoemd. Dat zijn mensen met inhoudelijke kennis van zaken die verantwoordelijk gemaakt worden voor de gegevens.



Data stewards zijn mensen met inhoudelijke kennis van zaken die verantwoordelijk zijn voor de gegevens.

7.2.4 Voorbeeld met Microsoft MDS

In deze paragraaf zullen we een eenvoudig voorbeeld uitwerken van MDM op basis van Microsoft SQL Server 2016 Master Data Services (MDS). Het doel is om de bovenstaande theorie concreet te maken. De uitwerking van dit voorbeeld (met daarin de figuren 7.5 t/m 7.7) kun je online bekijken op de website bij dit boek



7.2.5 Tot slot

Waar het bovenstaande voorbeeld stopt, begint het in de praktijk pas. Een data steward wordt verantwoordelijk voor de leveranciersgegevens. Het is zijn taak om de gegevens in MDS correct te houden. Wijzigingen kunnen met Excel doorgevoerd worden. Ook nieuwe leveranciers kunnen toegevoegd worden. De gegevens moeten vervolgens gecontroleerd worden. Daarna kunnen ze geaccepteerd worden. Dat is een belangrijke stap hier.
Als MDS als hub moet gaan dienen, zoals getoond in figuur 7.3, moeten er synchronisatiemechanismes opgezet worden. Via de gemaakte view kunnen de gegevens gelezen worden om ze over te halen naar andere systemen.
Hiervoor is ervan uitgegaan dat we een specifieke set van software gebruiken om aan Master Data Management te doen. In veel bedrijven wordt een van de operationele databases aangewezen als 'leading'. Die database bevat dan wat we beschouwen als de Master Data. Met goede koppelingen tussen de verschillende systemen en goede processen rond synchronisatie kan dat uiteraard ook een oplossing zijn.

7.3 Implementeren van het ETL-proces

Na het uitstapje naar Master Data Management gaan we nu verder kijken naar het ETL-proces zelf. Ook voor het ETL-proces geldt, net als voor alle andere onderdelen van Business Intelligence, dat de informatieanalyse leidend moet zijn bij het nemen van ontwerpbeslissingen. Er moeten beslissingen genomen worden over welke gegevens overgehaald moeten worden, maar ook over de architectuur en de frequentie van het ETL-proces. Daarnaast moeten beslissingen genomen worden over de technische implementatie. Ga je de taal SQL (Structured Query Language) gebruiken om gegevensmanipulatie (de T van ETL) te doen, of gebruik je een van de vele beschikbare tools?

7.3.1 Resultaten van informatieanalyse

Uit de informatieanalyse moet een aantal inhoudelijke zaken duidelijk worden. Daarnaast moet ook een aantal meer technische zaken helder worden. De inhoudelijke zaken hebben betrekking op de gegevens zelf, maar ook over hoe up-to-date rapportages moeten zijn, hoe met wijzigingen van gegevens wordt omgegaan – zowel bij dimensiegegevens alsook bij feiten – en hoe lang gegevens in het datawarehouse (of datamart) beschikbaar moeten blijven. De technische aspecten hebben betrekking op het auditen en monitoren van de ETL en het omgaan met foutsituaties.

Hieronder worden de volgende aspecten van ETL die in de informatieanalyse aandacht dienen te krijgen, kort belicht:

- Welke kolommen moeten meegenomen worden?
- Data latency
- Historische gegevens
- Oude gegevens
- Auditing
- Fouten
- Monitoring

Welke kolommen?

Duidelijk is dat de informatiebehoefte (welke informatie moet op welke rapporten beschikbaar worden) stuurt welke kolommen overgehaald moeten worden naar het datawarehouse. In paragraaf 7.3.3 zullen we daar iets meer over zeggen.

Data latency

Het tweede punt waar aandacht aan besteed moet worden tijdens de informatieanalyse is vragen met betrekking tot het realtime zijn van gegevens. Wanneer zien gebruikers wijzigingen die zij aanbrengen in hun operationele applicaties terug in de rapporten? Als iemand op personeelszaken een nieuwe ziektemelding invoert, zien we die ziekte-uren dan meteen terug op het dashboard dat aangeeft of we qua ziekteverzuim op schema liggen om de doelstelling te halen? Of komt deze nieuwe ziektemelding pas morgen of overmorgen op de rapporten terecht?



De **latency** van een datawarehouse is de tijd die zit tussen het ontstaan van gegevens en het moment dat deze gegevens beschikbaar komen in het datawarehouse.

De eisen die gesteld worden aan de latency hebben in eerste instantie betrekking op de frequentie waarmee het ETL-proces uitgevoerd moet worden. Bij veel Business Intelligence-oplossingen zien we dat de ETL elke nacht wordt uitgevoerd. De latency is daarmee een dag. Gegevens die vandaag ontstaan, zijn pas morgen terug te vinden in het datawarehouse.

Voor de meeste managementinformatie (stuurinformatie) is het niet erg als de gegevens een dag oud zijn. Steeds vaker willen mensen echter (near) real-time-informatie en is een latency van een dag niet meer aanvaardbaar. Alles is mogelijk. Wel worden de oplossingen die nodig zijn complexer en duurder als de eisen omtrent latency stricter worden.

Historische gegevens

In paragraaf 3.7 is al het ontbreken van historische gegevens in veel operationele databases besproken. In paragraaf 4.1.4 is besproken hoe Slowly Changing Dimensions (SCD) een technische oplossing bieden om met historische gegevens om te gaan. In het ETL-proces moet je de gekozen manier om met veranderende gegevens om te gaan implementeren. Voor elke kolom die vanuit het bronsysteem wordt overgehaald, moet bekend zijn wat gedaan moet worden met gewijzigde waarden. Dit moet uit de informatieanalyse duidelijk worden. Vervolgens moet bedacht worden hoe je erachter komt of waarden gewijzigd zijn. Tot slot moet het juiste type SCD geïmplementeerd worden.

Naast dimensies die kunnen veranderen, kunnen ook records in feitentabellen veranderen. In paragraaf 4.1.7 is de accumulating snapshot-feitentabel besproken. Voor 'gewone' feitentabellen geldt dat je alleen nieuwe records hoeft toe te voegen aan de feitentabel. Bij een accumulating snapshot-tabel moeten echter ook al bestaande records gewijzigd worden. Uit de informatieanalyse moet duidelijk worden of dit scenario nodig is.

Naast accumulating snapshots zijn er ook situaties denkbaar waar feiten later aangepast moeten worden. Als je een feit overschrijft met de nieuwe waarde, veranderen rapporten met terugwerkende kracht. Is dat wenselijk?

De organisatie kan eisen dat elk rapport een peildatum krijgt. De waarden die getoond worden zijn de waarden die actueel waren op het moment van de peildatum. Bij een wijziging van het feit moet je een nieuw record aanmaken. Op basis van de peildatum moet dan het juiste record gelezen worden.

Als je van een feit twee records bijhoudt, gaan totaaltellingen niet meer goed. Een record opnemen met een tegenboeking kan dit oplossen.

De moraal van het verhaal is dat feiten achteraf kunnen wijzigen. Uit de informatieanalyse moet blijken hoe de ETL hiermee moet omgaan.

Oude gegevens

Naast historische gegevens in de zin van veranderende gegevens, is het interessant om te weten hoe ver terug in de geschiedenis het datawarehouse of de datamart moet gaan. Is het nog interessant om omzetgegevens van vijftien jaar geleden te bekijken? En zo ja, moeten die dan geïndexeerd worden om ze te kunnen vergelijken met de gegevens van het huidige jaar?

In paragraaf 6.4 zijn verschillende performance features besproken om query's op datawarehouses sneller te maken. De belangrijkste is misschien wel tabellen klein te houden. Voor een deel gaat dat in tegen het doel van een datawarehouse. Voor een datamart is het belangrijk om te bepalen hoe ver terug in de tijd gegevens vastgehouden moeten worden. Als uit de informatieanalyse blijkt dat alle

rapporten en analyses betrekking hebben op het huidige jaar en er hooguit wordt vergeleken met dezelfde periode in het voorgaande jaar, zorg dan dat er niet meer dan twee jaargangen aan gegevens in de datamart aanwezig zijn. Dit houdt in dat er in het totale ETL-proces ook een onderdeel moet zitten dat oude gegevens verwijdert.

Auditing

Alle bovenstaande eisen zijn functioneel van aard. Ze hebben rechtstreeks te maken met hoe mensen welke gegevens gaan zien op de rapportages. Er moet echter ook over een aantal meer technische specificaties worden nagedacht alvorens de ETL te implementeren. Eén daarvan is auditing.

Auditing is het toevoegen van metadata aan de gegevens. In de context van ETL-auditing komt dit neer op het toevoegen van informatie over de ETL zelf aan de gegevens die geladen worden. Meestal gaat het dan over het laden van feiten, maar auditing hoeft zeker niet beperkt te blijven tot feiten. Van elk geladen feit wordt bijgehouden wanneer het is geladen, door wie, door welk proces (package) en welke versie, en welke transformaties zijn uitgevoerd op de gegevens voor het laden.



Auditing is het toevoegen van metadata aan de gegevens in het datawarehouse zodat is te herleiden wie of welk proces de gegevens wanneer en hoe heeft geladen.

Door audit-informatie toe te voegen als dimensie aan het datawarehouse, kunnen vragen beantwoord worden als:

- Waar komt deze informatie vandaan (data lineage)?
- Hoe komen we aan deze getallen?

Audit-informatie hoeft niet per se als dimensie te worden toegevoegd. Andere manieren zijn ook denkbaar.

Auditing is cruciaal als je moet voldoen aan compliance-eisen. In steeds meer situaties moet je voldoen aan wetten en regels. Accountants eisen dat je precies kunt herleiden hoe en waar informatie tot stand is gekomen. Auditing is daarmee vaak een must. Uit de informatieanalyse moet naar voren komen aan welke compliance-eisen voldaan moet worden.

Fouten

ETL is software en bij software moet je nadenken over hoe je wilt omgaan met foutsituaties. Het kan bijvoorbeeld voorkomen dat je een numeriek gegeven als tekst krijgt aangeleverd. Tijdens de conversie van tekst naar numeriek blijken er toch alfanumerieke karakters voor te komen bij enkele records. Hoe moet het ETL-proces hiermee omgaan? Vervangen we de niet te vertalen gegevens naar 0 of wellicht naar NULL en wordt het record gewoon geladen? Of wordt het hele record geweigerd en bijvoorbeeld voor handmatige correctie in een aparte tabel weggeschreven?

Monitoring

Beheerders van ETL-processen moeten weten of de processen hebben gedraaid, of ze succesvol of met een fout zijn geëindigd, wat de totale doorlooptijd van het proces was en misschien nog wel andere zaken. Zorg daarom dat er in de informatieanalyse ook aandacht voor dit soort zaken is.

Nadat over alle genoemde zaken goed is nagedacht kun je een ETL-architectuur gaan bedenken.

7.3.2 Architectuur

Uit de informatieanalyse komen verschillende eisen naar voren die van invloed zijn op de opzet van de ETL, zoals in de vorige paragraaf duidelijk is geworden. Naast deze eisen is er nog een aantal factoren die een rol spelen:

- de hoeveelheid gegevens (in MB/GB/TB) die per keer overgehaald moeten worden;
- het aantal en soort bronnen;
- tijdstip en tijdsduur dat gegevens uit de bronnen ingelezen mogen worden.

Deze factoren bepalen welke architectuur het beste is. Grofweg kunnen er drie architecturen worden onderkend, maar er zijn vele variaties mogelijk. In hoofdlijnen zijn de oplossingen:

- zonder staging-laag;
- met staging;
- met twee staging-lagen.

De keuze is ook nog afhankelijk van de methodiek: Kimball, Inmon of Data Vault.

Geen staging



Figuur 7.8 Architectuur zonder staging database

Figuur 7.8 laat de architectuur zien zonder staging database. Een staging database of staging-laag is een plek waar de gegevens tijdelijk worden geparkeerd alvorens ze worden overgebracht naar het datawarehouse.

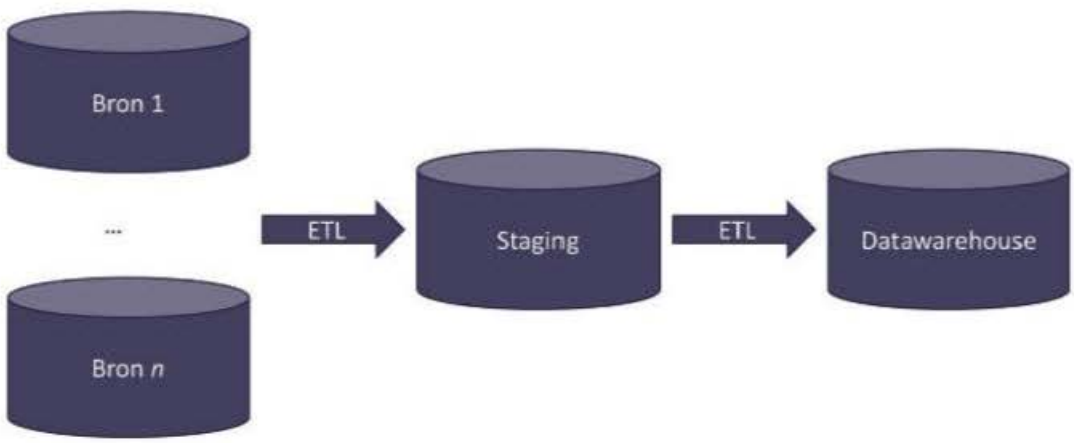


Een **staging database** is een database waar gegevens tijdelijk worden opgeslagen alvorens ze worden doorgestuurd naar het datawarehouse.

De architectuur zonder staging wordt niet veel gebruikt. Die is vooral geschikt als je weinig (misschien maar één) bronsystemen hebt en de hoeveelheid gegevens die overgehaald moeten worden beperkt is. Je hebt de hele nacht de tijd om de ETL te doen, gedurende die hele tijd kun je doen en laten wat je wilt met zowel het bronsysteem als het datawarehouse. Als er iets fout gaat tijdens de nacht, kun je de ETL overdag tijdens kantooruren opnieuw draaien. Ook met een inefficiënte aanpak is de ETL gemakkelijk klaar voordat de eerste mensen op kantoor komen en rapporten gaan gebruiken. Dit scenario is niet ondenkbaar, maar het zijn wel een heleboel 'als-en' bij elkaar.

Eén staging database

Figuur 7.9 laat een architectuur zien met één staging database. Deze architectuur geeft veel meer flexibiliteit dan een architectuur zonder staging en is daarom ook een veel toegepaste architectuur. In een eerste stap worden de benodigde gegevens uit de verschillende bronnen gehaald en in de staging database gezet. De structuur van de staging database lijkt voornamelijk op die van de bronsystemen. Tabellen worden een-op-een overgehaald van de bron naar de staging-omgeving. Het belangrijkste is om de gegevens zo efficiënt mogelijk uit de bronsystemen te halen.



Figuur 7.9 Architectuur met staging database

Vaak zijn er eisen met betrekking tot de tijden waarop de ETL de bronsystemen mag benaderen. De ETL heeft een nadelige impact op de performance van andere workloads die tegelijkertijd plaatsvinden op de bron. Je kunt dan denken aan bijvoorbeeld het maken van back-ups en het reorganiseren van indexen op de bronsystemen. Deze zaken vinden vaak net als de ETL 's nachts plaats. Deze werkzaamheden zorgen er (soms) voor dat je beperkt toegang krijgt tot de brondatabases (*limited window of opportunity*).

Met meer bronnen zou het ook nog eens zo kunnen zijn dat je één bron al vroeg in de avond moet benaderen, terwijl een andere bron pas later in de nacht beschikbaar is. In een 24/7-omgeving komt daar nog bij dat de ETL een zo min mogelijke impact moet hebben op het primaire proces. Een staging database helpt bij al deze overwegingen.
Tijdens het laden van het datawarehouse kan er altijd iets fout gaan. De vraag is dan: wat doe je als je er 's ochtends achter komt dat het datawarehouse niet (correct) geladen is? Moet je de gebruikers vertellen dat ze een hele dag met rapporten moeten werken gebaseerd op oude gegevens, of kun je de ETL opnieuw starten zodat in de loop van de ochtend alsnog up-to-date rapporten beschikbaar komen? Met de gegevens in staging is er een grote kans dat je de ETL kunt herstarten.

De voordelen van het gebruik van een staging database zijn:
- meer flexibiliteit;
- herstarten van de ETL is gemakkelijker;
- omgaan met verschillende databases en verschillende beschikbaarheid van databases wordt gemakkelijker;
- in staging kunnen bewerkingen uitgevoerd worden zoals uitzoeken welke gegevens gewijzigd zijn.

De ETL wordt nauwelijks complexer door het gebruik van een staging database. Daarom zien we dat deze architectuur veel gekozen wordt.

Twee staging databases

In figuur 7.10 zie je dat er nog een extra database is toegevoegd aan de architectuur, de landing database. De staging database in de architectuur van figuur 7.9 lijkt qua opzet voornamelijk op de bronsystemen. Dat houdt in dat de meeste transformaties om de gegevens te krijgen in de structuur van het datawarehouse plaatsvinden in de stap van staging naar datawarehouse. Dit kan invloed hebben op de laadsnelheid van het datawarehouse.



Figuur 7.10 Architectuur met twee staging databases

Bij een multinational met kantoren over de hele wereld is er altijd wel ergens een kantoor open. Vanaf dat kantoor worden corporate rapporten opgevraagd die gebaseerd zijn op het datawarehouse. Het datawarehouse is dus een 24/7-database

geworden. Dat houdt in dat de impact van het verversen geminimaliseerd moet worden.
Door een extra laag in te bouwen, kun je een groot deel van de tijdrovende transformaties doen tussen landing en staging database. De landing database lijkt nu qua ontwerp op de bronnen, de staging lijkt juist op het datawarehouse. Het vullen van het echte datawarehouse is nu het een-op-een overhalen van gegevens uit staging naar datawarehouse.
Een nog belangrijker aspect is het ontkoppelen van de bronsystemen en het datawarehouse. In veel implementaties zal de landing database geen database zijn, maar een fileshare waar operationele systemen exports kunnen plaatsen. Welke systemen dat zijn of hoe ze dat doen, maakt voor het datawarehouse niet uit. De ETL haalt nu de geëxporteerde gegevens op en zet ze in de staging database. Dat is wel een echte database met alle kracht en functionaliteit die daarbij hoort. In deze implementatie lijkt een landing zone op de sectie 'Ruwe Data' van het in hoofdstuk 5 besproken Data Lake.
De voorgaande discussie over wel of geen staging gaat min of meer uit van een op de Kimball-methode gebaseerd datawarehouse. Voor Inmon en Data Vault gelden echter grotendeels dezelfde argumenten. Een belangrijk verschil is wel dat een Data Vault nooit rechtstreeks gebruikt wordt. Er is nog een ETL-slag nodig van het datawarehouse naar de verschillende datamarts. Datzelfde geldt voor het gebruik van een Data Lake. De datamarts zijn de databases die gebruikt worden voor rapportages en analyses.

Fysieke architectuur

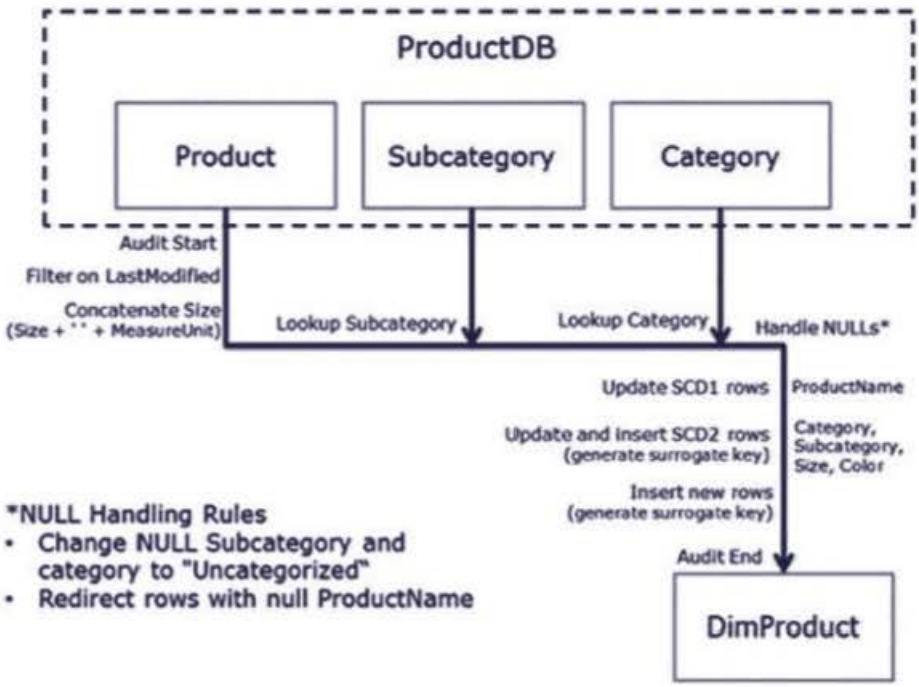
Bovenstaand verhaal schetst de logische architectuur. Je moet ook nog nadenken over de fysieke architectuur. Is elke laag ook echt een aparte database? En zo ja, staat elke database op een aparte server?
In sommige omgevingen, waaronder Azure-implementaties met Synapse SQL Pools, zie je nog wel eens dat wat hier de staging database wordt genoemd, gewone tabellen zijn in het datawarehouse. In het geval van twee staging databases kan het best zijn dat landing en staging andere tabellen in dezelfde database zijn. Grootte, belasting van de database en duurzaamheid van de inhoud bepalen wat wijsheid is.
Of verschillende databases op dezelfde server kunnen draaien of niet, hangt af van de overall workload van die server. Vaak zal staging overdag helemaal niets doen, waardoor het de performance van het datawarehouse niet in de weg zit. Er is dan niets mis met het combineren van beide op dezelfde server. Bij een 24/7-datawarehouse wordt dat natuurlijk anders.

7.3.3 Documenteren

Het ETL-proces is een geautomatiseerd proces. Er wordt software geschreven om de gegevens aan te passen en over te halen naar de volgende stap in de architectuur. Zowel logisch als technisch moet die software gedocumenteerd

worden. Logisch willen we weten welke bronnen er gebruikt worden en welke kolommen er gebruikt worden. Mapping-diagrammen en tabellen kunnen daarbij helpen.

Kolom-mapping



Figuur 7.11 ETL Mappingdiagram

Figuur 7.11 is een mappingdiagram dat weergeeft hoe de dimensie DimProduct gevuld wordt. Er zijn twee redenen om mappingdiagrammen te maken:

- om helder te krijgen welke brontabellen nodig zijn;
- om de ETL te documenteren.

Niet elke datawarehousekolom is een-op-een hetzelfde als een kolom uit een bronsysteem. Het is daarom van belang om zaken met betrekking tot validaties en transformaties op te nemen. Omdat het hier om een high level-diagram gaat, mist de detailinformatie. In een mappingtabel, vaak een Excel-document, kun je in detail aangeven welke kolom uit het bronsysteem op welke manier in het doelsysteem terechtkomt.

Figuur 7.12 is een voorbeeld van een Excel-spreadsheet met gedetailleerde mappinginformatie. Het hier getoonde workbook komt van Kimball en is te downloaden van zijn website. De eerste kolommen beschrijven de kolommen in het doelsysteem. Als dit het datawarehouse is, kunnen deze dienen als documentatie. De sheet van Kimball heeft zelfs een macro die de SQL DDL (Data Definition Language) genereert om de tabel te maken. Daartoe gebruikt de sheet de datatype-informatie die je kunt/moet opgeven. Ook informatie over foreign keys wordt opgenomen.

Table Name	BlankDimension	
Table Type	Dimension	
Display Name	Blank	
Database Schema		
Table Description	Always create a table description, which becomes a table extended property.	
Comment	You can put technical information here -- it doesn't get copied into the database in any way.	
Biz Filter Logic		
Size		
Generate Script?	N	

Home Page

									Target							Source			
Column Name	Display Name	Description	Unknown Member	Example Values	SCD Type	Display Folder	ETL Rules	Comments	Datatype	Size	Precision	Key?	FK To	NULL?	Default Value	Source System	Source Schema	Source Table	Source Fie
BlankDimKey	Blank Dim Key	Surrogate primary key	-1	1, 2, 3...	key				int			PK ID		N		Derived			
BKBlankDimID	Blank Dim ID	Business key from source system (aka natural key)	-1		key				nvarchar	10				N					
Attribute1	Attribute1	Always describe your columns	Unk Attribute1		1				nvarchar	50				N					
Attribute2	Attribute2	Always describe your columns	Unk Attribute2		2				nvarchar	50				N					
Attribute3	Attribute3	Always describe your columns	Unk Attribute3		2				nvarchar	50				N					
RowIsCurrent	Row Is Current	Is this the current row for this member (Y/N)?	Y	Y, N	n/a	Exclude from cube	Standard SCD-2		nchar	1				N		Derived			
RowStartDate	Row Start Date	When did this row become valid for this member?	1-1-1900	24-1-2011	n/a	Exclude from cube	Standard SCD-2		datetime					N		Derived			
RowEndDate	Row End Date	When did this row become invalid? (12/31/9999 if current row)	31-12-9999	1/14/1998, 12/31/9999	n/a	Exclude from cube	Standard SCD-2		datetime					N	31-12-9999	Derived			
RowChangeReason	Row Change Reason	Why did the row change last?	N/A		n/a	Exclude from cube	Standard SCD-2		nvarchar	200				N		Derived			
InsertAuditKey	InsertAuditKey	What process loaded this row?	-1		n/a	Exclude from cube	Standard Audit dim		int			FK	DimAudit.AuditKey	N		Derived			
UpdateAuditKey	UpdateAuditKey	What process most recently updated this row?	-1		n/a	Exclude from cube	Standard Audit dim		int			FK	DimAudit.AuditKey	N		Derived			

Figuur 7.12 ETL Mappingtabel

De mappingtabel bevat typische ETL-informatie. Per kolom kun je vermelden of het een Slowly Changing Dimension (SCD)-kolom is, en zo ja welk type. Verder kun je transformaties invullen. Welke berekeningen worden uitgevoerd op de gegevens alvorens ze te laden? Eventueel nodige datatypeconversies kunnen afgeleid worden uit de source en target datatypekolom. In de commentaarkolom kun je aangeven wat te doen als deze conversie niet lukt.
De resulterende Excel-sheet is tegelijkertijd documentatie en input voor de ETL-programmeur. Het enige dat nog moet worden bedacht is: met welke tools en met welke technieken gaan we de ETL implementeren? Daarvoor moeten de bronsystemen nog wat nader bekeken worden.

Bronnen

Er kunnen veel soorten bronnen zijn. Uiteraard zijn er de relationele databases waar een groot deel van de gegevens in zitten. Maar Big Data en NoSQL-databases komen ook steeds meer voor. Daarnaast kunnen gegevens komen uit tekstfiles, JSON-bestanden, REST API's en ga zo maar door. De tool die je kiest om het ETL-proces mee te bouwen, moet in staat zijn om deze verschillende bronsystemen te kunnen uitlezen.
Sommige van deze bronnen houden gegevens permanent vast. In andere bronnen kan data vluchtig zijn en is het weg als je te laat bent. Dat heeft uiteraard direct gevolgen voor de frequentie van de extractie.
Security is ook belangrijk. Met welke credentials ga je de bronnen benaderen? Mag je wel bij de benodigde gegevens?

7.3.4 Tools en technieken

De mappingtabel beschrijft de hele ETL. De volgende stap is een ETL-tool te kiezen. Vaak is er helemaal geen keuze omdat er al in een eerder stadium (een vorig project) voor een tool is gekozen. Soms kan die keuze nog wel worden gemaakt. Er zijn heel veel tools op de markt, allemaal met hun eigen goede en zwakke punten. Op www.etltools.org kun je een uitgebreide vergelijking kopen. Het gaat voor dit boek te ver om een vergelijking te doen. In paragraaf 7.5 zullen we een paar voorbeelden geven op basis van Microsoft SQL Server en SQL Server Integration Services.

SQL

Los van welke tool je kiest, is het nog wel de moeite waard om stil te staan bij de kracht van SQL. Daar waar gegevens uit relationele databases komen, kun je SQL gebruiken om de gegevens te manipuleren. SQL is een heel krachtige en efficiënte taal om gegevensmanipulatie mee te doen.

Er zijn drie argumenten bij het kiezen van de juiste tool, waarbij SQL een van de mogelijke tools is:

- benodigde functionaliteit: zowel pure ETL-functionaliteit als ook monitoring- en logging-functionaliteit;
- performance;
- onderhoudbaarheid.

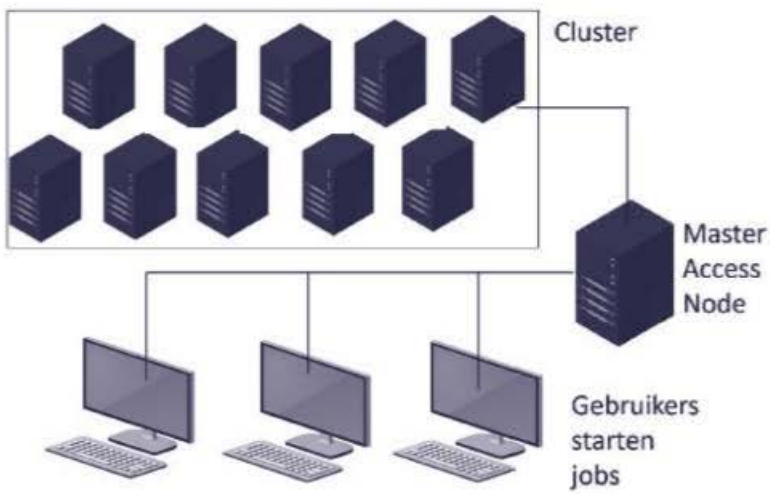
Een combinatie van een ETL-tool en SQL is vaak het beste. Wees je bewust van de sterke en zwakke kanten van beide en gebruik de mogelijkheden slim. Belangrijk is om consequent te zijn in de oplossingen en misschien wel vooraf standaarden af te spreken en vast te leggen. Documenteer vervolgens goed wanneer en waarom je van die standaard bent afgeweken.
Een kenmerk van Big Data is de variety en variability. SQL is voornamelijk geschikt voor gestructureerde gegevens. De taal Python is veel beter dan SQL in staat om te gaan met semi-gestructureerde gegevens. Python is daardoor de laatste jaren gegroeid in populariteit als taal om ETL-processen te programmeren.
In hoofdlijnen weet je nu wat ETL is en wat je moet doen en weten om een ETL-proces te kunnen implementeren. Wat nog nauwelijks aan bod is gekomen is de kwaliteit van gegevens en hoe ETL kan helpen om die kwaliteit te verbeteren. Daarom gaan we in de volgende paragraaf kijken naar het opschonen van gegevens om de gewenste kwaliteit te bereiken.

7.4 ETL, ELT en Big Data

In paragraaf 7.3 zijn de taal SQL en de tool SQL Server Integration Services genoemd als mogelijke middelen om ETL-processen te bouwen. Beide hebben als eigenschap dat ze goed kunnen omgaan met gestructureerde gegevens. We hebben echter geleerd in hoofdstuk 5 dat we steeds meer semi- en ongestructureerde gegevens hebben. Dit heeft voor de ETL aardig wat gevolgen.
Ten eerste wordt er in Big Data-architecturen vaak gebruikgemaakt van ELT in plaats van ETL. Het verschil zit hem in de omdraaiing van de letters en daarmee van de stappen. Bij Big Data wordt vaak data eerst geladen in een Big Data-platform zodat het daarna verwerkt, getransformeerd, kan worden. Kijk bijvoorbeeld naar de Data Lake-architectuur van figuur 5.2. De gegevens worden eerst in het Data Lake gezet in de sectie 'Ruwe Data'. Daarna worden ze aangepast en in de sectie 'Bewerkte Data' gezet. Eerst laden, dan transformeren.
De kenmerken van Big Data leiden ertoe dat ETL (of ELT) een veel zwaarder proces is geworden dan in klassieke datawarehousing. Door de volumes aan data hebben we veel meer kracht nodig om de gegevens binnen acceptabele tijd te kunnen verwerken. Maar omdat de gegevens minder gestructureerd zijn, zijn de toe te passen bewerkingen vaak ook nog complexer. Dat maakt dat we nog meer extra rekenkracht nodig hebben om de gegevens te verwerken. Big Data-databases maken daarom gebruik van clusters.

7.4.1 Big Data-clusters

Een cluster is een verzameling gekoppelde servers die samenwerken. Voor de gebruiker fungeren ze als een eenheid. Er is vaak één server, de master of de head node, waarmee een gebruiker connectie maakt. Achter de schermen wordt het werk echter verdeeld over verschillende nodes (servers) in het cluster.



Figuur 7.13 Big Data-cluster

Een **cluster** is een verzameling gekoppelde servers die samenwerken alsof ze één zijn.

Een goed cluster is lineair schaalbaar. Dat betekent dat als je een query twee keer zo snel wilt maken, je die query door twee keer zoveel servers moet laten verwerken. Net zo goed is dezelfde query op een twee keer zo grote dataset even snel als voorheen indien je hem weer over twee keer zo veel servers verdeelt.
Bij een cluster hoort ook een distributed file system. Zoals hierboven beschreven wordt het rekenwerk, de CPU-belasting, verdeeld over meer computers. Ook de opslag van bestanden moet verspreid over meer servers plaatsvinden. Als meer servers tegelijkertijd met een bestand gaan werken en dat bestand staat in zijn geheel op één harde schijf, ontstaat er een bottle neck. Het schaalt niet meer.
Een distributed file system laat gebruikers werken met bestanden zoals we dat gewend zijn. Eén bestand is één bestand. Achter de schermen wordt het bestand in kleine stukken geknipt en verspreid over meerdere servers opgeslagen. Als je het een beetje slim doet, kan elke server die een stukje van het bestand heeft, dat stukje verwerken als er query's gebruikmaken van het bestand.
Een van de bekendste voorbeelden van zo'n Big Data-oplossing is waarschijnlijk Hadoop. Hadoop is een opensourcesoftwareraamwerk waarmee computerclusters tot vele petabytes aan ongestructureerde data op commodity hardware kunnen verwerken (computerworld.nl). Belangrijk hier is het woord softwareraamwerk.



Hadoop is een opensource-raamwerk van software voor Big Data-toepassingen.

Het hart van Hadoop wordt gevormd door HDFS, MapReduce en Yarn. HDFS is het Hadoop Distribited File System. MapReduce is een programmeer-framework dat er automatisch voor zorgt dat werk verspreid wordt over meerdere nodes in het Hadoop-cluster. Yarn, ten slotte, is een resource manager die coördineert dat hardware resources efficiënt worden benut in multi-user-omgevingen. Dit zijn de belangrijkste kenmerken van een Big Data-omgeving.
Spark is een in-memory versie van Hadoop en tegenwoordig erg populair voor het runnen van ETL-processen.
Een Data Lake is in alle waarschijnlijkheid geïmplementeerd op een HDFS- of HDFS-compatible file system. De ETL-processen kunnen dan optimaal gebruikmaken van de rekenkracht van een cluster-implementatie.

7.4.2 Big Data ETL-tools

Zoals gezegd hebben we bij Big Data en Data Lakes veel rekenkracht nodig om ETL-processen uit te voeren vanwege de grote hoeveelheden gegevens. Door een cluster te gebruiken krijg je die rekenkracht. Maar het soort gegevens vereist ook meer complexe logica om ze te prepareren voor gebruik. De benodigde transformatie van de ETL zijn vaak complexer en hebben daarom meer functionaliteit nodig. Dit wordt niet altijd geboden door een taal als SQL of door 'oude' ETL-tools zoals SQL Server Integration Services.
Zoals gezegd is Spark een populair Big Data-platform om ETL-processen op te draaien. Spark biedt de mogelijkheid tot het gebruik van meer verschillende talen. SQL is nog steeds een optie. Maar Python is dat bijvoorbeeld ook. Python is een populaire opensource-taal die veel gebruikt wordt in de wereld van de data science en artificial intelligence. Ook daar zijn complexe gegevensmanipulaties nodig en ook daar is veel rekenkracht nodig om algoritmes door te rekenen. Python biedt meer flexibiliteit als het aankomt op gegevens manipuleren dan SQL. Dat is soms hard nodig gezien de variety en variability waarmee we te maken krijgen.
Uiteraard zijn er ook grafische Big Data ETL-tools beschikbaar op de markt. Data Flow als onderdeel van Azure Data Factory is daar een voorbeeld van.

Nu je veel weet over ETL-processen is het goed om nog eens te kijken naar datakwaliteit, een van de problemen waar ETL zich mee dient bezig te houden.

7.5 Datakwaliteit

7.5.1 Slechte gegevens

Als een van de redenen waarom rapporteren en analyseren op operationele databases niet goed werkt, is de kwaliteit van de gegevens in bronsystemen genoemd. Er zijn verschillende redenen dat de kwaliteit van de gegevens niet is wat het zou moeten zijn:

- Er zijn meer bronsystemen. Elk bronsysteem heeft zijn eigen versie van de waarheid.
- Situaties veranderen.
- Gegevens worden foutief en onvolledig ingevoerd.

Meer bronnen

Master Data Management kan een oplossing zijn als dimensiedata in meer bronnen voorkomen. Dat klinkt gemakkelijk, maar dan moet je nog wel zorgen dat de kwaliteit van de Master Data goed is. Zonder MDM-oplossing moet je bij het laden van dimensies iets met de kwaliteit van de gegevens doen.

Situaties veranderen

Naast het probleem van meerdere bronnen blijkt dat de gegevens zelf inherent slecht van kwaliteit zijn. Soms zijn gegevens niet meer waar. De contactpersoon bij een klant krijgt bijvoorbeeld een nieuwe baan zonder dat je dat weet, maar in de database staat hij nog altijd geregistreerd als contactpersoon van zijn oude bedrijf. Misschien dat een klant failliet is gegaan. De klant kan een preferred supplier-contract hebben afgesloten met een concurrent. Er kan van alles gebeuren buiten het zichtveld van een organisatie, dat ervoor zorgt dat de gegevens niet meer kloppen.

Foutieve invoer

Soms worden gegevens door mensen ingevuld en mensen maken fouten. Typ-fouten zijn niet te voorkomen. Maar ook als er procedures en best practices zijn met betrekking tot gegevensinvoer, zullen medewerkers er zich niet altijd aan houden. Je zou kunnen afspreken dat als er 'bv' in de naam van een klant voorkomt, je dat altijd opschrijft zonder puntjes en met kleine letters. Het is echter een kwestie van tijd voordat er namen voorkomen met B.V. erin, met puntjes en hoofdletters.

Bij webshops is het probleem nog veel groter. Veel mensen laten dummy-gegevens achter uit privacyoverwegingen. Anderen maken elke keer dat ze terugkeren naar de webshop een nieuw account aan. Je hebt zelf weinig tot geen controle over deze gegevens.

7.5.2 Data Cleansing

Voor correcte analyses zijn goede gegevens nodig. De gegevens zullen moeten worden opgeschoond. Voor het opschonen van gegevens worden de termen data cleansing, data cleaning en data scrubbing gebruikt. Volgens Wikipedia is data cleansing '*the process of detecting and correcting (or removing) corrupt or inaccurate records from a record set, table or database. The term refers to identifying incomplete, incorrect, inaccurate, irrelevant, etc. parts of the data and then replacing, modifying or deleting this dirty data.*'



Data cleansing is het opsporen en verbeteren of verwijderen van inconsistente en foutieve records uit een verzameling, tabel of database.

In paragraaf 3.5 zijn veel van de mogelijke problemen en de gevolgen van slechte gegevens al besproken. We gaan die hier niet herhalen. In de volgende paragraaf zullen we een concrete implementatie laten zien van hoe je de kwaliteit van gegevens kunt verbeteren. Gezien het bovenstaande is wel duidelijk dat het hier niet om een eenmalig iets gaat. Als je brongegevens eenmalig opschoont, is het een kwestie van tijd voordat de problemen weer terug zijn. Daarom moet het opschonen een regelmatig terugkerende routine worden. Het is misschien wel de belangrijkste stap van het ETL-proces.



Als je gegevens overhaalt naar een MDM-implementatie of naar datamarts, moet de kwaliteit goed zijn.

7.5.3 Voorbeeld met Microsoft DQS



Om een en ander concreet te maken zullen we een eenvoudig voorbeeld uitwerken. Je vindt dit voorbeeld weer online via www.boomstudent.nl.

7.6 Scenario's

In deze laatste paragraaf van dit hoofdstuk zullen we weer met voorbeelden gebaseerd op SQL Server proberen om het verhaal concreet te maken. In eerste instantie zullen we SQL Server 2016 Integration Services gebruiken om een dimensietabel in het datawarehouse te vullen met gegevens uit de brondatabase. Twee van de attributen zijn gedefinieerd als Slowly Changing Dimension type 2-attributen.
In deze paragraaf gaan we ervan uit dat je een server hebt met daarop Microsofts voorbeelddatabases AdventureWorks2014 en AdventureWorksDW2014. In ons geval staan die databases op een server genaamd SQL2016. Overal waar

je in voorbeelden, plaatjes of code SQL2016 (of Denali) ziet staan, moet je dat vervangen door de naam van jouw server. Omdat deze paragraaf weer op concrete voorbeelden ingaat die productspecifiek zijn, vind je de rest van de paragraaf op de website bij dit boek.



7.7 Tot slot

In dit hoofdstuk heb je in de breedste zin van het woord geleerd wat ETL is. In zijn meest basale vorm is het niets anders dan het kopiëren van gegevens van een bronsysteem naar een doelsysteem. In een breder perspectief is het ook het aanpassen van de gegevens aan de eisen van het doelsysteem. Herleidbaarheid, auditing en eventueel compliancy komen dan om de hoek kijken. Het aanpassen zelf kan technisch van aard zijn (bijvoorbeeld datatype-conversies) of meer inhoudelijk.
Datakwaliteit is belangrijk. Met gedistribueerde systemen is Master Data en Master Data Management een uitdaging. Wat is de bedrijfsbrede waarheid ten aanzien van onze klanten, producten, leveranciers et cetera? Daarnaast zit er veel 'rotzooi' in bronsystemen. Voor analyses geldt het aloude adagium: *garbage in, garbage out*. Je moet dus zorgen dat de input die de data-analisten krijgen (de gegevens uit het datawarehouse) geen garbage bevat maar kwalitatief goede gegevens.
Om dit alles te bereiken zijn er veel tools op de markt. Er zullen veel keuzes gemaakt moeten worden. Welke tools en welke combinatie van tools moeten gebruikt worden?
Als het je ten slotte is gelukt om een goed datawarehouse (of een goede datamart) te maken, qua design maar zeker ook qua inhoud, kunnen analisten goede analyses gaan doen. In hoofdstuk 8 gaan we kijken naar analytische modellen. Wat kunnen we nu uiteindelijk doen met al die mooie gegevens?

Op de website bij dit boek kun je terecht voor de oefenvragen bij hoofdstuk 7.




Business Chart - Visual
1,505
18,321

Semantische modellen

8

Business Intelligence is veel meer dan het bouwen van een datawarehouse. OLAP (Online Analytical Processing) of OLAP-kubussen zijn een traditionele component van BI. In het vorige hoofdstuk zijn ze al genoemd onder de noemer semantisch model. In dit hoofdstuk kun je lezen over OLAP, semantische modellen en kubussen. Wat is het en welke rol spelen ze?

8.1 Inleiding

Om goed, gemakkelijk en snel gegevens te kunnen analyseren zijn datamarts in de vorm van een sterschema uiteindelijk niet toereikend. Het sterschema is als het goed is intuïtief voor iemand die het gemodelleerde proces kent, voor de gebruiker dus. Hij herkent de dimensies als de grootheden waarmee hij dagelijks werkt en de feiten zijn de relevante getallen voor zijn rapporten. Waarom voldoet een datamart dan toch niet?

Laten we, om die vraag te beantwoorden, eerst eens kijken naar drie eisen die worden gesteld aan gespecialiseerde OLAP-databases.

- Een OLAP-database moet een conceptueel en intuïtief gegevensmodel opleveren.
- Een OLAP-database moet een heel goede performance hebben, ook bij grote datasets.
- Een OLAP-database moet (relatief) gemakkelijk voor analysedoeleinden nuttige berekeningen kunnen uitvoeren.

In de komende paragrafen komen deze aspecten nader aan bod.

8.1.1 Semantisch model

Hoewel zonet is gezegd dat een sterschema – als het goed is – intuïtief is, blijkt dat in de praktijk toch tegen te vallen. Een stermodel heeft de volgende tekortkomingen om het een intuïtieve bron voor eindgebruikers te laten zijn:

- Een stermodel is een technisch gegevensmodel met 'technische' kolommen als ProductKey en SourceKey.

- Een stermodel bevat voor eindgebruikers irrelevante technische kolommen om Slowly Changing Dimensions bij te houden zoals RowStartDate en RowEndDate.
- Een stermodel bevat auditing-gerelateerde kolommen.
- Kolomnamen sluiten niet aan bij het bedrijfsjargon.
- De tabelstructuur levert problemen op voor query-generatoren.
- Nuttige berekende kolommen en meetwaarden ontbreken.

Niet-relevante kolommen

Een stermodel is nodeloos complex, omdat er voor de analysedoeleinden allemaal overbodige technische kolommen in zitten. Deze complexiteit kan de acceptatie van de eindgebruikers in de weg staan.

Een semantisch model bevat alleen de voor de gebruikers relevante kolommen en entiteiten.

Kolomnamen

Naast kolommen die voornamelijk technisch van aard zijn, hebben kolommen vaak ook niet de goede namen. Stel bijvoorbeeld dat je van producten het gewicht in aantal gram bijhoudt. Je hebt dan waarschijnlijk een kolom met een naam als GewichtInGram. In relationele databases is het een best practice om geen spaties in namen te gebruiken. Spaties kunnen allerlei problemen opleveren (overigens voornamelijk als er slordig wordt geprogrammeerd). Daarom is besloten om de kolomnaam niet uit drie aparte woorden te laten bestaan, maar er één woord van te maken en met hoofdletters de oorspronkelijke woorden toch nog te benadrukken. Vanuit een databaseperspectief is dit zeker aanvaardbaar, maar niet vanuit een rapportageperspectief.
Rapporten moeten heldere, leesbare kolomnamen hebben, die ook nog eens consistent over alle rapportages heen gebruikt worden. Deze namen komen overeen met de taal van het bedrijf. Dat kan een spreektaal zijn, of het jargon van het specifieke bedrijf. Met namen als GewichtInGram moet de maker van rapporten de naam op het rapport gaan wijzigen. Iedereen zal dat op zijn eigen manier doen, wat de eenduidigheid niet ten goede zal komen.



Headers op rapporten moeten duidelijk zijn, in de taal van het bedrijf geschreven en eenduidig over alle rapporten heen.

Erger zou nog zijn geweest als de kolom Gewicht had geheten. Het is nu een kwestie van tijd dat er rapporten komen met kolommen *Gewicht*, *Gewicht in gram*, *Gewicht in KG*. Drie namen voor hetzelfde databaseveld met ook nog eens een verkeerde interpretatie.
In een semantisch model worden kolomnamen gebruikt zoals later de koptekst van kolommen op rapporten moeten zijn. Namen hoeven op rapporten niet meer te worden aangepast. De eenduidigheid van de rapportages wordt

beter. Bij een semantisch model is het eindresultaat altijd leidend bij het maken van het model. Technische argumenten spelen geen of een ondergeschikte rol bij het maken van een goed semantisch model.

De tabelstructuur levert problemen op voor query-generatoren

Een sterschema bestaat uit meer tabellen: de feitentabel met daaraan via foreign keys gelinkte dimensietabellen. Iemand die rapporten of analyses uitvoert met het sterschema als basis, moet wel op de hoogte zijn van hoe deze relaties lopen. Er zijn veel grafische query-tools op de markt die gebruikers in staat stellen om met drag-and-drop query's te schrijven. Vaak gaat dat automatisch genereren van query's goed. Maar als er meer dan één relatie is gedefinieerd tussen twee tabellen, wordt waarschijnlijk de verkeerde query gegenereerd. Ook als er geen relaties zijn gedefinieerd gaat het nogal eens fout. In die gevallen zal de gebruiker zelf inzicht moeten hebben in het technische aspect van de database.

Informatie die laat zien wat er echt is gebeurd, is vaak redelijk gemakkelijk te tonen. Bijvoorbeeld een lijst van alle landen met daarachter de in dat land gerealiseerde omzet in een bepaalde periode. Maar stel nu eens dat er landen zijn waar in de gekozen periode geen omzet was? Kun je ook een lijst maken van alle landen die in die periode geen omzet hadden? Dat kan een zeer relevante vraag zijn. In technische termen is dat het verschil tussen een inner en een outer join. Gebruikers hebben dus technische kennis van SQL nodig om dit soort vragen te beantwoorden.

In een semantisch model wordt vastgelegd hoe verschillende entiteiten uit de organisatie verband met elkaar houden. Deze informatie stelt query-tools in staat om altijd de juiste query's te schrijven zonder dat de gebruiker kennis van de onderliggende structuur of oplossing nodig heeft. Kennis van de bedrijfsprocessen is alles wat de gebruiker nodig heeft. De gebruiker schrijft zelf helemaal geen query's meer. De rapport- en analyse-tool genereert automatisch de juiste query's.

Berekende kolommen

Query-tools stellen gebruikers in staat om bestaande kolommen te selecteren. Soms moeten kolommen echter nog berekend worden. Om een simpel voorbeeld te noemen: in de database staat een prijs exclusief btw en op het rapport wil je de prijs inclusief btw tonen. Dit is natuurlijk een makkelijke berekening die in elke grafische tool ook gemakkelijk zal zijn. Maar de berekeningen kunnen veel complexer worden en het gebruik van moeilijke functies noodzakelijk maken. Nu moet een gebruiker opeens weten wat functies zijn en hoe hij ze moet gebruiken.

In een goed semantisch model zal bovenstaande niet nodig zijn. Met een goed model bedoelen we een model dat is gebaseerd op een gedegen informatieanalyse. Vooraf is (voor zover mogelijk) duidelijk geworden welke velden gebruikt gaan worden. In het technisch ontwerp is bepaald waar die velden gedefinieerd worden. Dat kan zijn als kolommen in de datamart, maar het kunnen ook berekende velden in het semantische model zijn. De complexe berekening met

functies zitten nu in het model, de gebruiker kan weer 'gewoon' de benodigde kolom selecteren.



Een **semantisch model** vertaalt een technische database-implementatie in een voor eindgebruikers leesbaar en begrijpelijk model dat bruikbaar is zonder over (veel) technische kennis te beschikken.

Belangrijker nog dan berekende kolommen zijn berekende meetwaarden. In veel rapportages zal een gebruiker bijvoorbeeld de omzet van een bepaalde periode willen vergelijken met de omzet in diezelfde periode een jaar geleden. Met periode kan hier weken, maanden, kwartalen of nog iets anders bedoeld worden. In een relationele datamart is dit lastig vast te leggen. Bij het gebruik van SQL-query's heeft de rapportmaker kennis van SQL nodig, is er kans op fouten en de performance zal potentieel in het geding komen. In een goede OLAP-database kunnen dit soort berekende waarden (relatief) gemakkelijk gedefinieerd worden. De rapportmaker hoeft nu enkel de al vooraf gedefinieerde berekende waarde te gebruiken.

Tot slot

We begonnen deze paragraaf met te stellen dat een OLAP-database een conceptueel en intuïtief gegevensmodel moet opleveren. Dat wordt bereikt door die database een semantische model te laten zijn.

Twee belangrijke criteria bepalen hoeveel meerwaarde een semantisch model heeft:

- het technische niveau van diegenen die rapporten gaan bouwen en die analyses gaan doen;
- hoe goed de gebruikte datamart aansluit bij het gebruik.

Als alleen BI-specialisten rapporten maken en analyses doen, dan mag worden aangenomen dat er geen technische beperkingen zijn bij deze mensen. Ze zullen altijd in staat zijn om de juiste query's te schrijven.

8.1.2 Performance

De tweede eis die we aan het begin van deze paragraaf hebben gesteld aan OLAP-databases is een goede performance. In dit boek hebben we het al veel gehad over performance. In eerdere hoofdstukken is benadrukt dat de acceptatie van de BI-oplossing door de eindgebruikers een kritische succesfactor is voor de oplossing. Als gebruikers het systeem (te) traag vinden, gaat het niet goedkomen met die acceptatie en dus niet met de BI-oplossing. Bij het gebruik van datamarts komt de performance op twee vlakken in het geding.

- Ten eerste is er de tijd die het duurt om de juiste query te schrijven.
- Ten tweede is er de tijd die het duurt om de query uit te voeren.

Een OLAP-database zou beide zorgen uit de weg moeten nemen.

Benodigde tijd om query te schrijven

Voorbeeld 8.1



Iemand zit met een vraag. Hij belt met de Business Intelligence-afdeling met het verzoek of zij een query voor hem kunnen schrijven. Dat kan bijvoorbeeld zijn omdat deze gebruiker niet de technische kennis heeft om de benodigde informatie zelf uit het systeem te halen. Dat kan ook zijn omdat de gebruiker niet de juiste rechten heeft om zelf de gegevens op te vragen. Of misschien omdat hij zelf geen zin heeft om achter de benodigde informatie aan te gaan.
Uiteraard kan de BI-afdeling aan de gestelde vraag voldoen. Vandaag zijn ze echter net bezig met een migratie naar een nieuwere versie van het DBMS, maar overmorgen zullen ze zeker de query gaan schrijven. In de communicatie ging echter iets fout waardoor initieel de verkeerde query is geschreven. Een dag later wordt dat rechtgezet.

In bovenstaand voorbeeld is nu drie en een halve dag verstreken tussen het ontstaan van de vraag en het krijgen van het antwoord. Met een beetje pech is de informatie al niet meer relevant.
Met een goed semantisch model zou een dergelijke situatie niet meer voor moeten kunnen komen. Belangrijk is weer de acceptatie. Die moet ervoor zorgen dat de derde reden waarom de gebruiker aan het BI-team vraagt om informatie (hij heeft geen zin om het zelf te doen) niet meer voorkomt. Het model moet dusdanig intuïtief zijn dat de gebruiker geen technische tekortkomingen tegenkomt. Ten slotte moet het model een security-implementatie bevatten zodanig dat de gebruiker alle voor hem relevante gegevens kan opvragen.

Query-performance

Nadat de benodigde query is geschreven is er natuurlijk nog de performance van de query zelf. Hoe lang duurt het om de query uit te voeren? In hoofdstuk 6 is een aantal 'trucs' besproken die kunnen helpen om de performance van een database positief te beïnvloeden. Voor het merendeel gingen deze trucs ervan uit dat precies bekend was welke query's werden gedraaid, om vervolgens die specifieke query's sneller te maken. Voor veelgebruikte rapporten is dat geen vreemde aanname. In een analyseomgeving gaat dat echter meestal niet op. Er kunnen willekeurige vragen gesteld worden en er zullen telkens weer andere vragen opduiken. Veelal zullen analyses beginnen op hoog aggregatieniveau. Dus grote datasets geaggregeerd tot een aantal getallen. In technische termen: dure group by's. In vervolgstappen is het niet onwaarschijnlijk dat er drill-downs uitgevoerd worden om informatie op lagere aggregatieniveaus te bekijken.

Als we kijken naar producten op de markt waarmee we semantische modellen kunnen maken, gaat er ten aanzien van performance een verschil optreden tussen producten die 'alleen' een semantisch model zijn, echte OLAP-databases, en de veel nieuwere in-memoryproducten. Er zijn veel producten op de markt die semantische modellen leveren. Bekende voorbeelden zijn de Universe van Business Objects en het framework van Cognos. Deze modellen moet je eigenlijk zien als SQL-query-generatoren. Ze bieden de gegevens via een intuïtief model aan de gebruikers aan. De gebruiker sleept een rapport bij elkaar en het model genereert de onderliggende SQL-query die wordt afgevuurd op de datamart.
Bij deze tools is performance juist vaak een probleem. De automatisch gegenereerde query's zijn bijna nooit heel efficiënt. En zelfs als ze dat wel zijn, is de beste performance die je kunt krijgen de performance die het DBMS te bieden heeft. Met zware aggregaties op grote, gejoinde tabellen, kan dat wel eens een probleem zijn. Uiteraard is dat een kwestie van hoe groot de database is, welk DBMS je gebruikt en welke query's je uitvoert.
Een OLAP-database moet een betere performance bieden door de gegevens zelf in een ander formaat op te slaan. De gegevens komen dan niet meer uit de (relationele) datamart maar rechtstreeks uit de OLAP-database. In paragraaf 8.2 kun je daar meer over lezen.
De laatste tijd komen er steeds meer in-memoryproducten. Met de huidige hardware kunnen grote datasets in het werkgeheugen van servers geladen worden. Klassieke databasesystemen en ook OLAP-databases gaan uit van de opslag van de gegevens op disk. Schijven zijn echter eindeloos veel trager dan geheugen. Door gegevens permanent in het geheugen te houden kan de bottleneck die vaak gevormd wordt door de IO (verplaatsen van gegevens van schijf naar geheugen en vice versa) geëlimineerd worden. Bovendien kunnen andere structuren gebruikt worden om nog eens extra voordeel te creëren. Voorbeelden van in-memoryproducten zijn SAP Hana, QlikView en Microsoft SQL Server Analysis Services Tabular Models. In paragraaf 8.3 kun je daar een voorbeeld van zien.
Met een OLAP-database of een in-memorydatabase kunnen performanceproblemen die bij datamarts een rol spelen voorkomen worden.

8.1.3 *Query-performance en Big Data*

Op dit punt is het goed om even terug te keren naar Big Data. Als de query-performance slecht is vanwege de grootte van de gebruikte datasets, moeten we dan geen Big Data-clusters gebruiken? Hoewel dat aannemelijk klinkt, is het antwoord nee.
Het doel van OLAP-databases is dat mensen interactief, realtime, 'spelen' met gegevens. Daar hoort bij dat je ad hoc andere dwarsdoorsnedes wilt maken, drill-downs of drill-throughs wilt doen, of andere meetwaarden wilt bekijken. Dat is niet de sterkte kant van Big Data-tools. Het is wel de kracht van kubussen.

Veel Big Data-architecturen maken een indeling van de gebruikte technieken in drie lagen. Deze lagen zijn:

- batch layer;
- speed layer;
- serving layer.

Batch layer

De meeste Big Data-tools werken nog met jobs. Jobs zijn stukken werk die zich goed laten schedulen. De doorlooptijd van deze jobs is vaak zodanig dat je er niet op wilt wachten. Bij kleine datasets praten we vaak al over minuten, bij grote datasets kan dat uren duren. Het is het handigst om deze jobs in de nacht te laten uitvoeren als niemand al duimendraaiend zit te wachten. We noemen dat batchverwerking. In een Big Data-architectuur vormen deze jobs de batch layer.

Speed layer

Om realtime analytics mogelijk te maken, kunnen we naast de batch layer ook een zogenaamde speed layer bouwen. Deze gebruikt vaak streamingtools zoals Apache Storm of Microsoft Azure Stream Analytics in combinatie met Event Hub of IoT-hub. Met deze tools kunnen we gegevens in realtime analyseren en de resultaten gebruiken.

Serving layer

De serving layer dient om de gegevens aan te bieden aan de gebruikers. Hij kan de resultaten van de batch en de speed layer combineren, maar dat is niet noodzakelijk. Een OLAP-database zoals een kubus kan heel goed fungeren als de serving layer. Sterker nog, dat is een van de belangrijkste doelen van een OLAP-database.

Met andere woorden: de batch layer zal de heavy lifting van veel Big Data-analyse en preparatie doen. Dit is met andere woorden de ETL-laag van de oplossing. Via de serving layer, ofwel via een OLAP-database of semantisch model, worden de resultaten beschikbaar gesteld aan de gebruikers.

8.1.4 OLAP query engine

De derde eis die we aan het begin van deze paragraaf hebben gesteld aan OLAP-databases is de capaciteit om berekende velden te maken. De mogelijkheden die je hier vindt, hangen sterk af van het product waar je mee werkt. Goede OLAP-databases hebben krachtige scripttalen om berekeningen te doen. De taal MDX hoort bij kubussen zoals SQL hoort bij relationele databases. DAX is de expressietaal die bijvoorbeeld Power BI gebruikt.

8.1.5 Tot slot

Hoewel sterschema's voor technici een intuïtieve omgeving vormen die de processen binnen de organisatie beschrijven, laten ze voor businessanalisten en rapportbouwers nog weleens te wensen over. Semantische modellen kunnen helpen om de problemen op te lossen. Modellen laten de beschikbare gegevens zien aan de gebruikers in hun eigen taal zonder technische randzaken. Met de juiste tools is nu alleen nog maar inhoudelijke kennis nodig om met de gegevens te werken. Technische kennis is niet meer nodig, die is al verwerkt in het model.
Ook de performance van sterschema's kan problemen opleveren. Producten die alleen een model zijn, blijven last houden van dat probleem. OLAP-databases (OLAP-kubussen) kunnen dat probleem oplossen. Naast kubussen zijn er steeds meer producten op de markt die de performance aanpakken door volledig vanuit het werkgeheugen van de server te werken, de in-memorymodellen of databases. In paragraaf 8.2 kun je meer lezen over kubussen en in paragraaf 8.3 werken we een voorbeeld uit van een in-memorymodel.

8.2 OLAP-kubus

Zoals je in de vorige paragraaf hebt kunnen lezen is een OLAP-database een oplossing om eindgebruikers met goede performance zelf analyses te laten uitvoeren. De eindgebruikers hebben weinig technische kennis nodig om dit te kunnen.
De analyses waar het hier om draait, zijn veelal multidimensionele analyses, waarbij vanuit verschillende invalshoeken (dimensies) wordt gekeken naar meetwaarden zoals omzet en kosten. Dit wordt ook wel '*to slice and dice*' genoemd. Willekeurig, telkens met een ander perspectief, door de gegevens browsen. Free form-analyses kunnen uitvoeren zonder vast te zitten aan vooraf vastgestelde vragen. Out-of-the-box kunnen denken en analyseren. Dat is wat analisten willen en dat is wat kubussen beloven. Maar wat is nu eigenlijk een OLAP-database en een kubus?

8.2.1 Wat is een kubus?

Over het algemeen zijn OLAP-database en OLAP-kubus, of kortweg kubus, synoniemen. Een derde naam is multidimensionele database. Deze naam benadrukt dat het gaat om een analysedatabase waarbij vanuit meerdere perspectieven naar de gegevens gekeken moet kunnen worden.
De term kubus is zowel heel goed als heel slecht gekozen. Goed, omdat het in eenvoudige omgevingen ruimtelijk inzichtelijk kan maken wat (ongeveer) bedoeld wordt met een multidimensionele database. Slecht, omdat een kubus

een driedimensionaal object is dat lang geen recht doet aan de complexiteit van de multidimensionele database die je wilt maken.



Figuur 8.1 OLAP-kubus

Figuur 8.1 laat zien wat een kubus is. Je ziet een kubus met twee dimensies en vier feiten. De feiten zijn Sales Amount, Sales Cost, Sales Qty en Profit Margin%. Deze feiten kunnen geanalyseerd worden per productcategorie en over de tijd, de twee dimensies. Een kubus is opgebouwd uit allemaal kleine kubussen. De kleinste kubus is een cel, een blokje dat één waarde kan bevatten. Op een van de assen van de kubus staan alle feiten. Op de andere assen staan de dimensies. Omdat het plaatje een driedimensionaal object is, blijven er twee assen over voor de dimensies. In figuur 8.1 zijn dat de dimensies Productcategorie (Product Categories) en Tijd (Time). In de praktijk kan een kubus vele dimensies hebben.
Op elke as van de kubus staan de zogenaamde members. Members zijn gegevens in de database. Blijkbaar is er in het voorbeeld sprake van productcategorieën Diesel, Premium Gas en Regular. Dat is de inhoud van de productdimensie in het datawarehouse.

Members in een kubus worden gevormd door de inhoud van een kolom in een datawarehouse.

Bij het bepalen van de hoeveelheid assen die een kubus heeft, moet je onderscheid maken tussen de assen en de dimensies. In hoofdstuk 4 over dimensioneel modelleren heb je geleerd dat er een vuistregel is die zegt dat een stermodel uit niet meer dan zeven dimensies moet bestaan. Dat vertaalt zich echter niet naar een kubus met zeven assen. Het zal een veelvoud daarvan zijn.

Stel dat je naast de naam van een product ook het merk van het product bijhoudt. Op de kubus zou dit tot een aparte as leiden die bevolkt wordt met alle verschillende merken die in de database voorkomen. Een kolom prijs leidt tot een as die bevolkt wordt met alle prijzen vanuit de database. Eigenlijk geldt dat voor elk attribuut van elke dimensie een as wordt gemaakt die bevolkt wordt door een SELECT DISTINCT op de bijbehorende kolom in de dimensietabel. Met zeven dimensies die elk bestaan uit twintig kolommen, hebben we het dus al over honderdveertig assen. Probeer dat maar eens in een plaatje te tekenen.

8.2.2 *Waarom kubussen?*

De mooiste beschrijving van wat een kubus is, is de volgende: *Excel on steroids*! Wat is nou een kubus en waarom zijn ze zo handig?
Jaren geleden deed Microsoft nog helemaal niets op het gebied van Business Intelligence, maar ze zagen wel aankomen dat Business Intelligence een grote markt ging worden die ze niet links konden laten liggen. Ze hebben toen laten uitzoeken wie de grote spelers op de BI-markt waren en welke producten die spelers maakten. Verrassenderwijs kwam daaruit naar voren dat Microsoft zelf marktleider was op het gebied van BI-software. En wel met Excel!
Business Intelligence was heel lang, en is bij sommige bedrijven nog steeds: export maken van gegevens uit een database en die gegevens analyseren met behulp van Excel. Kijk nog maar eens terug naar de beschrijving van de levels van volwassenheid zoals beschreven in hoofdstuk 2. Tegenwoordig heeft Microsoft die boodschap begrepen en Excel wordt op de markt gezet als belangrijke Business Intelligence front-endtool. Ook veel nieuwe features van Excel zijn gericht op BI.

De vraag rijst waarom Excel, zelfs toen Microsoft het nog niet wist, zo'n belangrijke BI-tool was. Daar zijn, naast de populariteit van Office en het wijdverbreid zijn van Excel, twee hoofdredenen voor aan te wijzen:
- de draaitabel;
- formules en functies.

Draaitabel

Een draaitabel (matrix, pivottable) is een handige en veelgebruikte manier om grote hoeveelheden gegevens eenvoudig inzichtelijk te maken. Zet de namen van alle verkopers op de verticale as, de maanden van het huidige jaar op de horizontale as en je hebt snel inzichtelijk welke verkoper het in welke maanden goed of slecht heeft gedaan. Potentieel liggen hier honderden, zo niet duizenden, records aan ten grondslag, toch is het inzicht snel tot stand gekomen.
In het boek *Business Intelligence – Making better decisions faster* (Vitt, Luckevich & Misner, 2002) staat het volgende voorbeeld. Op een gewone database worden drie query's afgevuurd die respectievelijk de omzet per kwartaal, per markt en per product laten zien. Onderstaand zie je het resultaat:

Periode	Omzet
Q1	€ 16.000
Q2	€ 16.000
Totaal	€ 32.000

Plaats	Omzet
Amsterdam	€ 8.000
Rotterdam	€ 8.000
Den Haag	€ 8.000
Utrecht	€ 8.000
Totaal	€ 32.000

Product	Omzet
Appels	€ 8.000
Kersen	€ 8.000
Druiven	€ 8.000
Meloen	€ 8.000
Totaal	€ 32.000

Figuur 8.2 Eendimensionale query's

Dergelijke gegevens zijn redelijk saai en leiden niet tot speciale inzichten. Onduidelijk is welke vraag we verder nog kunnen stellen. Totdat we bedenken dat we geen eendimensionale vragen moeten stellen, maar multidimensioneel moeten denken en het resultaat moeten weergeven in de vorm van een draaitabel. Die gegevens kunnen er ook uitzien zoals in figuur 8.3.

		Amsterdam	Rotterdam	Den Haag	Utrecht	Totaal
Q1	Appels	-	-	€ 2.500	€ 1.500	€ 4.000
	Kersen	-	-	€ 2.000	€ 2.000	€ 4.000
	Druiven	€ 1.000	€ 3.000	-	-	€ 4.000
	Meloen	€ 2.000	€ 2.000	-	-	€ 4.000
	Totaal	€ 3.000	€ 5.000	€ 4.500	€ 3.500	€ 16.000
Q2	Appels	€ 4.000	-	-	-	€ 4.000
	Kersen	€ 1.000	€ 3.000	-	-	€ 4.000
	Druiven	-	-	€ 1.500	€ 2.500	€ 4.000
	Meloen	-	-	€ 2.000	€ 2.000	€ 4.000
	Totaal	€ 5.000	€ 3.000	€ 3.500	€ 4.500	€ 16.000
Totaal		€ 8.000	€ 8.000	€ 8.000	€ 8.000	€ 32.000

Figuur 8.3 Multidimensionele draaitabel

Als je de gegevens die deze draaitabel bevat als gewone rijen zou weergeven, dan zijn dat minimaal vijftien rijen. Daar zitten dan de totalen niet bij. Bovendien zitten daar de records niet bij die in de draaitabellen de lege cellen representeren. En juist die lege cellen geven hier verrassende en belangrijke informatie. Hoewel alle steden en alle producten het even goed lijken te doen (bij de eendimensionale views staat de omzet overal op dezelfde achtduizend), blijken sommige producten in sommige steden helemaal niet verkocht te worden!
In de praktijk zijn de in de draaitabellen getoonde getallen aggregaties van vele rijen die corresponderen met individuele transacties. Uit een lijst van honderden rijen had je nooit het inzicht gehaald dat de draaitabel wel heeft gegeven.
Dit voorbeeld laat zien wat de kracht van draaitabellen is. Met Excel is het gemakkelijk om draaitabellen te maken. De kracht van het resultaat en de eenvoud van het maken verklaren het succes op dit gebied van Excel.

De stap naar een kubus is nu niet meer zo groot. Een draaitabel is eigenlijk een kleine kubus met maar twee assen. De werkelijkheid is complexer dan de getoonde draaitabel. We hebben niet alleen omzet, maar ook het aantal verkochte stuks fruit. Er moet dus een cel bij voor dit nieuwe feit. Kwartaal kan verder uitgesplitst worden naar maanden. Daar moet een as voor komen. Er is nog een dimensie verkoper waar een as voor moet worden gemaakt. In essentie blijft het hetzelfde, een draaitabel met op de assen members van dimensies (Appels, Kersen, Amsterdam, Utrecht, ...) en op de snijpunten (de cellen) de feiten. Dat is precies wat een kubus is. Een kubus is dus een draaitabel, maar dan in het groot: *on steroids*!

Functies en formules – MDX

Een tweede reden van het succes van Excel zijn de functies en formules. Die stellen de gebruiker in staat om relatief eenvoudig bewerkingen toe te passen op de gegevens. De tegenhanger daarvan in kubussen is MDX, Multi Dimensional eXpressions. Van MDX kan zeker gezegd worden dat het krachtig is, helaas wordt het door de meeste mensen niet ervaren als makkelijk, integendeel zelfs. MDX moet gezien worden als de tegenhanger van SQL voor kubussen. Wat SQL is voor relationele databases, is MDX voor kubussen. MDX is de query-taal van kubussen. Net zoals je SQL gebruikt om gegevens uit een relationele database te halen, gebruik je MDX-query's om gegevens uit een kubus te halen. Helaas is de syntax van beide talen wel anders. Dat kan ook niet anders. Een multidimensionele database is totaal anders van opzet dan een relationele database en dat heeft zijn weerslag op de bijbehorende taal.



Multi Dimensional eXpressions (MDX) is de taal die hoort bij OLAP-kubussen.

Zoals in paragraaf 8.1.3 is gezegd, heeft een OLAP-database een krachtige query engine nodig om met multidimensionele analyses om te kunnen gaan. Denk bijvoorbeeld eens aan een berekening van marge. Je hebt een kolom voor kosten en een kolom voor omzet. De marge in procenten is:
((omzet – kosten) / omzet) * 100%
In bijna alle programmeertalen is dit een eenvoudig te schrijven formule. Maar stel dat je in een SQL-query deze formule toepast en in de draaitabel van figuur 8.3 de omzet vervangt door marge. De resultaten zullen totaal verkeerd uitpakken!
Als de marge die je in Amsterdam haalt 10% bedraagt en de marge in Utrecht is 20%, dan is uiteraard de totale marge niet 10+20 is 30%. Was het maar waar! Je kunt ook niet zomaar middelen en zeggen dat de marge 15% is. Misschien is Amsterdam naar omzet gemeten wel veel kleiner dan Utrecht en draagt het dus minder bij aan de overall marge. Berekeningen in kubussen worden soms veel complexer, omdat je altijd rekening moet houden met de multidimensionele wereld van de kubus.

Los van dit voorbeeld waarbij marges niet op te tellen zijn, moet ook rekening gehouden worden met de grote hoeveelheid cellen waar een berekening op van toepassing is. Stel dat de draaitabel van figuur 8.3 de hele kubus omvat en de gegevens toont op het laagst beschikbare detailniveau. In totaal bestaat de kubus dan uit 32 cellen (2 kwartalen * 4 plaatsen * 4 producten). Veelal worden totalen in een kubus vooraf berekend en opgeslagen en daar zijn ook cellen voor nodig. Daarmee komen er nog eens 18 cellen bij, wat het totaal aantal cellen brengt op 50. Dat houdt in dat er ook 50 keer een marge berekend moet worden. En voor elk van die berekeningen geldt dat de juiste aggregaties op elkaar gedeeld moeten worden, anders kloppen de uitkomsten niet. Probeer je eens voor te stellen hoeveel berekeningen dat worden in een echte kubus met honderden producten, duizenden klanten en tientallen kenmerken voor zowel producten als klanten!
MDX en de OLAP query engine kennen deze problematiek. Als je de taal eenmaal kent, blijkt het alsnog redelijk eenvoudig om een berekend veld zoals marge te maken. Sommige berekeningen blijken in MDX zelfs veel makkelijker te maken, of met een veel betere performance, dan in SQL. Op die manier kan een kubus dus ook functioneel nog meerwaarde bieden ten opzichte van een relationele datamart.
Een kubus is dus een soort draaitabel met een krachtige functietaal. Het eerste kenmerk sluit goed aan bij het gebruik van gegevens, het tweede verrijkt de gegevens ten opzichte van het datawarehouse. Dit zijn twee belangrijke argumenten voor het gebruik van kubussen. Performance van kubussen is een derde reden.

Helaas is MDX minder een algemeen aanvaarde taal geworden dan SQL. We zien steeds meer dat producten op de markt een eigen functie- en/of scripttaal hebben. Zo gebruikt Microsoft in de nieuwere versie van Analysis Services niet meer MDX, maar DAX om berekende cellen te maken. DAX is eigen aan Microsoft. Ook Qlik heeft een eigen taal om berekende velden te maken in Qlik Sense en zo zijn er veel voorbeelden te noemen.
Een deel van de redenen dat iedereen zijn eigen taal gebruikt, is dat er nieuwe technologieën worden gebruikt. Wat een semantisch model is en wat het nut ervan is, is in de loop van de jaren niet veranderd. Echter, hoe producten dat geïmplementeerd hebben is technisch heel wezenlijk veranderd. Zozeer zelfs dat leveranciers hun product niet meer het 'stempel' kubus willen geven. Maar los van het woord 'kubus' is wat in dit hoofdstuk staat nog volledig actueel.

8.2.3 Aggregaties

In paragraaf 8.1 is een goede performance genoemd als eis die we stellen aan OLAP-databases. Er is een aantal redenen waarom kubussen veelal een betere performance bieden dan relationele databases. Dat zijn onder andere:
- een andere opslagstructuur;
- betere compressie;

- andere query engine;
- pre-aggregaties.

Zelden worden bij analyses de gegevens geraadpleegd op het laagst beschikbare detailniveau. In het voorbeeld van figuur 8.3 is de grain van de onderliggende feitentabel waarschijnlijk een record voor elke transactie waar een klant fruit koopt. De getoonde getallen zijn dus de omzet van alle klanten van alle dagen binnen het kwartaal bij elkaar opgeteld. Dat kan om duizenden records gaan. De onderliggende dataset is dus groot en dat kan op zichzelf al leiden tot slechte performance. Bovendien moeten vanuit die datasets nog de getoonde totalen berekend worden.
In paragraaf 6.4.4 (te vinden op de website bij het boek) zijn aggregatietabellen besproken om het hierboven beschreven performanceprobleem op te lossen. Bij kubussen doe je over het algemeen iets vergelijkbaars. Aggregaties worden vooraf (meestal 's nachts) berekend zodat alleen de gevraagde getallen hoeven te worden ingelezen als er een query wordt uitgevoerd. Er is dan geen sprake meer van een grote dataset (de onderliggende details worden ongemoeid gelaten) en er hoeft helemaal niets meer te worden berekend.
Een kubus gaat alleen veel slimmer met deze vooraf berekende aggregaties om dan een relationele database. Om dat uit te leggen moeten we eerst even kijken naar hiërarchieën.

Hiërarchieën

In veel analyses en rapportages wordt begonnen met het kijken naar gegevens op een hoog aggregatieniveau. Daarna kan worden ingezoomd naar gedetailleerdere levels. Neem bijvoorbeeld omzetten. Je begint te kijken naar jaaromzet om een jaar vervolgens uit te splitsen in vier kwartalen. Elk kwartaal kan weer verder worden uitgesplitst naar maanden en een maand kan weer worden opgesplitst in dagen. Je bent zo in een aantal stappen van hoog geaggregeerde informatie naar gedetailleerde informatie gegaan. De velden jaar, kwartaal, maand en dag vormen in deze volgorde een natuurlijke hiërarchie.
Er zijn veel voorbeelden te bedenken van natuurlijke hiërarchieën. Je kunt bijvoorbeeld omzet per land uitsplitsen naar omzet per provincie en verder naar per gemeente en per postcodegebiedje. Ook dit is een voorbeeld van een natuurlijke hiërarchie. Bij dit tweede voorbeeld moet je echter wel oppassen bij multinationals. Verschillende landen hebben een andere manier van het land verdelen in kleinere eenheden. De verschillende manieren van opsplitsen in verschillende landen hoeven niet overeen te komen. In Amerika hebben ze nog staten. Maar komt een staat in Amerika overeen met een provincie in Nederland?
Een natuurlijke hiërarchie is een drill-downpad waarvoor geldt dat er een een-op-veelrelatie bestaat van het hogere niveau naar lagere niveaus. Een jaar bestaat altijd uit meer kwartalen, maar één kwartaal is maar onderdeel van één specifiek jaar. Kwartaal 1 van 2021 bestaat uit drie verschillende maanden, maar januari 2021 is onderdeel van slechts één kwartaal, kwartaal 1 van 2021.

Een **natuurlijke hiërarchie** is een drill-downpad waarvoor geldt dat er een een-op-veel-relatie bestaat van het hogere niveau naar lagere niveaus.

In analyses is het ook gebruikelijk om niet-natuurlijke drill-downpaden te volgen. Je kunt bijvoorbeeld voor een kledingwinkel de omzet uitsplitsen per maat. Je zou vervolgens de omzet verder kunnen uitsplitsen naar kleur. Misschien is het zo dat kinderen voornamelijk felle kleuren kiezen terwijl volwassenen eerder kiezen voor neutrale kleuren. Als dat zo is, dan moet de afdeling inkoop dat natuurlijk wel weten. Dat maakt bovenstaande uitsplitsing dus zinvol. Tussen maat en kleur zit echter geen verband. Alle maten kunnen voorkomen in alle kleuren en alle kleuren kunnen voorkomen bij alle maten. Het drill-downpad van maat naar kleur wordt een niet-natuurlijke hiërarchie genoemd.

Voorbeeld 8.2

Kijk nog eens terug naar de draaitabel van figuur 8.3. Laten we voor het gemak er eens van uitgaan dat er per maand per stad duizend transacties plaatsvinden en dat elke transactie een record is in de database. Om de draaitabel van figuur 8.3 te maken moeten dan 6 maanden * 4 steden * 1000 records = 24.000 records gelezen worden. Daarna moeten deze records verwerkt worden door alle individuele omzetten op te tellen om tot de kwartaaltotalen per stad per fruitsoort te komen.
Stel nu eens dat de omzet per maand, per stad, per soort fruit al eerder berekend is. Dat betekent dat elke cel in de draaitabel berekend kan worden door slechts drie vooraf berekende maandtotalen uit de database te halen. Van 24.000 op te halen records gaat de query dan terug naar 32 cellen * 3 maanden = 96 records. Per cel hoeven nu nog maar drie getallen bij elkaar opgeteld te worden. Dit levert een enorme performance-winst op.

In deze berekening gaan we uit van de voorkennis dat de kalender een natuurlijke hiërarchie bevat. Ondanks dat het rapport kwartaalgegevens toont, kunnen we handig gebruikmaken van vooraf berekende aggregaties op een ander niveau, namelijk maand. Dat kan echter alleen met de kennis van hoe onze kalender in elkaar zit. Als je niet weet welke maanden samen een kwartaal vormen, valt dit betoog uiteen.
Ook als er geen natuurlijke hiërarchie bestaat, gaat de vlieger niet op. Stel dat je niet de maandtotalen maar weektotalen vooraf had berekend. Hoeveel winst had je dan behaald? Helaas blijkt dat weektotalen zinloos zijn voor dit rapport. Een kwartaal kan halverwege een week beginnen en/of eindigen en dus kun je vanuit weektotalen nooit met zekerheid zeggen wat het kwartaalresultaat is.
Waarom dit hele verhaal over aggregaties en hiërarchieën? Je kunt bij het ontwerpen van een kubus opgeven dat de gegevens natuurlijke hiërarchieën bevatten. Dat is mooi voor het model, want de gebruiker kan nu gemakkelijk in rapportages en analyses gebruikmaken van deze hiërarchieën door drill-down-rapporten te maken. Nog mooier is dat de query engine van een kubus in staat is om deze hiërarchieën te gebruiken zoals hierboven beschreven. Automatisch

zullen bij het verwerken van query's vooraf berekende aggregaties gebruikt worden waar dat mogelijk is. Het grote verschil met SQL is dat je bij een relationele database die kennis zelf moet hebben en moet verwerken in de query. Je moet de aggregatietabel gebruiken in plaats van de detailtabel. Bij MDX blijft de query precies hetzelfde. De kennis zit nu in de query engine, niet in het hoofd van de query-programmeur.
Hoe een en ander precies in zijn werk gaat hangt af van het product waar je mee werkt. Steeds meer producten gebruiken, zoals al eerder gezegd, het feit dat we veel goedkoop geheugen beschikbaar hebben door hele OLAP-databases in het geheugen van de computer te zetten. Vooraf berekenen van aggregaties is dan wellicht niet meer nodig. Bij grote gegevenssets zal vooraf aggregeren echter altijd performance-winst opleveren. Hoe dat exact werkt zal per tool verschillen.

8.2.4 Model of database?

Een kubus is een OLAP-database, ofwel een multidimensionele database. Microsoft noemt hun SQL Server Analysis Services-kubus tegenwoordig ook wel de UDM, Unified Dimensional Model. We noemen dat hier vanwege het woord model. Is het nu een model of een database of misschien wel beide?
Een kubus is in alle gevallen een model zoals bedoeld en beschreven in paragraaf 8.1. Bij het ontwerpen van de kubus moet de ontwerper altijd de eindgebruiker in gedachten hebben. Op basis van de informatieanalyse weet je welke rapporten er gewenst zijn in de organisatie, hoe kolommen moeten gaan heten en welke grootheden tegen elkaar uitgezet moeten worden. Dat alles komt samen in het model dat de kubus is. De interface van de kubus, het model, is volledig in de taal (het jargon) van de gebruiker. Er zijn geen technische naamgevingsconventies meer, geen technische of auditing-kolommen, geen inner en outer joins. Het model is een abstractielaag voor al die technische zaken.
Is een kubus naast een model ook een database? Eigenlijk is dat een keuze. In de praktijk wordt er onderscheid gemaakt tussen MOLAP en ROLAP en ook de varianten HOLAP en DOLAP komen voor.

MOLAP

MOLAP staat voor 'multi dimensional OLAP' en wordt vaak gezien als hetzelfde als OLAP. Bij MOLAP worden alle gegevens in de kubus zelf opgeslagen. Het is dus een echte database. De gegevens zijn opgeslagen in een relationeel datawarehouse én redundant ook nog eens in de kubus. Dat kost uiteraard opslagcapaciteit. Bovendien moeten de gegevens eens in de zoveel tijd (veelal elke nacht) vanuit het datawarehouse opnieuw in de kubus geladen worden. De informatie die je uit de kubus krijgt, loopt dus altijd een beetje achter. Het voordeel van MOLAP, de meestgebruikte vorm bij kubussen, is de goede performance die het levert.

Om Microsoft nogmaals als voorbeeld te gebruiken: Power BI heeft bij rapporteren op een relationele datamart twee manieren van werken: Import en DirectQuery. Het algemene MOLAP komt overeen met import: Power BI leest bij de opslagmodus Import alle gegevens in-memory in. Deze gegevens moeten eens in de zoveel tijd ververst worden. Met import laat Power BI de beste performance zien.

ROLAP

ROLAP staat voor relational OLAP. De gegevens zitten in een onderliggende relationele database (waarschijnlijk de datamart). De kubus is nu niets anders dan een semantisch model. Alle query's die worden uitgevoerd op de kubus, worden on-the-fly vertaald naar SQL statements die worden uitgevoerd op de datamart. Er is geen sprake van gegevensredundantie maar dat gaat ten koste van de performance. ROLAP heeft een slechtere performance dan MOLAP.
Sommige mensen zeggen dat ROLAP staat voor realtime OLAP. Een nadeel van MOLAP is dat de kubus op basis van een schema ververst moet worden. Het duurt daarmee enige tijd (vaak een dag) totdat wijzigingen ook zichtbaar worden in de kubus. Als de kubus bij alle gebruik zijn gegevens uit een onderliggende database haalt, worden wijzigingen in die onderliggende database dus meteen zichtbaar in de kubus; realtime.
ROLAP is hetzelfde als wat in Power BI DirectQuery wordt genoemd. In Power BI maak je een model, de gegevens blijven in de achterliggende datamart en worden niet in het geheugen van Power BI opgeslagen. Dit stelt je in staat om met grotere datasets te werken, maar de performance van Power BI wordt iets slechter.

HOLAP en DOLAP

HOLAP staat voor Hybrid OLAP, een mengvorm van MOLAP en ROLAP. Vooraf berekende aggregaties worden opgeslagen in de kubus (MOLAP) maar de detailgegevens blijven achter in de datamart (ROLAP). DOLAP staat voor Desktop OLAP en representeert technieken om de informatie offline beschikbaar te maken zodat bijvoorbeeld thuis met de laptop ook gewerkt kan worden. Ook deze vorm kent Power BI. In Power BI kun je een aggregatietabel maken gebaseerd op een grote tabel die de modus DirectQuery gebruikt. Dimensietabellen krijgen de modus Dual. Dat is een hybride vorm: soms Import en soms DirectQuery.

Dat in bovenstaande Power BI wordt gebruikt als voorbeeld komt door de ervaring van de auteur. Uiteraard hadden andere producten kunnen worden gebruikt als voorbeeld. Per tool moet gekeken worden naar de mogelijkheden. Deze mogelijkheden moeten uiteraard al een rol spelen bij de toolselectie.

8.2.5 Tot slot

Samengevat heeft een kubus drie voordelen te bieden:
- Een kubus is een semantisch model.
- Een kubus heeft een krachtige rekentaal: MDX (of iets vergelijkbaars).
- Een kubus heeft goede performance.

De interface van een kubus vormt een semantisch model van de bedrijfsprocessen. Met de juiste front-endtools kunnen gebruikers die geen technische kennis hebben maar wel verstand van de processen hebben, goed, gemakkelijk en snel analyses doen op hun gegevens.
Ontwikkelaars kunnen door de kracht van de OLAP query engine en de taal MDX meerwaarde toevoegen aan de gegevens ten opzichte van diezelfde gegevens in een relationele database. MDX heeft meer relevante statistische mogelijkheden dan SQL.
Een kubus kan een betere performance bieden dan de meeste relationele databases. Het doel is willekeurige analyses op de gegevens te doen (*to slice and dice*) en direct antwoorden krijgen (*answers at the speed of thought*). Of de praktijk zo mooi is, is de vraag, maar een kubus komt hiermee een stuk verder dan een relationele database.

8.3 Voorbeeld in-memorymodel

Kubussen bieden veel functionaliteit en worden veel gebruikt in huidige Business Intelligence-omgevingen. In de praktijk blijkt dat het vaak moeilijk is om goede kubussen te bouwen. Daardoor worden BI-projecten duur en komt de gewenste informatie misschien niet op het goede moment. Bovendien werkt de kubus met MDX als vraagtaal, wat met regelmaat een te hoge drempel opwerpt.
Met de toegenomen geheugencapaciteit van servers en het toegenomen aantal cores in servers, komen er steeds meer producten die dezelfde soort mogelijkheden bieden als kubussen, maar dan met eenvoudiger techniek. De gewenste performance wordt gehaald door alles in-memory, dus in het werkgeheugen van de server te doen. Er wordt afgestapt van het idee dat een database bestaat uit bestanden op harde schijven waar gegevens inzitten. De hele database, dus de volledige dataset, zit in het geheugen.
Net zoals in de vorige hoofdstukken gaan we een en ander concreet maken door een voorbeeld uit te werken. Ditmaal niet met SQL Server maar met Excel 2016. Tegelijkertijd maken we nog wel gebruik van de SQL Server-database AdventureWorksDW2014 die we ook in andere hoofdstukken hebben gebruikt.



Lees de rest van de paragraaf op de website bij dit boek.

8.4 Tot slot

Een voordeel van Excel in combinatie met een model zoals je dat in de vorige paragraaf hebt gezien, is de analytische kracht van de combinatie. In plaats van dat de gebruiker een vrij statische lijst met gegevens krijgt, is er een model dat willekeurig naar eigen inzicht en ad hoc gebruikt kan worden. Een gebruiker kan de winst per product bekijken. Hij kan deze winst uitsplitsen naar de kleur van het betreffende product en in een vervolgstap kijken of de maand van invloed is op de kleur. Het maakt niet uit of deze vraag (is maand van invloed op welke kleur producten verkocht worden?) in de informatieanalyse zo letterlijk naar voren is gekomen.

Hiermee is de stap gemaakt van rapporteren naar analyseren. Rapporteren is vrij statisch. De aandeelhouders willen maandelijks inzicht in kosten en opbrengsten. Daarvoor zijn grootboekposten afgesproken en in een rapport dat elke maand hetzelfde is (behalve de inhoud natuurlijk) wordt de stand van zaken gerapporteerd. Omdat dit elke keer hetzelfde is, kan het tijdens een informatieanalyse goed beschreven worden. Alle eisen zijn duidelijk en de structuur en inhoud zijn duidelijk.

Er is echter veel meer te doen met alle informatie die een bedrijf tot zijn beschikking heeft. Sales en marketing zijn geïnteresseerd in wie welke producten koopt. Wat koopt een klant nog meer? Zijn er cross-sellingmogelijkheden? Zijn er trends te ontdekken die ons in staat stellen om eerder en beter in te spelen op de vraag vanuit de markt? Trends die in traditionele rapportages pas achteraf, en dus te laat, zichtbaar worden? Bij dit soort vragen is het doel van de vragen duidelijk, maar op voorhand zijn de gestelde vragen zelf niet duidelijk. De data-analist moet met de gegevens kunnen spelen. Dat is wat kubussen en in-memorymodellen brengen.

In de hoofdstukken 3 tot en met 7 lag de nadruk op het verzamelen van de juiste gegevens om die vervolgens met de juiste kwaliteit in de juiste structuur op te slaan. In dit hoofdstuk lag de focus echt op het gebruik van de gegevens. Hoe kijkt de gebruiker ertegenaan? Hoe moeten we de gegevens presenteren naar die gebruiker? Toepassen van de kleine BI-cyclus is makkelijker met goede semantische modellen.

Maar het echte presenteren, de communicatie uit analytics, hebben we nog niet gedaan. In het volgende hoofdstuk ga je kijken naar het eindresultaat. Wat zien de gebruikers van Business Intelligence? Hoofdstuk 9 gaat over het front-end.

Op de website bij dit boek kun je terecht voor de oefenvragen bij hoofdstuk 8.





Het front-end 9

Tot nu toe is veel van wat in dit boek is beschreven niet zichtbaar voor de mensen die aan Business Intelligence moeten doen. Of beter gezegd, de mensen die op basis van informatie beslissingen moeten nemen. Goed geïnformeerde beslissingen nemen en daardoor voordeel behalen is immers het doel van Business Intelligence. Dit hoofdstuk gaat over het presenteren van de gegevens als informatie aan de eindgebruikers (de information workers).

9.1 Inleiding

Je zou kunnen zeggen dat je zonder goed fundament geen huis kunt bouwen. Maar andersom ook dat je zonder huis niets hebt aan een fundament. Een architect ontwerpt eerst het huis, alvorens over het benodigde fundament na te denken.

Het huis in deze metafoor is wat we zien en wat we gebruiken. Dat zijn de rapportages, dashboards, scorecards en andere 'tastbare' zaken van een Business Intelligence-oplossing. We noemen dit ook wel het front-end. Het fundament is het datawarehouse, het Data Lake en/of de modellen die je ter beschikking stelt aan de makers van het front-end, inclusief de ETL die voor de juiste vulling van de datamarts zorgt.

Dit hoofdstuk gaat over het front-end. Dat wat de gebruikers straks zien en/of moeten gebruiken. Afhankelijk van je invalshoek is dit hoofdstuk daarmee het belangrijkste van het boek en had het misschien wel voorin moeten staan in plaats van het laatste hoofdstuk te zijn. In ieder geval moet je het front-end al in gedachten hebben gehad bij het ontwikkelen van al het andere. Vanuit het oogpunt van de bouwvakker beschrijft dit hoofdstuk de laatste stap. Nu het fundament ligt, kunnen we gaan bouwen aan het zichtbare gedeelte van onze oplossing.

Houd bij bovenstaande wel voor ogen dat vanuit een projectperspectief een incrementele aanpak meestal de voorkeur verdient. Dat houdt in dat je een stukje datawarehouse ontwikkelt en vult en daar dan een front-end bij bouwt. Daarna breid je het datawarehouse uit, waarna er weer nieuwe rapportages gemaakt kunnen worden. Een rapport kan een mooie deliverable zijn in een scrumsprint.

Dit hoofdstuk gaat naast het visuele front-end ook over wat je kunt met de gegevens die via datawarehouses, Data Lakes en semantische modellen beschikbaar zijn. Met dat laatste gaan we beginnen.

9.2 Analytics

In hoofdstuk 2 is besproken dat Business Intelligence een werkwijze is. Daarbij zijn de grote en de kleine BI-cyclus ter sprake gekomen. Alle technische zaken zoals datawarehousing, ETL en semantische modellen die daarna zijn besproken zijn slechts bedoeld om deze werkwijze te ondersteunen. Mensen in een organisatie hebben de juiste gegevens in het juiste formaat nodig om daadwerkelijk data-driven te gaan werken en zo (uiteindelijk) de intelligente organisatie te gaan vormen. Het daadwerkelijk gebruiken van de gegevens is wat we Analytics noemen. Data Analytics is het analyseren van gegevensverzamelingen om besluitvorming te ondersteunen en theorieën te testen. Deze definitie lijkt erg op de definitie van Business Intelligence.



Data Analytics is het analyseren van gegevensverzamelingen om besluitvorming te ondersteunen en theorieën te testen.

Misschien trapt het boek een beetje in de valkuil die enkele keren werd benoemd: de hoofdstukken 3 tot en met 8 richten zich erg op het bouwen van IT-oplossingen. Het ging over de technieken. Het draait echter om het nemen van besluiten. Daarom draait het om Data Analytics. In een datawarehouse worden interessante gegevens verzameld, maar het gaat erom wat je met die gegevens doet. Het werken met die gegevens is het veld van de Analytics. De hoofdstukken 3 tot en met 8 zijn natuurlijk niet voor niets geschreven. De ervaring leert dat door het opzetten van datawarehouses en daaraan verwante technieken het proces van Data Analytics het best wordt gefaciliteerd.
Bij Analytics hoort ook het communiceren van de ontdekte patronen en van het verkregen inzicht. Kijk nog maar eens terug naar de stappen van de kleine BI-cyclus in hoofdstuk 2.
Analytics is daarmee een methode, een werkwijze, waar data-analyse een onderdeel van is. Hierin zie je weer terug dat Business Intelligence meer is dan een datawarehouse. Het is een werkwijze die een onderdeel van de bedrijfscultuur en van de organisatie moet zijn.



Analytics is een werkwijze die een onderdeel moet zijn van de bedrijfscultuur.

Een datawarehouse of Data Lake, samen met technieken en tools om analyses uit te voeren, al dan niet op Big Data, is niet genoeg. Als je terugdenkt aan de Business Intelligence Maturity Matrix, hoort wat mensen doen met de geboden

informatie bij de kenmerken van een volwassen organisatie. Het proactief toepassen van informatie en aansturing door de CIO worden expliciet genoemd. Als dat er is, kun je spreken van Analytics. Vaak wordt onderscheid gemaakt tussen Basic Analytics en Advanced Analytics. We nemen beide hier wat verder onder de loep.

9.2.1 Basic Analytics

Wat precies onder Basic en Advanced Analytics wordt verstaan, verschilt per situatie. In dit boek definiëren we Basic Analytics als bestaand uit:

- Descriptive (Beschrijvende) Analytics;
- Diagnostic (Diagnostische) Analytics.

Descriptive Analytics

Descriptive, ofwel beschrijvende, Analytics doet eigenlijk precies wat de naam suggereert. Het beschrijft een situatie of een proces. We doen dat meestal door ruwe gegevens te aggregeren. De som van alle orders beschrijft de gehaalde omzet. We tellen hoeveel verschillende klanten we hebben gehad in een periode. We berekenen de gemiddelde marge en tonen de gemaakte winst.

De grootste beperking is dat we op deze manier altijd het verleden beschrijven. Je gebruikt immers gegevens van al geplaatste orders, al verstuurde facturen, al behandelde patiënten, et cetera. De gegevens zijn de registratie van al uitgevoerd werk. Descriptive Analytics is terugkijken in het verleden.

Descriptive Analytics is niet alleen terugkijken in de tijd, het is ook vrij statisch. Vaak gaat Descriptive Analytics bijvoorbeeld concreet om een lijstje van getallen dat een proces beschrijft. Je kunt bijvoorbeeld denken aan een lijst met de tien best verkochte producten in het afgelopen jaar of de tien meest voorkomende soorten criminaliteit in de afgelopen zes maanden.

Descriptive Analytics is over het algemeen kwantitatief. Met eenvoudige statistische berekeningen (analyses) zoals gemiddeldes en sommaties beschrijven we met getallen wat er is gebeurd. Getallen zijn objectief en gemakkelijk vergelijkbaar.

Een dimensioneel gemodelleerde datamart zet gegevens klaar in een ideaal formaat voor Descriptive Analytics. De complexiteit wordt erg laag. Met eenvoudige rechttoe rechtaan query's kunnen de getallen berekend worden. Met elke rapportagetool kunnen de resultaten gecommuniceerd worden met alle betrokkenen.

Diagnostic Analytics

Diagnostic Analytics probeert een stap verder te zetten dan Descriptive Analytics. Het probeert de vraag 'Waarom is iets gebeurd?' te beantwoorden. We zijn dus niet tevreden met een getal dat de omzet laat zien of met hoeveel de omzet is gegroeid of gedaald. We willen weten waarom we zoveel groei hebben laten zien.

Voor een deel komt hier misschien de ervaring van een persoon bij kijken. Maar die persoon zal ook vaak een verdere analyse op de gegevens willen doen. Hij zal zaken als drill-down en drill-through (zie paragraaf 9.5.2) willen doen en verder willen kijken naar bijvoorbeeld individuele orders. Hij wil bijvoorbeeld weten wat voor soort klanten welke orders hebben besteld en of er een verband bestaat tussen bestelde producten en het weer. Was het een goede of een slechte zomer? Voor een ijsverkoper kan dat van wezenlijk belang zijn in de verklaring waarom een jaar qua omzet goed of slecht was.
Dus waar Descriptive Analytics bijvoorbeeld laat zien dat we vorig jaar in juli 10 procent meer omzet hebben gedraaid dan dit jaar, zegt Diagnostic Analytics dat dat komt omdat we vorig jaar een warmere zomer hadden dan normaal. Dat levert meer meerwaarde op, maar is ook complexer.
Klassieke datawarehouses en dimensioneel gemodelleerde datamarts zijn een goede ondersteuning voor Basic Analytics.

9.2.2 Advanced Analytics

Met het ontstaan van Big Data en de toegenomen rekencapaciteit van computers is het tegenwoordig voor veel bedrijven economisch haalbaar om verder te gaan in hun analyses dan de zojuist beschreven Basic Analytics. Bedrijven beginnen andere vragen te stellen. We begeven ons dan op het terrein van de Advanced Analytics. Advanced Analytics is onder te verdelen in:

- Predictive (Voorspellende) Analytics;
- Prescriptive (Voorschrijvende) Analytics.

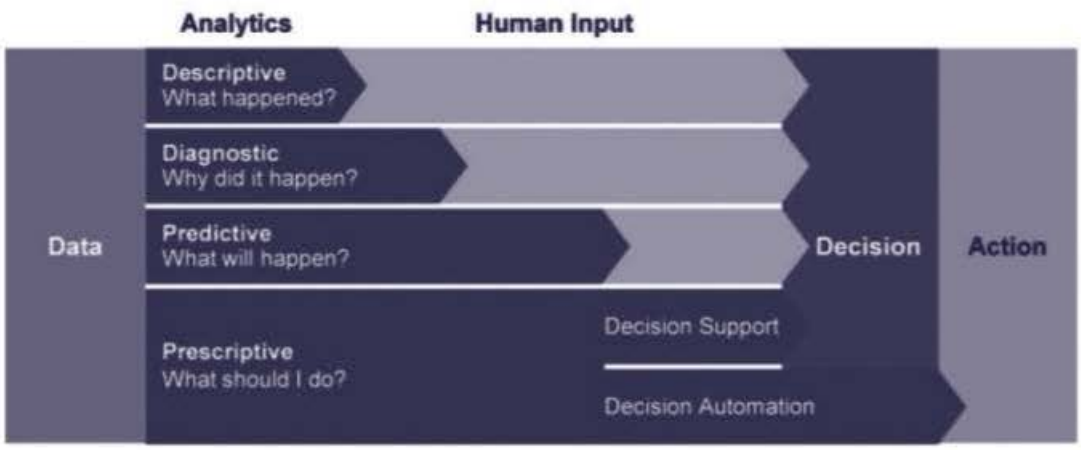
Predictive Analytics

Denkend aan de ijsverkoper van de vorige paragraaf is het eigenlijk helemaal niet zo interessant om te weten hoeveel omzet hij gisteren heeft gehaald. En ook niet of dat komt door het goede weer, doordat het vakantie is of dat er een andere reden is voor de goede dag. Veel interessanter is het om te weten hoeveel omzet hij morgen gaat halen. Business Intelligence gaat immers over beslissingen nemen. Vandaag moet de ijsverkoper beslissen hoeveel ingrediënten hij moet inkopen en hoeveel ijs hij daarvan gaat maken. Ook moet hij vandaag al de mensen optrommelen die morgen het ijs moeten gaan verkopen. Er moeten besluiten genomen worden die invloed hebben op hoe morgen zal verlopen. Te veel ijs maken en te veel mensen in de winkel hebben staan, is geld weggooien. Maar met te weinig ijs loop je omzet mis.
Verder in de toekomst kijken dan een dag of een paar dagen kan voor de ijsverkoper ook zinvol zijn. Met die kennis kan het bijvoorbeeld een stuk makkelijker worden om bij een bank een lening te krijgen om te investeren in een nieuwe ijsmachine. Resultaten uit het verleden zeggen niets over de toekomst. Laten we proberen de toekomst te voorspellen.

Enige nuance is hier natuurlijk wel op zijn plaats. De toekomst voorspellen bestaat niet. Het weerbericht is ook een weersverwachting, niet een weersvoorspelling. Met statistische modellen kunnen we berekenen met welke *waarschijnlijkheid* iets zou kunnen gaan gebeuren.
Met Predictive Analytics komen we uit bij datamining en machine learning. Om deze technieken goed toe te passen is extra, gespecialiseerde, kennis nodig. Data scientists hebben die expertise. Ze hebben gespecialiseerde tools en programmeertalen en vooral goede gegevensverzamelingen nodig om hun werk goed te doen. Klassieke BI-oplossingen voldoen vaak niet aan de eisen die data scientists stellen.

Prescriptive Analytics

Als we nog eens goed nadenken over de ijsverkoper, dan is het nu al weten wat morgen de omzet gaat worden nog altijd niet echt interessant. We leven immers pas nu. Wat moeten we nu doen? We moeten, zoals gezegd, nu twee besluiten nemen: hoeveel ingrediënten gaan we inkopen en moeten we het uitzendbureau bellen om extra mensen voor achter de balie in te huren? Uiteraard volgen de antwoorden uit de voorspelling hoeveel ijs je gaat verkopen of potentieel zou kunnen verkopen. Je moet wel genoeg ijs hebben om te verkopen en genoeg mensen hebben om de klanten te bedienen. Prescriptive Analytics schrijft voor wat je nu moet doen.
Hiermee zijn we terug bij zowel de definitie van BI als bij die van Analytics: het gaat om het nemen van beslissingen. Figuur 9.1 laat dat nog eens zien.



Figuur 9.1 Analytics (gartner.com)

Figuur 9.1 laat zien dat het draait om beslissingen en de acties die daaruit volgen. Belangrijk hier is de human input. Analytics communiceert de bevindingen van de data-analyse. Dat moet leiden tot het nemen van beslissingen. Maar bij Descriptive Analytics zit er nog een aardig gat tussen een rapport en het nemen van een beslissing. Bij Prescriptive Analytics is dat gat helemaal weg. Als je in staat bent om accuraat te voorspellen hoeveel ingrediënten je nodig hebt om voor morgen ijs te maken, kun je geautomatiseerd een bestelling naar de groothandel sturen!

Het voordeel van minder mensenwerk is dat er minder fouten gemaakt zullen worden en dat beslissingen sneller en accurater genomen kunnen worden.

9.2.3 Top-down versus bottom-up

Er is al enkele keren gezegd dat Advanced Analytics minder goed ondersteund wordt door klassieke BI en datawarehouses dan wat we in dit boek Basic Analytics noemen. Klassieke BI gebruikt een top-downbenadering. Voor de Predictive en Prescriptive Analytics heb je een bottom-upaanpak nodig. In figuur 9.2 wordt dit getoond.

Top-down

In een top-downbenadering ga je uit van een bestaande theorie. Vertaald naar Business Intelligence betekent dit dat je uitgaat van je kennis van het proces waar je aan werkt. Denk eens terug aan dimensioneel modelleren uit hoofdstuk 4. Je neemt een bestaand proces als uitgangspunt en dat modelleer je. In dat model stop je als het ware je kennis van het systeem.

Vervolgens maak je een hypothese over iets wat je waarneemt of iets wat je wilt onderzoeken. De ijsverkoper van eerder neemt bijvoorbeeld waar dat hij 10 procent meer omzet heeft gehaald in juli dan vorig jaar. Zijn aanname, hypothese, is dat het warme weer daar de oorzaak van is. Hij bedenkt dus vooraf eerst een mogelijke verklaring.

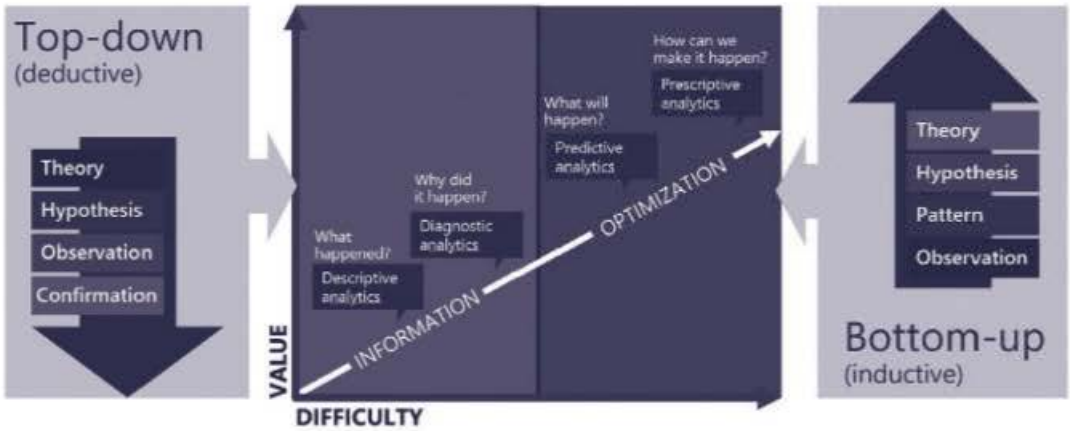
De hypothese moet in een volgende stap gecontroleerd worden. Misschien is het waar, misschien moeten we verder analyseren omdat er een andere oorzaak aan het waargenomen feit ten grondslag ligt. We controleren de hypothese door een rapport te maken (gebaseerd op het stermodel) dat het verband aantoont tussen omzet en temperatuur.

Het gemaakte rapport bevestigt de hypothese: de omzetstijging is te verklaren door de hogere temperatuur. De zojuist gedane analyse noemen we wel confirmatory analyse omdat we proberen om een vooraf geopperd idee te bevestigen. Als de hypothese door de analyse wordt ontkracht, herhaalt het hele proces zich met een nieuwe hypothese.



Confirmatory analyse gebruikt een top-downbenadering om een vooraf gestelde hypothese te bevestigen.

Top-down gaat uit van kennis vooraf. De analyse kan gedaan worden op basis van het datawarehouse of op basis van een semantisch model dat gebaseerd is op het datawarehouse, omdat er met voorkennis al rekening is gehouden met de mogelijkheid dat dit soort analyses uitgevoerd gaat worden.



Figuur 9.2 Top-down versus bottom-up

Bottom-up

De top-downbenadering ondersteunt Descriptive en Diagnostic Analytics, maar schiet tekort voor Predictive Analytics. Je zou kunnen denken dat als je het weerbericht opvraagt en de te verwachten temperatuur bekijkt, je nu kunt voorspellen hoeveel ijs je morgen gaat verkopen. Hoewel dat niet raar gedacht is, blijkt zo'n benadering in de praktijk te simplistisch. Je voorspelling zal nauwelijks beter zijn dan gewoon gokken.

Voor Predictive Analytics draaien we de zaken om. Als uitgangspunt nemen we data, in figuur 9.2 Observation genoemd. Het liefst beginnen we met ruwe, nog niet bewerkte, data. In bewerkte gegevens zijn misschien al (foutieve) aannames geïntroduceerd of zijn mogelijk relevante gegevens of patronen verwijderd. De dimensionele datamart is dus eigenlijk niet geschikt omdat tijdens de ETL en de cleansing allemaal bewerkingen op de gegevens zijn gedaan.

De ruwe gegevens worden geanalyseerd en tot een voor deze toepassing bruikbare verzameling omgevormd. Dit is een moeilijke maar zeer belangrijke stap. De gegevens waarop je voorspellingen baseert, bepalen grotendeels de betrouwbaarheid van de voorspelling die eruit komt. Je leest daar meer over in paragraaf 9.3 over machine learning.

Vervolgens analyseer je de gegevens, al dan niet met machine learning-algoritmes, om te proberen patronen in je data te ontdekken. Er zijn verschillende algoritmes die je kunt gebruiken. Welke je kiest en hoe je dat algoritme configureert heeft ook weer invloed op de betrouwbaarheid van de voorspelling. We noemen deze manier van analyseren exploratory analyse, omdat je als het ware op ontdekkingstocht gaat in je gegevensverzameling.

Exploratory analyse is een bottom-upbenadering waarbij je probeert om zonder beïnvloeding vooraf patronen in datasets te vinden.

De patronen die gevonden worden in de gegevens zijn uiteindelijk leidend. Uit de patronen kunnen we hypotheses afleiden. We noemen dat inductie. Bij inductief redeneren komt men tot een algemene regel, generalisatie geheten,

op grond van een aantal specifieke waarnemingen. Daarmee heb je dan uiteindelijk het proces beschreven. De algemene regel kan gebruikt worden om voorspellingen te doen.

9.2.4 Tot slot

Het woord analytics omschrijft een breed scala aan handelingen waar vooral het analyseren van gegevens en het communiceren van de resultaten centraal staan. Bij Advanced Analytics wordt over het algemeen gebruikgemaakt van machine learning en datamining. In de volgende paragraaf kun je daar meer over lezen.

9.3 Machine learning en datamining

Machine learning en datamining zijn niet hetzelfde. Het verschil is echter zo klein dat het voor veel discussies niet zinvol is om onderscheid te maken. We zullen proberen eerst via definities het theoretische verschil duidelijk te maken. Daarna bespreken we machine learning, ervan uitgaande dat er voor bijna alle praktische toepassingen eigenlijk geen onderscheid bestaat.

9.3.1 Definities

De definitie van datamining volgens Gartner is: '*The process of discovering meaningful correlations, patterns and trends by sifting through large amounts of data.*' (gartner.com)



Datamining is het achterhalen van verbanden, patronen en trends in gegevensverzamelingen.

Merk op dat in deze definitie geen doel van datamining staat. Waarom zou je datamining doen? Misschien is het doel wel puur het achterhalen van de verbanden en patronen. Vaak wordt als doel nog gesteld dat je met de opgedane kennis in staat bent om bepaalde waarnemingen te begrijpen. Dan zijn we eigenlijk nog bezig met Diagnostic Analytics.
In veel definities van datamining wordt expliciet gezegd dat het gaat om het ontdekken van tot nu toe onbekende patronen in de data. Daarmee komt een belangrijk verschil met machine learning naar boven, waar bekende patronen worden toegepast om voorspellingen te doen.
Machine learning is een techniek waarmee computers kunnen leren van bestaande gegevens en op basis van in die gegevens aanwezige patronen voorspellingen kunnen doen.

Machine learning is een techniek die tot doel heeft het doen van voorspellingen op basis van bekende patronen in gegevensverzamelingen.

Formeel is datamining dus het zoeken naar onbekende patronen, terwijl machine learning bekende patronen gebruikt om voorspellingen te doen. In de praktijk gebruiken bedrijven datamining om voorspellingen te doen zodra ze patronen ontdekt hebben. Net zo goed wordt machine learning gebruikt om patronen te zoeken om te kunnen gebruiken. Welke term gebruikt wordt, hangt soms af van degene aan wie je het vraagt.
De formele definitie van machine learning luidt: '*A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E*' (Tom M. Mitchell, 1997).
Belangrijk in deze definitie is het laatste stukje. Een machine learning-algoritme wordt niet statisch geprogrammeerd.
In de Engelse definitie van datamining staat nog het woord 'large'. Machine learning-algoritmes worden beter als je meer observaties hebt. Zowel machine learning als datamining worden vaak in één adem genoemd met Big Data. Dat is niet helemaal terecht. Datamining en machine learning bestonden al voordat de term Big Data was bedacht. Maar de link is logisch als je bij Big Data de nadruk legt op het aantal records (observaties) dat je hebt. Dat komt door de *'Law of Large numbers': 'The confidence with which predictions can be made increases as the size of the data being analyzed increases.'* Een meer formele beschrijving van deze wet kun je vinden op https://www.britannica.com/science/law-of-large-numbers.

De nauwkeurigheid waarmee voorspellingen gedaan kunnen worden, neemt toe als de gebruikte dataset groter wordt.

9.3.2 Machine learning-scenario's

Bij machine learning laten we dus een computer zelf leren op basis van (veel) gegevens. Hierbij wordt gebruikgemaakt van statistiek en andere wiskundige formules. Het gaat voor dit boek te ver om uit te leggen hoe dat precies werkt. Wel kunnen we op hoog niveau twee verschillende technieken onderkennen en uitleggen. Daarna kunnen we de verschillende scenario's bespreken.
De twee technieken die worden toegepast bij machine learning zijn:

- supervised learning;
- unsupervised learning.

Supervised learning

Supervised learning, ook wel learning by example genoemd, is min of meer hoe wij onze kinderen leren. Elke keer dat er een auto langsrijdt, zegt een ouder

tegen zijn baby in de kinderwagen: 'auto'. We doen dat misschien wel honderden keren. Elke keer ziet de baby iets anders. Soms is het groen, dan weer blauw, dan weer nog een andere kleur. Soms is het groot, soms niet zo groot. Je kunt ook zeggen dat het kind elke keer hetzelfde ziet. Een ding met kleur, vier wielen, een voorruit en twee koplampen.
Als er een fietser komt langsrijden, zegt de ouder misschien wel niets. Of hij zegt 'fiets'. De fietser kan een groene jas aan hebben, of een blauwe. Maar hij zit niet binnen en het ding waar hij op zit heeft geen vier wielen.
Op een dag komt er een auto langs en zegt de baby 'auto'. Na honderden voorbeelden heeft het kind geleerd wat wel en wat niet van belang is voor 'auto'. De kleur maakt niet uit, het aantal wielen wel. Hij heeft geleerd van eerdere observaties.
Bij supervised learning is er sprake van observaties, features en een label. Observaties zijn de rijen van onze dataset. De features zijn de kolommen in die dataset. En het label is het kenmerk, de kolom, die we willen voorspellen. In het voorbeeld willen we het type voertuig voorspellen. Fiets of auto? De features zijn in dit geval wat we allemaal zien, dus kleur, grootte, aantal wielen et cetera. Het uitgangspunt is dat we veel gegevens uit het verleden hebben waarvan het label al bekend is. Door die gegevens te analyseren leert de computer welke kenmerken belangrijk en bepalend zijn voor de inhoud van de labelkolom. De computer leert de patronen in de data kennen die leiden tot een bepaalde waarde van het label.



Voorbeeld 9.1

Rond 2002 installeerde ik een opensourcespamfilter. In de eerste dagen en weken maakte dat geen verschil, ik kreeg evenveel spam als voorheen. Wel markeerde ik vanaf nu elke keer als een mailtje spam was. Langzaam maar zeker kwamen er steeds meer mailtjes waarvan ik had aangegeven of ze wel of niet spam waren. Naar verloop van tijd werden steeds meer mailtjes automatisch naar de spamfolder verplaatst. De software had geleerd van alle keren dat ik het label (wel/geen spam) had ingevuld en bleek steeds beter in staat om te 'voorspellen' of een nieuw binnengekomen bericht spam was of niet.

Unsupervised learning

Unsupervised learning zit heel dicht aan tegen datamining. Er is geen label dat we willen voorspellen, we moeten nog onbekende patronen achterhalen. Meestal gaat het hier om het groeperen van gegevens.
Een winkelketen kan bijvoorbeeld zijn klanten indelen in groepen. Je kunt je eigen 'gevoel en ervaring' gebruiken en een indeling maken naar leeftijd of geslacht. Dat is een beetje top-downdenken. Je kunt ook de computer later zoeken naar klanten die qua aankoopgedrag meer of minder op elkaar lijken. Misschien komen er groeperingen uit die totaal onverwacht zijn of zelfs onlogisch klinken. Maar als het helpt om gerichter klanten te benaderen en daarmee de omzet te verhogen, is dat toch mooi meegenomen.

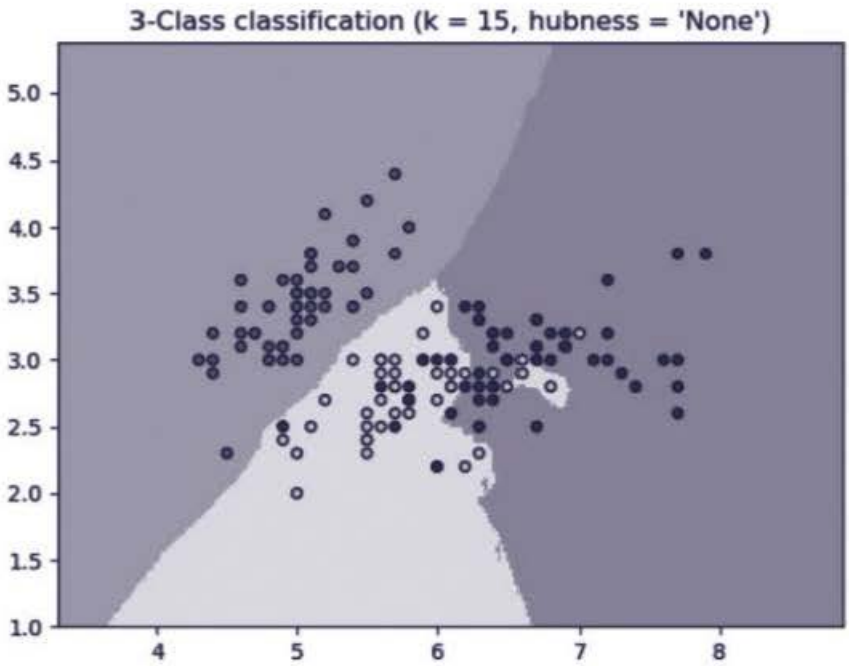
Nu we weten wat supervised en unsupervised learning is, kunnen we kijken naar toepassingen van machine learning. We zullen kort kijken naar:

- classification;
- clustering;
- outlier detection;
- regression.

Classification

Classification is een supervised learning-techniek waarbij records worden ingedeeld in van tevoren bekende categorieën. Het genoemde voorbeeld van het spamfilter is een voorbeeld van two-class classification. Fraudedetectie is een ander voorbeeld. Als er veel schadeclaims uit het verleden al zijn geoormerkt als wel of niet frauduleus, kan dat gebruikt worden om met classification van nieuwe claims te voorspellen of het om fraude gaat of niet.

Je kunt onderscheid maken tussen two-class- en multi-class classification. Bij two-class heeft het te voorspellen label maar twee mogelijke waarden, bijvoorbeeld wel of geen spam. Bij multi-class kunnen er meer mogelijke waarden zijn.



Figuur 9.3 Classification

Er zijn verschillende wiskundige algoritmes die je kunt toepassen bij classification. Support vector machines, decision trees, decision forests, logistic regression en neural networks zijn voorbeelden van algoritmes die hiervoor gebruikt worden.

Welk algoritme je kiest, hangt af van verschillende zaken. Hoeveel rijen heb je en uit hoeveel (relevante) features bestaan die? Welke patronen bevinden zich in de gegevens? Maar ook zaken als: hoeveel tijd hebben we om te trainen? Deze

keuzes zijn niet voor de hand liggend, maar kunnen wel bepalend zijn voor de uitkomst. Data scientists moeten deze keuzes maken.

Clustering

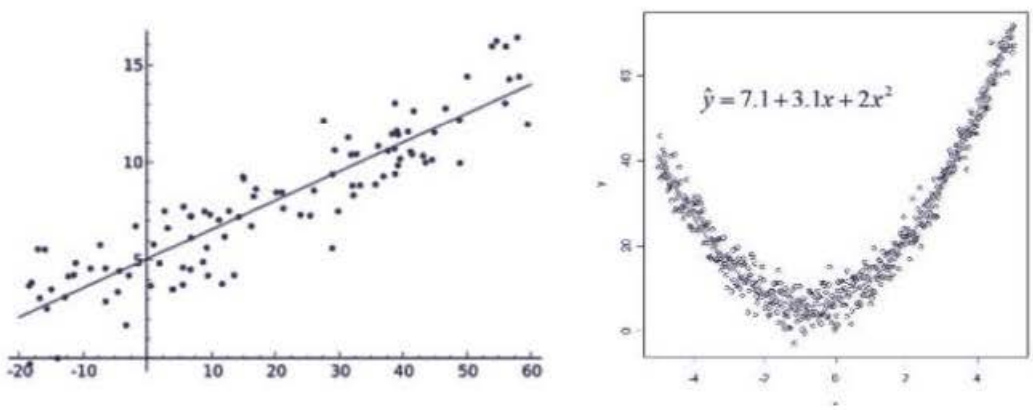
Clustering is een unsupervised learning-techniek die gegevens groepeert in 'clusters' van vergelijkbare records. Wat vergelijkbaar betekent, is niet van tevoren te voorspellen. Het hangt uiteraard weer af van de data die je hebt en het algoritme dat je gebruikt.
Het resultaat van clustering kan in een later stadium gebruikt worden in een classification-scenario.
K-means is een veelgebruikt algoritme voor clustering.

Outlier detection

Outlier detection is sterk verwant aan zowel classification als aan clustering. Het kan ook zowel supervised als unsupervised zijn. Het verschil zit hem voornamelijk in het aantal rijen (observaties) in een groep. Als je met clustering je gegevens probeert in te delen in twee groepen, kun je dat doen door twee groepen te maken die elk ongeveer evenveel rijen bevatten. Bij het voorbeeld van het spamfilter hoop je meer rijen te hebben die geen spam zijn dan wel spam, maar helaas zal 'wel spam' nog steeds regelmatig voorkomen.
Bij outlier detection gaat het om het vinden van een of enkele records die anders zijn dan alle andere.

Regression

Regression gaat uit van een oorzakelijk verband tussen twee variabelen, ofwel grootheden. In het voorbeeld van de ijsverkoper gaat de omzet omhoog als de temperatuur omhoog gaat. De temperatuur wordt nu de onafhankelijke variabele genoemd. De onafhankelijke variabele gebruik je om de afhankelijke variabele, in dit voorbeeld de omzet, te berekenen. Alles wat je moet weten is de wiskundige formule die de afhankelijkheid tussen temperatuur en omzet beschrijft.



Figuur 9.4 Regression

9.3.3 Tot slot

Er zijn veel algoritmes waaruit een data scientist kan kiezen bij de verschillende scenario's. Hij zal gedegen statistische kennis nodig hebben om de gegevens te analyseren, de juiste algoritmes te kiezen, de juiste parameters te kiezen en de uitkomsten op de juiste waarde te schatten.
Er zijn veel tools op de markt die de data scientist tot zijn beschikking heeft. De programmeertaal Python wordt veel gebruikt.
Verlies niet uit het oog waar het om gaat. Machine learning is een component van analytics. Het doel is om besluitvorming te ondersteunen. Uiteindelijk willen we er competitief voordeel uit halen. En daarmee vormt het een goede aanvulling (is het een component van) Business Intelligence.

9.4 Big Data en ethiek

The sky is the limit met Big Data Analytics. Hoe meer gegevens je hebt, hoe meer informatie je daaruit kunt halen. En informatie is kennis en kennis is macht. Het is goed om stil te staan bij de wenselijkheid van dit alles. Waar ligt de grens van wat wel en niet ethisch verantwoord is?
Ethiek is het nadenken over wat goed is en wat niet. Deze paragraaf is niet bedoeld om antwoorden te geven. Deze paragraaf is bedoeld om vragen op te werpen waar de samenleving als geheel, de politiek, en misschien jijzelf, over moet nadenken. Ook bedrijven die aan Big Data Analytics doen, zouden er goed aan doen om zich deze vragen te stellen.

9.4.1 Hoe wenselijk is Big Data Analytics?

Voorbeeld 9.2



Laatst hoorde ik van iemand dat hij was benaderd door een ziektekostenverzekeraar. Hij kreeg te horen dat hij een gezonde, sportieve, hoogopgeleide man van middelbare leeftijd was met een goed inkomen. Daarom kon de verzekeraar hem een zeer voordelige polis aanbieden.

Wat te denken van bovenstaand voorbeeld? In eerste instantie zou je misschien denken: iedereen blij. De persoon in kwestie een goedkope verzekering, de verzekeringsmaatschappij een goedkope klant. Maar riekt dit niet naar discriminatie? Wordt het racisme als we het kenmerk 'blank' toevoegen? Geldt omgekeerd dat een laagopgeleid persoon een hogere premie moet betalen? Of kunnen op den duur mensen die niet aan dit profiel voldoen helemaal geen ziektekostenverzekering meer afsluiten? Hoe ziet onze maatschappij er dan uit?

De Big Data Analytics die hierachter zit, is dat bedrijven in staat zijn om persoonsgegevens van klanten en potentiële klanten te achterhalen. Ook gegevens over leefstijl zijn zo voorhanden. Een eenvoudige statistische analyse leert welk type mens veel of weinig ziektekosten gaat hebben. Beide combineren levert al snel een aantrekkelijke marketingstrategie op.
In dit voorbeeld gebeurt niets illegaals, maar we bevinden ons wel op een hellend vlak. Misschien moet de overheid bepalen waar de grens ligt tussen wat is toegestaan en wat niet. Maar ook de overheid zet Big Data Analytics op grote schaal in. Onder het mom van de nationale veiligheid analyseren geheime diensten heel veel gegevens. Wie belt met wie? Hoe vaak en hoe lang? Neemt de intensiteit toe?
Met een beetje pech komt er uit de overheidsgegevens dat jij schuldig bent aan iets illegaals. Ben je nu onschuldig tot het tegendeel is bewezen of wordt de bewijslast omgedraaid en moet jij bewijzen dat je onschuldig bent? Als we niet oppassen, wordt de bodem onder onze huidige rechtsstaat weggeslagen.
Er zijn veel voorbeelden van overheidsdiensten die gegevens koppelen. Bijvoorbeeld om fraude op te sporen. Het argument is dat de samenleving als geheel erbij gebaat is, omdat zo 'het belastinggeld van de burger' beter besteed wordt. Maar wat betekent dit voor onze privacy?
Nog een voorbeeld is de aankoop van WhatsApp door Facebook. WhatsApp is een gratis dienst. Eentje die belooft dat je geen advertenties te zien krijgt. Hoe verdient Facebook de miljarden die ze betaald hebben terug? Als we ons meer gaan realiseren dat Facebook en Google bij alles wat we doen meekijken, gaan we ons waarschijnlijk anders gedragen. Als de glazenwasser door het raam naar binnen kijkt terwijl je in je ondergoed zit te ontbijten, trek je wellicht eerder kleren aan. Je verandert je gedrag omdat iemand meekijkt. Dat tast de kern van onze privacy aan. Is dat anders op het internet?
Op het internet lijkt er heel veel gratis. Spelletjes en sociale media, we betalen er geen geld voor. Maar als je niet met geld betaalt, betaal je met de gegevens die je achterlaat. Dat houdt wellicht in dat je met je privacy betaalt. Facebook analyseert ons gedrag en verkoopt/gebruikt de resultaten. Facebook weet vaak eerder wat jij leuk vindt dan dat je dat zelf weet!
Veel mensen vinden dat tegenwoordig niet erg omdat ze 'niets te verbergen' hebben. Het probleem is dat je vaak niet weet wat er met je gegevens gebeurt. Of wat er in de toekomst nog mee gedaan gaat worden. Daar zouden best eens zaken tussen kunnen zitten waar je eigenlijk wel bezwaar tegen hebt. Misschien krijg je wel nooit meer werk, omdat uit een of andere analyse komt dat je lui bent en slecht kunt samenwerken met anderen. Is het dan nog steeds oké?

9.4.2 Wet- en regelgeving

Bovenstaande komt erop neer dat we moeten discussiëren over de ethiek van Big Data en analytics. De overheid heeft met wet- en regelgeving vastgesteld wat wel en wat niet mag. Die regels zullen echter nooit in alle situaties kunnen

voorzien. Dus moeten bedrijven ook interne regels opstellen over het omgaan met de mogelijkheden van Big Data en de implicaties die dat heeft. Daarbij moet je nadenken over onder andere:

- privacy;
- transparantie;
- discretie;
- identiteit.

Privacy

Onder privacy verstaan we de regels die we opstellen over het vergaren van gegevens, het gebruik van die gegevens en de duur dat we die gegevens gaan bewaren. Bij het opstellen van regels hoort ook het naleven van regels en de controle op die naleving.

Als de overheid zegt dat camera's boven de weg alleen gebruikt worden voor snelheidscontrole, moet dat gegarandeerd en gecontroleerd worden. De gegevens gebruiken om files beter te voorspellen kan dan niet. Laat staan het controleren of je daadwerkelijk elke dag met je leaseauto naar je werk gaat!

Samen met privacy moet je nadenken over wie eigenaar is van de gegevens. Je moet zelf kunnen bepalen wat er met je gegevens gebeurt. Sommige sociale media hebben in de algemene voorwaarden wel zaken opgenomen. Voor sommige gebruiken is er een opt-in- of opt-outmogelijkheid. Regelmatig zijn die voorwaarden en opties zo goed verstopt, of zo complex opgesteld, dat mensen ze niet lezen of niet snappen. Daarmee kom je bij transparantie.

Transparantie

Het moet duidelijk zijn voor de eigenaar van de gegevens, of voor degene over wie de gegevens gaan, wat ermee gebeurt. Het al dan niet ethische zit vaak in het tweede gebruik. Dat een bank al je transacties bijhoudt om te weten wat je saldo is, het primaire gebruik, is oké. Maar wat als de bank die gegevens verkoopt aan winkelketens zodat zij dat kunnen gebruiken voor gerichte marketing en op maat gemaakte aanbiedingen? Misschien dat je dat juist fijn vindt, misschien ook wel niet.

Belangrijk bij dit secundaire gebruik is dat het duidelijk kenbaar wordt gemaakt welke gegevens door wie gebruikt gaan worden en met welk doel. Daarbij hoort dan weer de optie dat wel of niet toe te staan.

Een probleem kan ontstaan als alle banken in hun algemene voorwaarden opnemen dat ze klantgegevens gaan verkopen. Als je daarop tegen bent, kun je geen enkele bankrekening meer openen. Je werkgever eist echter dat je een rekening hebt, want ze betalen je salaris niet in contanten uit. Hier heeft de overheid duidelijk een rol te vervullen.

Discretie

Tenzij je je helemaal uit de maatschappij terugtrekt, ontkom je er niet aan om een digitaal spoor achter te laten. Het bewust geen mobiele telefoon nemen omdat je niet wilt dat telefoonaanbieders kunnen zien waar je bent, heeft grote

sociale gevolgen. Geen WhatsApp hebben omdat je Facebook niet vertrouwt, ook als bijvoorbeeld je voetbalteam alle afspraken en verzameltijden deelt via een WhatsApp-groep, heeft tegenwoordig verstrekkende consequenties voor je dagelijkse leven.
Belangrijk is dat je erop moet kunnen vertrouwen dat er discreet met jouw gegevens wordt omgesprongen. Je wilt niet dat je baas, of de overheid, via gegevens die voor andere doeleinden zijn bedoeld, weet waar jij 's avonds uithangt. Als je zelf op sociale media deelt waar je uithangt, is dat heel wat anders dan wanneer een telecombedrijf dat tegen betaling aan iemand vertelt doordat ze je telefoon traceren.

Identiteit

Mensen zijn kuddedieren. Of we het nu leuk vinden of niet, hoe individualistisch onze samenleving misschien wel is, de overeenkomsten tussen groepen mensen zijn groter dan de verschillen. Op basis van demografische gegevens kan het kiesgedrag van hele wijken worden voorspeld. Maar met de huidige mogelijkheden kunnen we zelfs kiesgedrag van individuen voorspellen. Campagneteams weten eerder dan kiezers zelf wat ze gaan stemmen. Ze weten ook hoe ze kiezers kunnen beïnvloeden om wel of niet te gaan stemmen. Is er zo nog wel sprake van eerlijke, vrije verkiezingen?
Het gaat er hier om dat we onze eigen identiteit kwijtraken. Dat gaat verder dan privacy. Het gaat hier om wie je bent. Het gaat erover dat onbekenden beter weten wie jij bent en hoe jij reageert, dan dat jij dat zelf weet. Met die kennis kunnen (en zullen) ze jouw gedrag gaan beïnvloeden. Dat gebeurt al jaren op grote schaal en dat heet reclame en marketing. Er is echter een verschil als groepen mensen het doel zijn dan wanneer mensen individueel getarget worden. Nog erger is als je je helemaal niet meer bewust bent dat je beïnvloed wordt, waar je voorheen nog wel wist dat je naar reclame keek.

9.4.3 Tot slot

Dit is geen boek over filosofie of ethiek. Maar het onderwerp van dit hoofdstuk werpt wel vragen op over ethische kwesties. Wet- en regelgeving vanuit overheden loopt (bijna) altijd achter bij de technische mogelijkheden. Zelfs waar dat niet zo is, zal de wet niet volledige duidelijkheid verschaffen voor de situaties die kunnen optreden. Niet alle vragen die je kunt stellen, zullen door de wet beantwoord worden. Het is dus goed om als bedrijf zelf ook met beleid omtrent Big Data te komen.
Gelukkig hoef je niet zelf het wiel uit te vinden. De Data Science Association (www.datascienceassn.org) heeft gedragsregels opgesteld voor data scientists als houvast bij ethische vragen. Er zijn ook andere organisaties die vergelijkbare initiatieven hebben.



Zowel als maatschappij als binnen bedrijven, is het noodzakelijk om stil te staan bij de ethische kant van Big Data Analytics.

Na deze korte uitstap over ethiek gaan we terug naar analytics. We beginnen te kijken naar de mensen met wie we de resultaten van de analytics zoals eerder besproken willen delen.

9.5 Wie maakt en wie gebruikt rapporten?

Een belangrijk onderdeel van analytics en een belangrijk onderdeel van de kleine BI-cyclus zoals besproken in hoofdstuk 2 is het delen van opgedane kennis met anderen en het communiceren van en over de inzichten en de bijbehorende besluiten. Een belangrijk onderdeel van die communicatie is het maken van rapporten en dashboards. Het visualiseren van de gegevens, bijvoorbeeld zoals figuur 9.4 duidelijk moet maken wat regressie is, is daarbij cruciaal. Hoe die visualisaties bij de beoogde gebruikers komen is ook van groot belang. Eerder is gehamerd op het feit dat de acceptatie van de gebouwde oplossing een van de belangrijke factoren is in de bepaling of een Business Intelligence-oplossing succesvol is. Hoewel inhoud en dus de kwaliteit van de informatie daarin een belangrijke rol spelen, is het front-end hoofdzakelijk verantwoordelijk voor deze acceptatie. Ziet het er leuk uit? Is het prettig om mee te werken? Leuke, mooie tools waar mensen graag mee werken bovenop slechte gegevens zullen toch eerder en meer gebruikt worden dan verkeerd gekozen tools op perfecte gegevens.

Om de juiste tools te kiezen moet je naar twee zaken kijken:
- Technische specificaties van de beschikbare tools.
- Wie is de doelgroep?

Technische specificaties

Uiteraard moet je bekijken welke tools er beschikbaar zijn. Kijk daarbij hoe deze tools aansluiten op de achterliggende database en/of achterliggend model. Bekijk ook voor welke doelgroep deze tools gemaakt zijn.

Wie is de doelgroep?

Dat laatste brengt je bij het tweede en belangrijkste aspect: wie vormen de doelgroep? Welke mensen binnen een organisatie moeten wat gaan doen? Over welke vaardigheden beschikken zij en welke tools sluiten daar goed bij aan? Moeten mensen op cursus gestuurd worden om te leren op de juiste wijze om te gaan met de tools?

In hoofdstuk 2 zijn al vier soorten gebruikers gedefinieerd, de farmers, de tourists, de explorers en de miners (zie paragraaf 2.4.3). Je moet weten welke mensen in welke groep vallen en bepalen welke tool aansluit bij de gestelde BI-doelen voor deze mensen. De gekozen tool moet aansluiten bij hun technische kennis en vaardigheden. Deze afweging moet het liefst zo vroeg mogelijk in de implementatie van Business Intelligence gemaakt worden, omdat de fundering moet aansluiten bij deze keuzes.

9.5.1 Farmers

Farmers zijn in hoofdstuk 2 gedefinieerd als mensen met een vaste, terugkerende informatiebehoefte. Dit kan om externe mensen gaan, zoals een moedermaatschappij of aandeelhouders die maandelijks of eenmaal per kwartaal een financieel overzicht willen. Intern kan het gaan om rapportage en vastlegging van gegevens voor maandverslagen. Ook mensen die telkens weer hetzelfde lijstje bekijken en op basis daarvan acties uitzetten, vallen onder farmers. Een verkoopmanager gebruikt bijvoorbeeld een lijst met gescoorde orders tot nu toe in de huidige maand, aangevuld met de top vijf best en top vijf slechtst scorende verkopers.
Over het algemeen is de farmer niet zelf de maker van het rapport. Vaak is de maker een BI-ontwikkelaar of iemand met een vergelijkbare rol. Veel bedrijven hebben een of enkele mensen in dienst speciaal voor het maken van dit soort rapporten.

Voor het soort rapporten dat door farmers wordt gebruikt, bestaan verschillende benamingen. Enkele hiervan zijn:
- productierapporten;
- pixel perfect-rapporten;
- operationele rapporten;
- enterprise reporting.

Pixel perfect verwijst letterlijk naar de grafische capaciteiten van de rapporttool. Voor sommige rapporten is het bijvoorbeeld goed genoeg dat het logo van het bedrijf rechtsboven in het rapport staat en dat de gebruikte kleur blauw is. Bij andere rapporten moet het logo op precies 12 millimeter van de bovenste kantlijn staan en moet de gebruikte kleur in RGB-codes exact 135-206-250 zijn. Ik heb weleens een rapport gemaakt waar een barcode op stond. Eenmaal geprint werd het rapport automatisch gescand om de informatie van de barcode weer in te lezen. Als de barcode een fractie te veel naar links of naar rechts stond op de geprinte versie, las de scanner die barcode niet. Met zo'n eis moet de gebruikte rapportagetool in staat zijn om tot op de pixel nauwkeurig de plaats van de barcode te bepalen.

De andere genoemde benamingen wijzen meer op het doel van het rapport dan op de technische vereisten.
Hoe farmers rapporten gebruiken komt overeen met Descriptive Analytics (zie paragraaf 9.2.1). Het rapport beschrijft iets, alle interpretatie ligt bij de gebruiker. Hoe de informatie wordt geïnterpreteerd is wel heel belangrijk. Goede rapporten kunnen verkeerd worden geïnterpreteerd wat (wellicht) tot verkeerde beslissingen leidt. Goede basisgegevens die slecht worden weergegeven, bijvoorbeeld door een verkeerde grafiekkeuze, kunnen net zo goed slecht worden geïnterpreteerd. Later in dit hoofdstuk kun je meer lezen over het kiezen van de juiste visualisatie.

Wie maakt de rapporten?
Rapporten die gebruikt worden door farmers worden meestal gemaakt door mensen wier werk het is om rapporten te maken. Het voordeel hiervan is dat de complexiteit van de gebruikte tool om rapporten te maken geen problemen zou moeten opleveren. Zelfs al moeten we iemand drie weken op cursus in Amerika sturen om de tool te leren kennen, de meerwaarde die zo iemand heeft door de rapporten die hij maakt, compenseert een dergelijke investering wel. Overigens moet er wel rekening mee worden gehouden dat mensen ziek kunnen worden of het bedrijf verlaten. Hoe gemakkelijk of moeilijk is het dan om ze te vervangen? Voor complexe, weinig gebruikte tools is het moeilijk om snel goede vervanging te vinden.
Omdat gespecialiseerde mensen met gespecialiseerde, en in potentie, complexe tools kunnen werken, kunnen er ook complexe (pixel perfecte) rapporten gemaakt worden. Meestal is het zo dat eenvoudige tools eenvoudige rapporten opleveren. Als er meer eisen gesteld worden aan het eindresultaat, zal de tool meer moeten kunnen. Daar worden tools complexer van. Voor een professional is dat geen probleem. Voor een 'gewone' gebruiker kan zo'n tool, zo'n ontwikkelomgeving, al snel te complex worden. Uiteraard kan een semantisch model zoals besproken in het vorige hoofdstuk dit probleem oplossen. Als alle complexiteit in een semantisch model is gestopt, is het maken van het rapport zelf vaak makkelijk.

9.5.2 Tourists

Tourists gaan, ten opzichte van farmers, iets meer de gebaande paden verlaten. Ze zoeken iets vaker hun eigen weg. Dat leidt tot vragen die vooral voor henzelf of slechts voor een kleinere groep collega's interessant zijn.
Nadat tourists de informatie uit de standaardrapporten tot zich hebben genomen, blijft er nog een informatiebehoefte achter. Dat kunnen eenmalige (ad-hoc-)vragen zijn die na te zijn beantwoord (misschien) hun relevantie verliezen. Het kan ook om aanvullende informatie gaan die van terugkerende aard is.

Eenmalige vragen

Bij eenmalige (ad-hoc-)vragen, moet je je afvragen wie deze vragen het beste kan beantwoorden. Moet een IT-professional een query schrijven of een rapport maken om aan deze ad-hoc-vraag te voldoen? Of moet de steller van de vraag zelf op zoek gaan naar het antwoord?

Het voordeel van de professional ad-hoc-vragen te laten beantwoorden, is zijn technische bagage. Daarmee is de tool-keuze weer geen beperking.

De nadelen van het inzetten van een specialist zijn:

- Het is (relatief) duur.
- De afdeling die rapporten maakt is druk, waardoor de vraag pas laat beantwoord wordt.
- Door slechte communicatie tussen de vrager en de IT-professional voldoet het rapport in eerste instantie vaak niet aan de vraag.

Als tourists zelf hun informatie gaan verzamelen, moet goed gekeken worden naar wie deze tourists nu eigenlijk zijn en wat ze kunnen. Zijn ze, naast het werken met een complexe omgeving, in staat om de benodigde query's zelf te schrijven?

Vaak zal het antwoord op dergelijke vragen 'nee' luiden. Andere tools kunnen dan een oplossing bieden. Semantische modellen kunnen hier helpen. Een model maakt de vertaalslag van techniek naar inhoud, wat het een stuk gemakkelijker zou moeten maken om met een model te werken dan rechtstreeks met een database.

Los van of er wel of geen model gebruikt wordt, is de tool zelf waar de tourists mee moeten gaan werken van belang. Hoe gemakkelijk of moeilijk is het om met die tool te werken, willen mensen er ook zelf daadwerkelijk mee aan de slag en voldoen de rapporten die ermee gemaakt kunnen worden aan alle eisen die aan de rapporten gesteld worden?

Self-service

Met goede modellen en self-service BI-tools zoals Power BI en Qlik kunnen tourists tegenwoordig vrij laagdrempelig zelf interactieve rapporten maken. Hoe ver ze komen is afhankelijk van de technische vaardigheden die ze zelf opdoen in combinatie met hoe goed en volledig het beschikbare semantische model is.

Terugkerende detailvragen

Naast de ad-hoc-rapportages hebben tourists behoefte aan gedetailleerdere informatie dan de standaardrapportages meestal leveren. Je kunt deze extra informatie leveren door rapporten interactief te maken. Het gaat dan vooral om rapporten die op het scherm bekeken worden. Bij interactieve rapporten kun je onderscheid maken tussen:

- drill-down;
- drill-through.

Drill-down staat voor het navigeren van hoog geaggregeerde gegevens naar steeds meer detail. Als je een rapport opent zie je bijvoorbeeld jaaromzetten met voor de jaartallen plusjes. Klikken op een plusje, klapt een jaar uit om kwartaalomzetten zichtbaar te maken. Ook voor elk kwartaal staat een plusje dat de gebruiker in staat stelt om dieper op de details in te gaan. De gebruiker kan naar maandomzetten en vervolgens kan daar ook weer op geklikt worden, en zo verder.
Over het algemeen is de detailinformatie al bij het openen van het rapport beschikbaar, dus ingelezen vanuit de database. Alleen worden de details in eerste instantie nog niet getoond, wat het rapport klein en overzichtelijk maakt. Pas als de gebruiker er expliciet om vraagt, worden details zichtbaar.

Drill-through betekent dat op een rapport hyperlinks staan naar andere rapporten. Een gebruiker bekijkt een rapport en kan vervolgens op items (zoals balkjes in grafieken of namen van klanten) klikken. Daarmee wordt een ander rapport geopend waar meer informatie te vinden is over datgene waarop is geklikt. Dat kan dezelfde soort informatie zijn als waarop geklikt is, maar dan in meer detail. Het kan natuurlijk ook om andere informatie gaan.

Conclusie
Tourists stellen andere eisen aan rapporten dan farmers. Ze bewegen zich langzaam een beetje van de Descriptive Analytics naar de Diagnostic Analytics. Alleen getallen zien, al dan niet in de vorm van grafieken, is niet genoeg. Door een drill-down of drill-through proberen we meer te weten te komen over hoe een waarde tot stand is gekomen.
Het onderkennen van de doelgroep tourists kan leiden tot andere keuzes ten aanzien van de te gebruiken tools. Dit geldt vooral als de tourists zelf in hun informatiebehoeften moeten/mogen voorzien. De gekozen tools hebben eventueel weer consequenties voor het soort modellen dat beschikbaar moet zijn. Veel tools bieden zowel een semantische laag als een front-end.
Minstens zo belangrijk als de tools is de realisering dat andere gebruikers andere soorten rapporten willen. Er moeten opeens navigatieopties komen zoals drill-down en drill-through. Dit kan ook andere visualisaties eisen. De rapporten moeten er, kortom, anders uitzien.

9.5.3 Explorers

Naast farmers en tourists hebben we de zogenaamde explorers. Waar tourists misschien enigszins buiten de gebaande paden treden, zijn dit de echte avonturiers. Het zijn de mensen die nieuwe wegen moeten en willen zoeken. Vooraf gedefinieerde rapporten beantwoorden vooraf gedefinieerde vragen. Dat zijn vragen die eerder gesteld zijn. Ze geven antwoorden op vragen gesteld vanuit het bestaande perspectief, vanuit bestaande inzichten.

Vragen stellen vanuit bestaande inzichten leidt niet tot nieuwe inzichten. Je moet af en toe 'out of the box' denken. Kies eens een andere insteek. Probeer eens vanuit een andere invalshoek naar de gegevens te kijken. Zoek verbanden, zoek informatie die nog niet bekend is. Dat is wat explorers moeten doen.
Klassieke rapportagetools zoals SQL Server Reporting Services of bijvoorbeeld Crystal Reports zijn niet geschikt voor explorers. Ze zijn te statisch en te veel gericht op het beantwoorden van die ene terugkerende informatiebehoefte. Explorers moeten tientallen, zo niet honderden, vragen kunnen stellen. Velen zullen niet echt terugkerende informatiebehoeften blijken te zijn. Een tool als Excel lijkt optimaal geschikt voor explorers. Haal maar gegevens binnen in Excel en gebruik vervolgens alle mogelijkheden van Excel zoals draaitabellen en grafieken, functies en formules, filtering en sortering en conditionele opmaak, om de gegevens vrij, naar eigen inzicht, te kneden tot informatie.
Semantische modellen kunnen hier een belangrijke rol spelen. Excel leest bijvoorbeeld, naast de gegevens, ook alle metadata van Analysis Services-modellen uit. De gebruiker kan daarmee gemakkelijk met alleen slepen door de gegevens heen browsen. '*To slice and dice*' heet dat, het maken van willekeurige doorsnedes van de gegevens.
Modellen zouden ook juist beperkingen kunnen opleveren. Zijn ze niet te veel gemaakt met bekende gegevens en op basis van bekende inzichten? Moeten explorers niet verdergaan dan wat IT in de modellen stopt? Met bijvoorbeeld Excel PowerPivot kan de explorer verder gaan waar de bestaande modellen stoppen. PowerPivot kan gebruikmaken van bestaande Analysis Services-databases, maar die gegevens aanvullen met willekeurige andere gegevens. Daarmee kan de explorer die extra stap maken.
De explorer is nog veel meer dan de tourist bezig met Diagnostic Analytics. De antwoorden waarom iets is zoals het is, liggen niet altijd besloten in de drill-down die in een rapport zit. Soms moet het antwoord gevonden worden buiten de eigen gegevens. Misschien is de huidige conjunctuur van invloed, of het weer. Een standaard rapport zal dat meestal niet laten zien. We moeten een stap extra zetten om dat soort invloeden te zien.

9.5.4 Miners

In het woord miners vind je de term datamining terug en dat is niet toevallig. Miners zijn mensen die bijvoorbeeld datamining inzetten om gegevens te analyseren. Zij hebben aparte tools nodig om dat te doen. Dit zijn de mensen die doen aan Advanced Analytics en dan voornamelijk de Predictive Analytics.
Een heel bekende tool die vanuit de academische wereld groot is geworden is SPSS. SPSS stond oorspronkelijk voor Statistical Package for the Social Science. Het wordt echter allang niet meer alleen gebruikt in de sociale wetenschappen. Het is tegenwoordig onderdeel van IBM Business Analytics en wordt in veel bedrijven toegepast. Ook de programmeertalen R en Python zijn populair, inclusief de tools om met deze talen te werken.

De term miner is misschien niet meer van deze tijd. De data scientist vervult deze rol.

9.5.5 Tools en acceptatie

Zoals ondertussen duidelijk moet zijn, moet je bij de keuze voor een rapportagetool naar een aantal aspecten kijken:

- Waarom heeft iemand welke informatie nodig en hoe vastomlijnd is die informatiebehoefte?
- Technische aspecten.
- Kennis en vaardigheden van de doelgroep.

Bij technische aspecten moet je denken aan vragen als:

- Wat moet de bron zijn om in deze informatiebehoefte te gaan voorzien?
- Wat voor eisen worden er gesteld aan de presentatie van de gegevens? Moet het vooral visueel zijn met plaatjes, grafieken en meters, of juist meer tekstueel met tabellen en draaitabellen?
- Moeten de rapporten interactief zijn en uitdagen om op zoek te gaan naar meer informatie of is het de bedoeling dat ze worden geprint?

De kennis en vaardigheden van de gebruikers van rapportages is van belang. Kennen de mensen het bedrijf en de bedrijfsprocessen goed genoeg om hun rol als bijvoorbeeld explorer te kunnen vervullen? Hebben ze genoeg wiskundige achtergrond om een miner te kunnen zijn? Zijn de mensen in staat om te werken met de tools die gebruikt (gaan) worden? En misschien nog wel belangrijker, willen ze ook werken met die tools? Er zijn BI-projecten mislukt, omdat de wil niet bestond om te werken met de bijgeleverde front-endsoftware.
Belangrijk bij het kiezen van de juiste front-endtools is inspraak geven aan de beoogde gebruikers. Alles wat wordt opgelegd brengt per definitie wat weerstand met zich mee. Die drempel is vast genomen als de mensen zelf mogen (mee) beslissen. Bovendien weten ze zelf het beste wat ze willen en kunnen. ICT mag de technische specificaties als inbreng hebben in het besluitvormingsproces.

9.6 Rapporten, scorecards en dashboards

In de vorige paragraaf is voornamelijk gesproken over rapporten. Een rapport is een manier om gegevens aan een gebruiker te tonen, een manier om in een informatiebehoefte te voorzien. Vaak wordt gekozen om informatie te presenteren op scorecards en/of in dashboards. Meestal gaat het dan om stuurinformatie. Zijn dat alleen chique termen voor gelikte interfaces of voegen ze echt iets toe?

Om scorecards te bespreken herhalen we eerst een stukje theorie over KPI's. Daarmee kunnen we scorecards bespreken en daarna de stap maken naar dashboards. De paragraaf eindigt met wat opmerkingen over rapporten.

9.6.1 KPI

Het acroniem KPI staat voor Key Performance Indicator of vertaald: Kritieke Prestatie-Indicator. Volgens Wikipedia is een KPI: 'een variabele om prestaties van ondernemingen te analyseren'. Verder zegt Wikipedia nog dat het een managementinstrument is.



Een **KPI** is een managementinstrument dat in één oogopslag de status van een proces laat zien.

Elke onderneming heeft een doel dat, als het goed is, is vastgelegd in een mission statement of een vergelijkbaar soort document. Om de vastgestelde missie na te streven wordt een strategie bedacht. Hoe denken we deze missie vorm te geven? De strategie van een bedrijf is de langetermijnplanning (vijf tot tien jaar). Wat gaan we als onderneming in die periode doen? Om de strategie uit te voeren en te controleren of we nog wel de goede koers varen, worden kortetermijndoelstellingen afgeleid uit de gekozen strategie.



Een **mission statement** is het algemene doel dat een organisatie zichzelf als geheel stelt.



De **strategie** van een organisatie is de wijze waarop de organisatie het doel zoals verwoord in het mission statement denkt te bereiken.

Zoals in hoofdstuk 2 al aangestipt moeten doelstellingen SMART zijn. SMART betekent:

- Specifiek. Is de doelstelling duidelijk en eenduidig?
- Meetbaar. Onder welke voorwaarde is de doelstelling gehaald en kunnen we dat verifiëren?
- Acceptabel. Kunnen alle betrokken zich vinden in deze doelstelling?
- Realistisch. Is de doelstelling wel echt haalbaar?
- Tijdgebonden. Wanneer moet de doelstelling zijn gehaald?

Doelstellingen kunnen op veel niveaus in de organisatie gesteld worden. Met niveaus wordt vaak strategisch, tactisch en operationeel bedoeld. Strategisch is zoals gezegd op lange termijn en speelt vooral bij het hoger management.

Operationeel zijn de kortetermijndoelstellingen (dit jaar) en speelt bij iedereen op de werkvloer en teamleiders. Tactisch zit daartussenin.
Zoals gezegd zijn KPI's een managementinstrument. In hoofdstuk 2 is al aangegeven dat het meetbaar zijn van een doelstelling in het kader van BI de belangrijkste component is van een SMART-doelstelling. Dat is de beperkte blik van de BI-technicus; vanuit een breder bedrijfsperspectief zijn de andere componenten van SMART even belangrijk. Met KPI's maken wij echter inzichtelijk of de doelstelling behaald wordt. Met de hele Business Intelligence-inspanning verzamelen we de gegevens die nodig zijn om de doelstellingen meetbaar te maken. Een KPI toont uiteindelijk het resultaat.

Zoals in hoofdstuk 2 ook al is beschreven, bestaat een KPI uit vier componenten:
- meetwaarde;
- doel;
- status;
- trend.

Meetwaarde

De eerste component van een KPI is de meetwaarde, of, in datawarehousetermen: het feit. Hierbij kun je aan van alles denken. Op het financiële vlak bijvoorbeeld aan omzet, winst en marge. Marktaandeel, aantal nieuwe klanten of aantal hits op de website zijn voorbeelden uit het klantenperspectief. Het interne perspectief heeft wellicht doelstellingen gedefinieerd voor het aantal ziektedagen en het verloop van werknemers. Het leer- en groeiperspectief kijkt naar bijvoorbeeld aantal gevolgde cursussen of het aantal gecertificeerde mensen. In de hier genoemde perspectieven herken je de balanced scorecard zoals beschreven in paragraaf 2.4.4.

Doel

Een kenmerk van alle hierboven beschreven voorbeelden is dat het om numerieke zaken gaat. Getallen kunnen gemakkelijk vergeleken worden en er bestaat geen onduidelijkheid over wat groter en wat kleiner is. Voor al deze voorbeelden kan dan ook eenvoudig een doelstelling worden gedefinieerd, ook weer in de vorm van een getal. Deze doelstelling is de tweede component van de KPI. Feit (meetwaarde) en doelstelling kunnen nu gemakkelijk vergeleken worden.

Status

De derde component van de KPI is de status. Hoe staan we ervoor? Worden doelstellingen gehaald of niet? Met twee getallen als invoer (meetwaarde en doelstelling) lijkt dit een makkelijke vraag. Het antwoord is ja of nee, want de gemeten waarde is kleiner dan de doelstelling of groter dan wel gelijk aan de doelstelling. Er bestaan echter vijftig tinten grijs (naar het schijnt). Soms ben je bijna tevreden of niet helemaal tevreden, omdat je doelstelling net is gehaald of slechts met de hakken over de sloot. Wanneer je tevreden bent of niet en

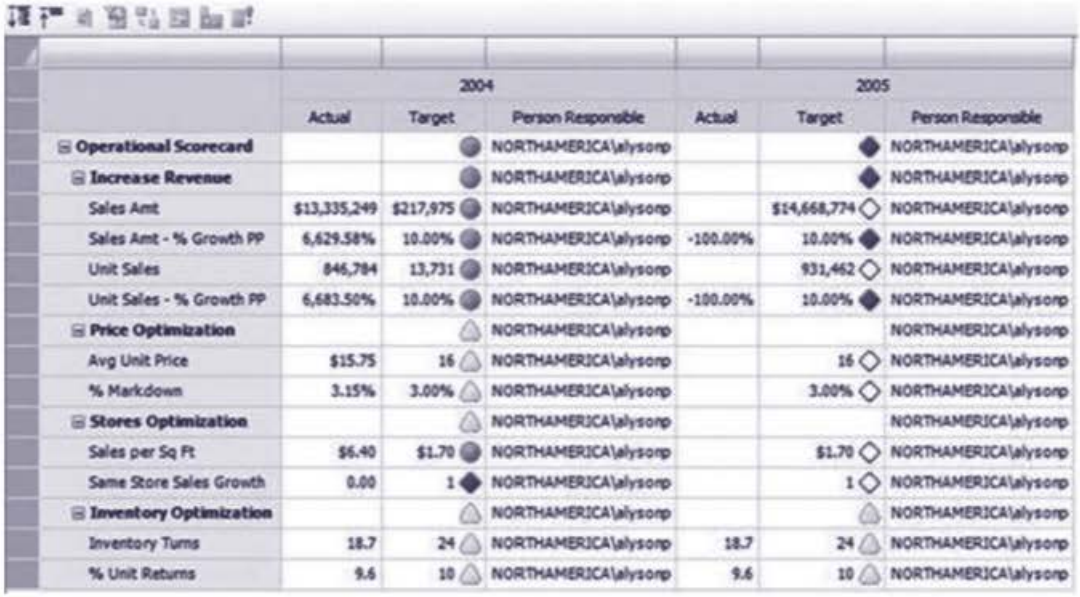
in hoeveel tinten grijs je dat wilt uitdrukken is aan de organisatie. Het is een business rule.
Meestal wordt de status van een KPI met een of andere grafische indicator weergegeven. Stoplichtjes en smileys zijn bekende voorbeelden. Het idee daarachter is dat een plaatje meer zegt dan duizend woorden. Met andere woorden, door de juiste icoontjes te gebruiken ziet iemand snel of alles oké is of niet. Een KPI wordt daarmee een visualisatie van hoe het met (een deel van) het bedrijf gaat.

Trend

Tot slot heeft een KPI vaak, maar niet noodzakelijkerwijs, een trendindicatie. Hoe verhoudt de status van de KPI zich met de status van dezelfde KPI vorige maand? Of met de status van dezelfde periode maar dan een jaar geleden? Een miljoen winst is een goed resultaat maar als de winst over dezelfde periode een jaar geleden nog een miljard was, is het toch wat mager. Vaak hoor je op het nieuws winstwaarschuwingen of dalende beurskoersen, omdat de winst minder is dan verwacht of dan in eerdere periodes. De huidige status alleen vertelt dus niet het hele verhaal.

9.6.2 Scorecard

De meeste mensen hebben meer dan één doelstelling te verwezenlijken en dus in de gaten te houden. Een scorecard is niets anders dan een lijstje van KPI's. Het doel van een scorecard is snel te kunnen zien hoe een organisatie, een afdeling of een persoon presteert. Figuur 9.5 is een typisch voorbeeld van een scorecard. Enkele KPI's zijn uitgezet tegen 2004 en 2005. Er zijn doelen gesteld voor verkoop, maar ook voor het prijsbeleid, voor winkels en voor het magazijn. De verkoopdoelen zijn weer onderverdeeld in onder andere omzet en aantal stuks verkocht.
Een deel van de meerwaarde zit in de icoontjes. Door icoontjes te gebruiken vereenvoudigt de scorecard en gaat de leesbaarheid omhoog. Een snelle blik op een scorecard kan al veel onthullen over de stand van zaken van een organisatie. Een scorecard is daarmee niets anders dan een speciaal soort rapport. Figuur 9.5 zou zomaar gewoon in Excel gemaakt kunnen zijn. Ook elke andere klassieke rapportagetool (zoals SQL Server Reporting Services of Crystal Reports) kan gebruikt worden om scorecards te maken. De icoontjes, ook wel indicators genoemd, zitten tegenwoordig standaard in de meeste rapportagetools. Met een beetje handige conditionele opmaak heb je snel iets gebouwd dat erop lijkt. Hoewel je scorecards (of rapporten die lijken op scorecards) kunt maken met elke tool, zijn er speciale tools op de markt om dashboards en scorecards te maken.



	2004			2005		
	Actual	Target	Person Responsible	Actual	Target	Person Responsible
Operational Scorecard			NORTHAMERICA\alysonp			NORTHAMERICA\alysonp
Increase Revenue			NORTHAMERICA\alysonp			NORTHAMERICA\alysonp
Sales Amt	$13,335,249	$217,975	NORTHAMERICA\alysonp		$14,668,774	NORTHAMERICA\alysonp
Sales Amt - % Growth PP	6,629.58%	10.00%	NORTHAMERICA\alysonp	-100.00%	10.00%	NORTHAMERICA\alysonp
Unit Sales	846,784	13,731	NORTHAMERICA\alysonp		931,462	NORTHAMERICA\alysonp
Unit Sales - % Growth PP	6,683.50%	10.00%	NORTHAMERICA\alysonp	-100.00%	10.00%	NORTHAMERICA\alysonp
Price Optimization			NORTHAMERICA\alysonp			NORTHAMERICA\alysonp
Avg Unit Price	$15.75	16	NORTHAMERICA\alysonp		16	NORTHAMERICA\alysonp
% Markdown	3.15%	3.00%	NORTHAMERICA\alysonp		3.00%	NORTHAMERICA\alysonp
Stores Optimization			NORTHAMERICA\alysonp			NORTHAMERICA\alysonp
Sales per Sq Ft	$6.40	$1.70	NORTHAMERICA\alysonp		$1.70	NORTHAMERICA\alysonp
Same Store Sales Growth	0.00	1	NORTHAMERICA\alysonp		1	NORTHAMERICA\alysonp
Inventory Optimization			NORTHAMERICA\alysonp			NORTHAMERICA\alysonp
Inventory Turns	18.7	24	NORTHAMERICA\alysonp	18.7	24	NORTHAMERICA\alysonp
% Unit Returns	9.6	10	NORTHAMERICA\alysonp	9.6	10	NORTHAMERICA\alysonp

Figuur 9.5 Voorbeeld van een scorecard

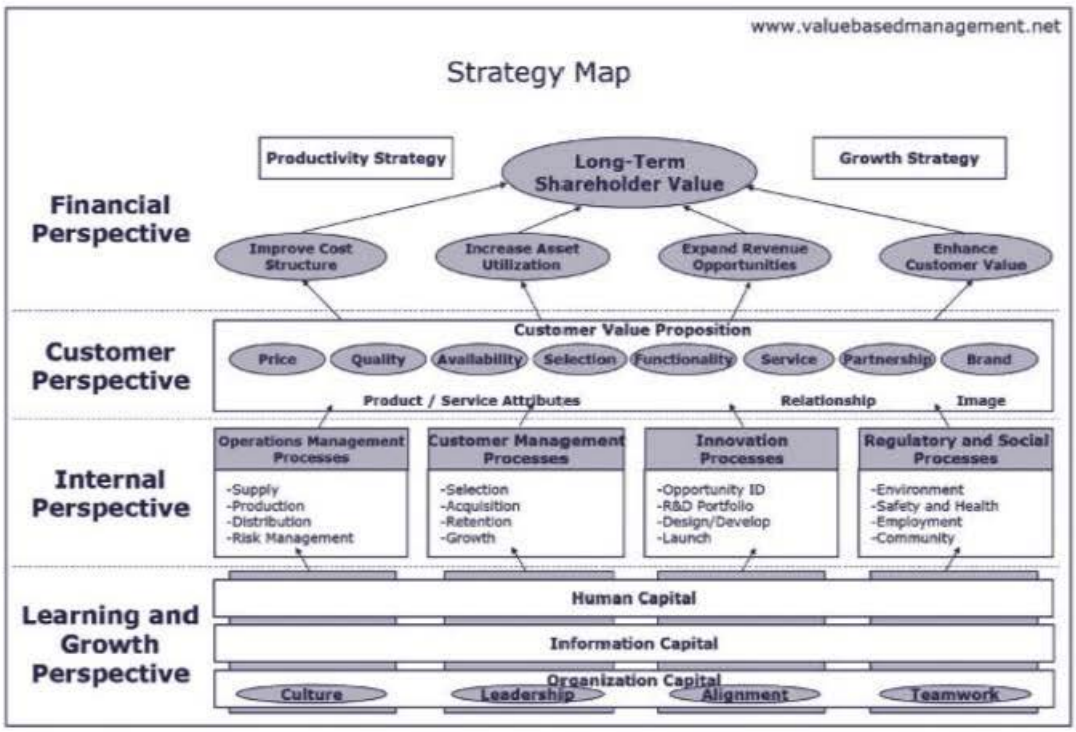
Balanced scorecard

Als de term scorecard valt, gaat het ook al snel over de balanced scorecard. Balanced scorecard is een techniek voor strategisch management en het behalen van langetermijndoelstellingen binnen bedrijven. In hoofdstuk 2 heb je al kunnen lezen over de balanced scorecard.

De balanced scorecard is een methode om bedrijven aan te sturen. Het gaat uit van een strategy-driven managementmodel. Het management bepaalt een strategie waar doelstellingen uit voortvloeien. Het bedrijf wordt gestuurd op het wel of niet halen van die doelstellingen. De balanced scorecard is hierbij dus de methode van aansturen en de scorecard is het hulpmiddel dat daarbij gebruikt wordt.

Strategy map

Bij de balanced scorecard-methode en de hulpmiddelen die daarbij gebruikt worden, hoort volgens Kaplan en Norton, de bedenkers van de balanced scorecard, ook een strategy map. Figuur 9.6 is een voorbeeld van een strategy map. Een strategy map is ingedeeld in dezelfde perspectieven als in hoofdstuk 2 zijn besproken voor de balanced scorecard. Het beoogt het verband tussen de strategie en de concreet gestelde doelstellingen duidelijk te maken.



Figuur 9.6 Strategy map

Een strategy map kan gezien worden als documentatie bij een scorecard die is opgezet volgens de theorie van de balanced scorecard, dus met KPI's in alle vier de perspectieven, uitgewerkt tot concrete SMART-doelstellingen op operationeel niveau. Je kunt een strategy map maken met tools als Word of Visio.
Natuurlijk zit er een hele denkwijze achter het verhaal van de balanced scorecard en de strategy map. Speciale scorecarding tools hebben die theorie geïmplementeerd en kunnen daarmee tot een grafisch mooier resultaat komen dan 'klassieke' rapportagetools.

9.6.3 *Dashboard*

Scorecards worden vaak in één adem genoemd met dashboards. Ze worden ook nog weleens door elkaar gehaald. Het zijn echter twee verschillende dingen. Figuur 9.7 toont een voorbeeld van een dashboard.
De term dashboard is gekozen naar de analogie met een dashboard van een auto. Je blik moet eigenlijk op de weg gericht zijn. Tegelijkertijd moet je weten hoe de auto presteert. Rijd je niet te hard, komen de toeren van de motor niet te hoog, heb je nog genoeg benzine? Een korte snelle blik op het dashboard van de auto kan deze informatie verschaffen. Een korte snelle blik is alles wat je hebt voordat je weer op de weg moet kijken. Het dashboard is een verzameling van meters en getallen van voor jou relevante, maar niet per se aan elkaar gerelateerde zaken.
Hetzelfde geldt voor een Business Intelligence-dashboard. Het is een verzameling van verschillende, voor de gebruiker relevante zaken. Vaak vormen

scorecards een onderdeel van het dashboard en worden de scorecards aangevuld met rapporten, al dan niet in de vorm van grafieken. Vaak staan er ook losse meters op het dashboard die de status van belangrijke KPI's weergeven.



Figuur 9.7 Voorbeeld van een dashboard

Een **dashboard** is een gemakkelijk te lezen, vaak realtime, grafisch overzicht van de status en de historische trends van de KPI's van een organisatie om onmiddellijk gefundeerde besluiten te nemen.

Bewust wordt hier elke keer het woord 'vaak' gebruikt. Het is geen vereiste van een dashboard om de genoemde elementen te bevatten. Eigenlijk is de enige eis dat een dashboard alle, voor een specifiek iemand relevante, informatie verzamelt en bijeenbrengt en op een inzichtelijke intuïtieve manier weergeeft. Daar horen over het algemeen wel visualisaties bij, dus metertjes en plaatjes, maar dat is weer geen harde eis. Andere eisen, dan wel kenmerken, die vaak worden genoemd, zijn interactiviteit van dashboards en het hebben van meer bronnen. Omdat we veel verschillende soorten informatie samenbrengen, zal een dashboard wel (bijna) altijd hoog geaggregeerd zijn. Voor heel gedetailleerde informatie is er geen ruimte.

Interactief

Sommige mensen zeggen dat een dashboard interactief moet zijn. De gebruiker start met high-levelinformatie, maar kan op de verschillende onderdelen klikken om meer gedetailleerde informatie te krijgen.

Ook hier geldt weer dat er nergens een harde eis bestaat dat dashboards interactief moeten zijn. Het hangt deels van de gebruikte tool af waarmee je het

dashboard maakt of die functionaliteit er is. Belangrijker is natuurlijk de soort gebruiker waarvoor het rapport is gemaakt. Krijgt hij/zij relevante informatie aangereikt in de juiste vorm? Doordat de gebruiker moet klikken naar de interessante informatie kan er wrevel ontstaan. Niet door kunnen klikken kan ook wrevel opwekken.

Meer bronnen

Een ander kenmerk dat dashboards kan onderscheiden van gewone rapporten is dat ze vaak uit meer dan één bron putten. Omdat een dashboard een overzicht moet geven, is er een grote kans dat er niet aan elkaar gerelateerde zaken op staan. Uiteraard kunnen al deze zaken geconsolideerd zijn in een datawarehouse. In dat geval kan het datawarehouse dienen als de (enige) bron voor het dashboard.

Daar waar meer operationele gegevens getoond moeten worden, of als er echte realtime-eisen gesteld gaan worden aan het dashboard, wordt de kans groter dat het datawarehouse niet de juiste informatie bevat of dat het te ver achterloopt. In die gevallen zul je zien dat een dashboard meer bronnen combineert.

Beschikbare tooling

Een dashboard zoals getoond in figuur 9.7, of iets dat erop lijkt, is nog wel te bouwen met gewone rapportagetools of met bijvoorbeeld Excel. Dat gezegd hebbende: een rapport dat eruitziet als een dashboard is nog niet meteen een dashboard. Bij de tool-keuze moet een bedrijf voornamelijk kijken naar het verwachtingspatroon van de beoogde gebruikers. Inhoudelijk moet het dashboard kloppende informatie tonen. Maar het moet ook mooi zijn, makkelijk en intuïtief te gebruiken en up-to-date.

9.6.4 Rapporten

Naast dashboards en scorecards zijn er nog de 'gewone' rapporten. Een rapport is een verzameling gegevens die zodanig is opgemaakt en weergegeven dat deze voor de gebruikers bruikbare informatie oplevert. Heel vaak zijn dit gewoon lijstjes. Bijvoorbeeld een lijst van grootboekrekeningen met de bijbehorende getallen. Of voor een zorginstelling een lijst van alle verrichte handelingen in een periode. Misschien dat zo'n lijst nog groeperingen kent, zoals alle verrichtingen van hetzelfde verzorgingstehuis gegroepeerd bij elkaar tonen.



Een **rapport** is een verzameling gegevens die zodanig is opgemaakt en weergegeven dat deze voor de gebruikers bruikbare informatie oplevert.

Uiteraard kunnen rapportagetools tegenwoordig veel meer dan alleen het maken van simpele lijstjes. Veelvuldig wordt gebruikgemaakt van draaitabellen.

Grafieken zoals lijngrafieken, balkgrafieken, taartpunten en andere varianten zullen in geen enkele tool ontbreken. Plaatjes zeggen immers meer dan woorden. Let daar wel weer op de beoogde gebruiker. Een analytisch ingestelde controller met een wiskundige achtergrond ziet wellicht veel liever cijfers. Een salesmanager met een alfa-achtergrond zou wel eens meer kunnen hebben aan de grafiek.

Parameters

Veel rapporten zullen gebruikmaken van parameters, filters of slicers. Teamleiders zouden bijvoorbeeld behoefte kunnen hebben aan een overzicht van het ziekteverzuim van hun mensen over de afgelopen periode. Je zou voor elk team in de organisatie zo'n rapport kunnen maken. Het is handiger om één rapport te maken en de gebruiker te laten kiezen van welk team informatie getoond moet worden. Die keuze wordt mogelijk door het gebruik van parameters. Je kunt hier ook kiezen voor de implementatie van Row Level Security (RLS), waarbij een ingelogde gebruiker automatisch zijn/haar eigen afdeling te zien krijgt omdat ze de gegevens van andere afdelingen niet mogen zien.
In het voorgaande voorbeeld wordt ook gesproken over de afgelopen periode. Dat kan natuurlijk de ene keer een maand zijn, de andere keer een kwartaal en de derde keer een zelfgekozen periode met willekeurige begin- en einddatum. Ook hier geven parameters weer de flexibiliteit om de verschillende wensen op te lossen met een en hetzelfde rapport. Dat betekent minder ontwikkel- en onderhoudswerk, maar ook minder kans op wildgroei.

Wildgroei

Een bekend fenomeen met rapporten is de wildgroei van beschikbare rapporten. Mensen bedenken informatiebehoeften en dienen een verzoek in voor een nieuw rapport. Dit rapport wordt ontwikkeld en in productie genomen. Vaak controleert niemand of er al een vergelijkbaar rapport bestaat. Soms is er zelfs al een (zo goed als) identiek rapport. Soms kan in de vraag worden voorzien door een bestaand rapport iets aan te passen, bijvoorbeeld door het van een extra parameter te voorzien.
Hoe meer rapporten er komen, hoe groter de kans dat mensen door de bomen het bos niet meer zien. Veel nuttige rapportages blijken er wel te zijn, maar de beoogde gebruikers weten het niet. De kans is groot dat daarom verzoeken om nieuwe rapporten komen. Als je voor elke nieuwe user story een nieuw rapport maakt, zul je snel duizenden rapporten hebben en zal niemand nog overzicht hebben over al de beschikbare rapporten.
Metadata over de rapporten kan hier helpen. Bijvoorbeeld een rapport dat een lijst toont van alle bestaande rapporten, inclusief een korte beschrijving van de gegevens en het doel van het rapport. Daarnaast is informatie over hoe vaak het rapport wordt gebruikt, door wie en wanneer dat voor het laatst is gebeurd, erg nuttig. Niet-gebruikte rapporten kunnen (wellicht) weggegooid worden, andere rapporten kunnen gecombineerd worden en weer anderen komen op een andere plek misschien beter tot hun recht (worden beter gevonden door gebruikers).

9.6.5 Security

Een onderwerp dat al vroeg in de informatieanalyse de aandacht moet krijgen is security. Vaak wordt security als belangrijk aangeduid, maar bijna als een after-thought geïmplementeerd. Bij de tool-keuze moeten de securityvereisten meegenomen worden. Bij datawarehouse design, bij de opzet van een Data Lake en bij BISM-design speelt security een rol en bij de ontsluiting van de data al helemaal. Voor rapporten en dashboards valt security uiteen in twee onderdelen. Allereerst moet de vraag beantwoord worden wie welke rapporten mag gebruiken. De tweede vraag is welke gegevens er op het gebruikte rapport getoond mogen worden. Technisch moet er ook nog gekeken worden met welke security credentials de op een rapport getoonde gegevens uit de gebruikte bron gehaald worden.

Wie mag welk rapport gebruiken?

De eerste vraag – wie een rapport mag gebruiken – is relatief het makkelijkst te beantwoorden. Er is bijvoorbeeld een aantal managementrapportages dat alleen bedoeld is voor het management. Iedereen die niet in het managementteam (MT) zit, mag deze rapporten niet inzien. Vaak is dat technisch gemakkelijk op te lossen door de rapporten in een afgeschermde folder te plaatsen (op een fileshare of in een portal). Alleen mensen met de juiste rechten kunnen bij de rapporten in deze folder.

Wie mag welke inhoud zien?

De tweede vraag (wie mag welke inhoud zien?) ligt iets genuanceerder. In het stukje over parameters werd gesuggereerd dat je door een parameter aan een rapport toe te voegen maar één rapport hoeft te maken om alle teamleiders te voorzien van informatie over ziekteverzuim binnen hun team. Houdt dat automatisch in dat teamleiders ook de mogelijkheid hebben om de gegevens van andere teams dan hun eigen team te bekijken? Zo ja, is dat gewenst?

Bovenstaande valt onder de term row level security. Het zou een functionele eis kunnen zijn dat iedereen het ziekteverzuim mag zien van die mensen die in de organisatiehiërarchie onder hen vallen, direct of indirect. Elke manager en teamleider mag het rapport dus gebruiken, maar iedereen moet wel een inhoudelijk andere lijst krijgen, gebaseerd op zijn plek in de organisatie.

Veel modellen zoals besproken in hoofdstuk 8 kennen dit soort security. Het model kijkt wie de vraag stelt en geeft automatisch alleen de relevante rijen terug. Uiteraard moet het model dan wel te horen krijgen wie het rapport op dit moment opvraagt.

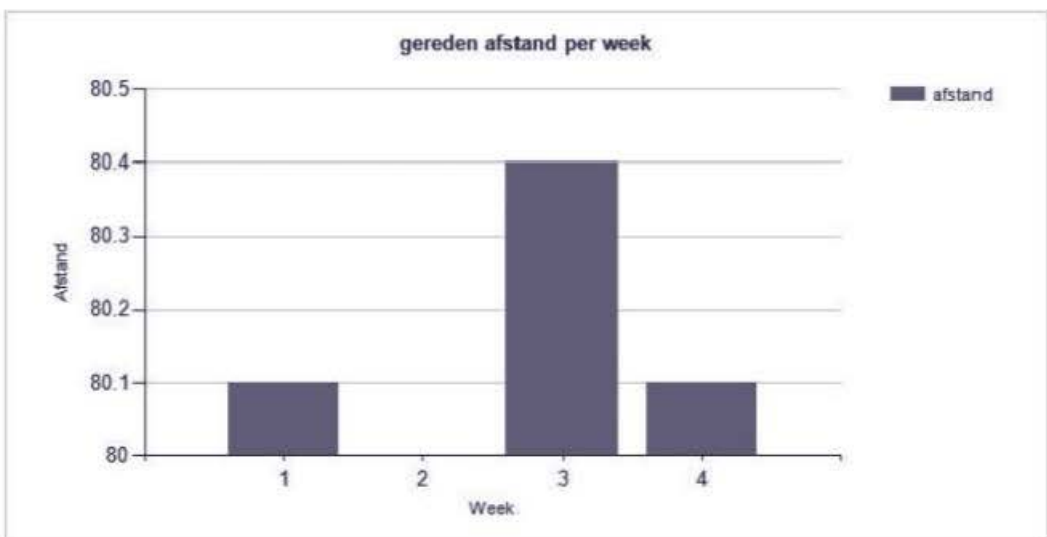
Niet alle databases kennen dit soort security. Je mag gegevens uit een tabel lezen, of niet. Instellen dat sommige rijen oké zijn en andere niet is dan vaak niet mogelijk. In dat geval moet je deze vorm van security inbouwen in de rapporten zelf. Afhankelijk van de gebruikte tool zijn daar verschillende opties voor.

Nu je de verschillende termen op een rij hebt, is het tijd om meer in detail te kijken naar visualisaties.

9.7 Ontwerpen van (goede) rapporten

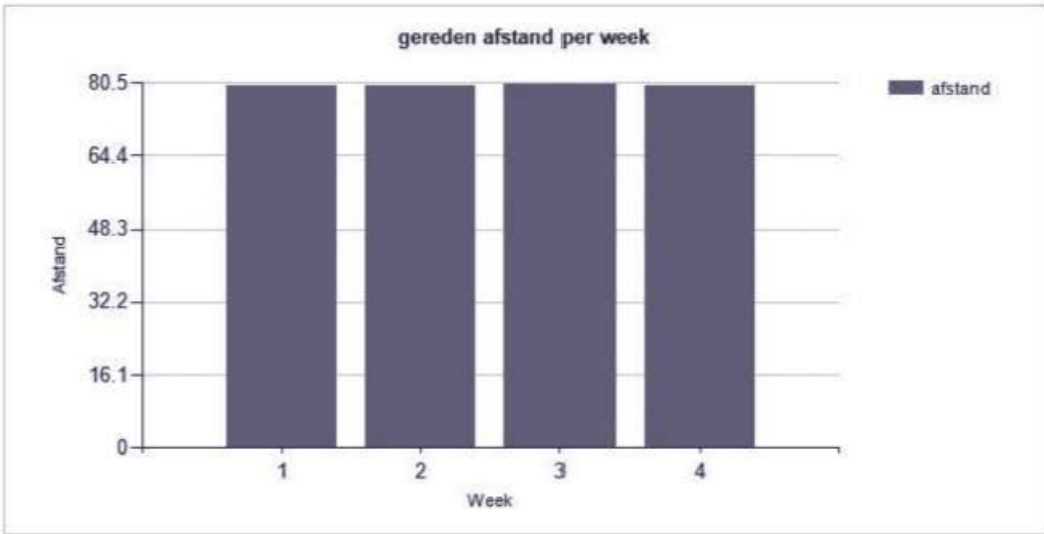
9.7.1 Inleiding rapportontwerp

Lang heb ik gedacht dat rapporten maken het makkelijkste onderdeel is van het bouwen van een Business Intelligence-oplossing. Het bouwen van goede rapporten is echter een vak apart dat je niet moet onderschatten. Neem bijvoorbeeld het rapport van figuur 9.8. Ik zag het getoonde grafiekje ooit op een sociale media-site waar sporters hun prestaties bijhouden. Ik had met oud en nieuw een gps-kilometerteller gekregen en bekeek eind januari mijn prestaties. Mijn conclusie was onmiddellijk dat er van de site niets klopte. Ik had immers in vier opeenvolgende weken precies hetzelfde rondje gefietst en dus precies dezelfde afstand gereden. Mijn verwachting was vier even hoge staven te zien!



Figuur 9.8 Rapport met aangepaste y-as

Een nadere, wat nauwkeuriger, blik op het rapport maakte mijn fout duidelijk. Mijn aanname was dat de y-as een schaal beginnend bij 0 toonde. De schaal ging echter van 80 tot 80,5. Daarmee gaat de grafiek in tegen een belangrijk ontwerpprincipe voor staafdiagrammen: de schaal begint altijd bij nul! Een staafdiagram laat goed en snel het relatieve verschil tussen waarden zien. Zo wordt een staafdiagram door de meeste mensen intuïtief ook geïnterpreteerd. Met een slecht ontworpen grafiek loop je het risico dat de informatie verkeerd wordt geïnterpreteerd en misschien wel desinformatie oplevert. Figuur 9.9 toont dezelfde gegevens op een wijze die waarschijnlijk beter wordt geïnterpreteerd. Als je de verschillen van week tot week wilt benadrukken, is een staafdiagram het verkeerde type visualisatie.



Figuur 9.9 Rapport met standaard y-as

Dit voorbeeld laat zien dat het kiezen van een verkeerde visualisatie, of het kiezen van een verkeerde schaal, kan leiden tot foutief geïnterpreteerde informatie. Dat leidt weer tot verkeerde beslissingen waarmee wij ons hele doel voorbijschieten. Figuur 9.5 is een ander voorbeeld. De scorecard van figuur 9.5 is in het boek een groot deel van zijn meerwaarde verloren omdat het zwart-wit gedrukt is. Het origineel, waar alle indicatoren groen, geel en rood gekleurd waren, gaf een heel andere eerste indruk. Realiseer je echter dat 10 procent van de mannen kleurenblind is en dat de eerste kleuren die ze waarschijnlijk niet goed kunnen onderscheiden precies de kleuren groen en rood zijn. Een groot deel van je doelgroep ziet alle rapporten dus in grijstinten! En dan heb ik het nog niet over dat bijvoorbeeld in China de kleur rood symbool staat voor geluk.

9.7.2 *Attentieve verwerking*

Belangrijk bij het ontwerpen van rapporten is het onderscheid tussen preattentive processing (preattentieve verwerking) en attentive processing (attentieve verwerking). Hoewel preattentive een slecht gekozen naam is, blijft hij wel in gebruik (zie *https://www.csc2.ncsu.edu/faculty/healey/PP/*). Het verwijst naar het feit dat mensen visuele zaken bijna meteen kunnen waarnemen, schijnbaar nog voordat ze er hun aandacht echt op richten. Op een bladzijde met tekst waar alle woorden in het zwart gedrukt zijn en één woord in het rood, valt onze aandacht ogenblikkelijk op het rode woord. In een tekening met alleen rondjes, zie je het ene vierkant direct. De verwerkingssnelheid van mensen is bij deze preattentieve verwerking slechts een fractie van een seconde.
De tegenhanger van preattentieve verwerking is attentieve verwerking. Attentieve verwerking is het heel bewust zoeken naar informatie. We zouden bijvoorbeeld kunnen vragen hoe vaak het woord 'is' voorkomt in deze alinea. Zonder visuele hulp zal het seconden duren om de hele alinea te scannen en

bewust het woord 'is' te filteren en te tellen. Dat is een voorbeeld van attentieve verwerking. Hadden we echter alle woorden waar het om ging in het rood afgedrukt of cursief gedrukt, dan had je het weer bijna onmiddellijk gezien: preattentieve verwerking!
De boodschap in het kader van dit hoofdstuk moge duidelijk zijn: door iets simpels als een afwijkende kleur kun je de aandacht van iemand onmiddellijk laten uitgaan naar datgene wat jij als belangrijkste beschouwt op jouw rapport. Dat ene getal dat een andere kleur heeft of een ander lettertype heeft, valt meteen op. Maar de werking is heel beperkt. Als er veel mooi gekleurde grafieken op een rapport staan, valt dat ene rode cijfer helemaal niet meer op tussen al die andere kleuren.

Kleur is in dit voorbeeld een attentief attribuut. De afwijkende kleur maakt dat mensen zonder zich te hoeven inspannen, extreem snel dat ene belangrijke element zien. Andere voorbeelden van attentieve eigenschappen zijn:

- lengte, breedte;
- oriëntatie;
- vorm;
- grootte;
- kader;
- kleur/intensiteit van de kleur;
- 2D-positie.

Van lengte maken we vaak gebruik, bijvoorbeeld bij staafdiagrammen. Door de afwijkende lengte van de individuele balken zien de meeste mensen in één oogopslag de langste en kortste staaf in de grafiek, zeker als er een echte uitschieter bij zit.
Bij oriëntatie kun je bijvoorbeeld denken aan de trend van een KPI. Een pijl die naar boven wijst (opwaartse trend) staat symbool voor goed, een horizontale pijl is neutraal en een neerwaartse pijl is negatief. Drie identieke pijlen die alleen door een andere richting een andere boodschap overbrengen.
Bij vorm zou je kunnen denken aan een scatterplot waarbij je bijvoorbeeld opleidingsniveau van mensen uitzet tegen hun BMI om te kijken of hogeropgeleide mensen meer of minder vaak een te hoog BMI hebben dan lageropgeleiden. Een vervolgvraag kan zijn: is er hierbij verschil tussen mannen en vrouwen? Je zou dan elk punt in je scatterplot dat een man voorstelt kunnen vervangen door een kruisje en van vrouwen maak je dan bijvoorbeeld een rondje. De vorm geeft het verschil aan en je ziet snel of kruisjes en rondjes willekeurig door elkaar staan (geen verschil) of dat er duidelijke clustering is van rondjes enerzijds en kruisjes anderzijds. De plek van elk symbooltje, ofwel zijn 2D-positie, laat gemakkelijk een overall trend (of het ontbreken daarvan) zien tussen opleiding en BMI.
Bovenstaande zijn voorbeelden van de bekende uitspraak: een plaatje zegt meer dan duizend woorden. Als je veel data hebt, kun je door het kiezen van een slimme visualisatie en door gebruik te maken van preattentieve attributen, heel heldere en gemakkelijk interpreteerbare rapporten maken. Omgekeerd is ook

waar: doe je het minder handig, dan komt je boodschap niet over of wordt hij verkeerd geïnterpreteerd.

Kwantitatief of categoriaal

Preattentieve eigenschappen zijn dus in staat om snel onze aandacht te trekken. Maar dan? Wat vertellen ze ons? Voor het beantwoorden van die vraag maken we onderscheid tussen wel of niet kwantitatief. De lengte van balken is een voorbeeld van een attribuut dat mensen gemakkelijk kwantitatief interpreteren. Een langere balk komt overeen met een groter getal. Sterker nog: een tweemaal zo lange balk zal geïnterpreteerd worden als overeenkomstig met een tweemaal zo groot getal.

Dit is waar het voorbeeld van figuur 9.8 fout ging. Door niet bij nul te beginnen, verliezen de staven hun natuurlijke manier van een kwantitatieve waarde weergeven. De natuurlijke interpretatie van de lezers verandert niet mee en dus 'lezen' we de verkeerde boodschap.

Kijk eens naar de visualisaties van figuur 9.10 en 9.11. Beide grafieken tonen een (door mij gefietste) afstand in kilometers per land. Grafiek 9.10 vult landen op en de intensiteit van de kleur is een maat voor de afstand. In donkerder gekleurde landen is meer kilometer gereden. Het is moeilijk (of eigenlijk gewoon onmogelijk) om te zeggen wanneer de intensiteit van de kleur tweemaal zo groot is als een andere intensiteit. Daarmee wordt het onmogelijk om te zeggen in welk land tweemaal zoveel kilometer gefietst is. Een legenda zou iets helpen, maar niet veel.



Figuur 9.10 Filled Map

In figuur 9.11 worden precies dezelfde gegevens getoond. Nu is echter de grootte van de cirkel de maat voor hoeveel afstand er is gereden. De oppervlakte van de cirkel is nu de maat voor afstand. Maar welke cirkel is groter, die in Australië of die in Amerika? En is Frankrijk tweemaal zo groot als Duitsland?



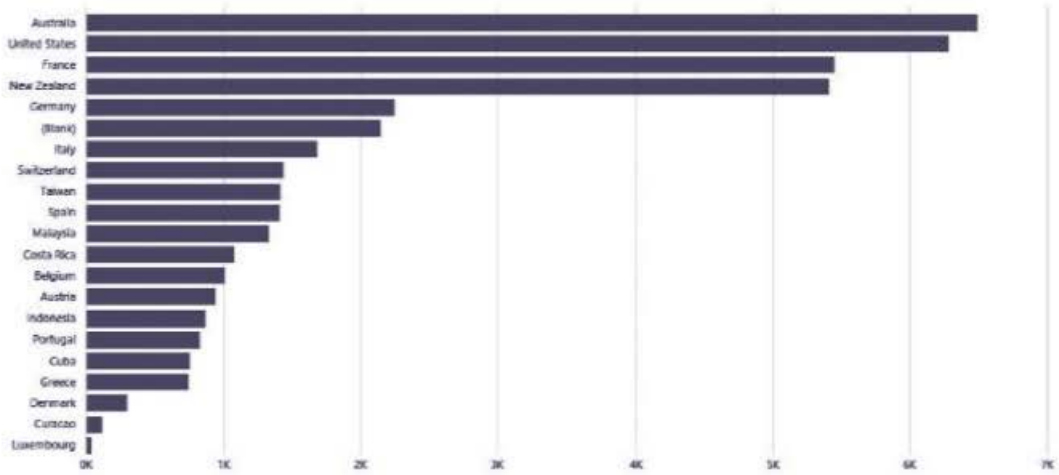
Figuur 9.11 Kaart met taart- of cirkeldiagram

Uit bovenstaande voorbeelden blijkt dat we kleur en grootte van objecten (cirkels) kunnen gebruiken om kwantitatieve zaken weer te geven. Beide zijn echter veel slechter in die weergave dan de lengte van een staafdiagram. Figuur 9.12 gooit een derde element in de strijd. In een tree map is de grootte van het oppervlak weer overeenkomstig de waarde die je wilt weergeven. Daarbij komt dat grotere vlakken meer linksboven staan en kleinere rechtsonder. Verticaal is belangrijker dan horizontaal. De plaats van een vlak compenseert op zo'n manier dat het oppervlak alleen eigenlijk te weinig zegt. De gebruiker moet echter wel weten hoe hij een tree map kan lezen. En dat doet hij vervolgens bewust, en dus niet meer preattentief.



Figuur 9.12 Tree map

Misschien is een ouderwets staafdiagram (Engels: bar chart) nog het beste in staat om de boodschap van bovenstaand voorbeeld over te brengen. Je moet echter ook rekening houden met het verwachtingspatroon van je gebruikers. Gebruikers willen mooie rapporten. Mooie visualisatie en infographics (informatie grafisch vormgegeven). Bijvoorbeeld door per land een aantal fietsjes te tekenen waarbij meer fietsjes staat voor meer gereden kilometers.



Figuur 9.13 Staafdiagram

Naast kwantitatieve informatie hebben we ook categoriale informatie. Bij categoriale informatie gaat het om een logische verdeling van gegevens. Of het in categorieën indelen van gegevens. Neem bijvoorbeeld het eerder genoemde voorbeeld van een scatterplot, waarbij het onderscheid tussen mannen en vrouwen wordt gemaakt door de vorm van het gebruikte symbool, een rondje of een kruisje. Of de zojuist genoemde voorbeelden met afstand per land. Afstand is een nominale/numerieke variabele, land is een categoriale variabele.

Van de attentieve attributen die eerder werden genoemd zijn eigenlijk alleen lengte en 2D-positie duidelijk als het gaat om overbrengen van kwantitatieve informatie. Zoals uit figuur 9.10 en 9.11 blijkt, kunnen ook kleur en grootte kwantitatieve informatie overbrengen, maar minder duidelijk en daarmee niet meer echt preattentief. Dat geldt ook voor breedte.

Voor de attributen oriëntatie, vorm, kader en kleur geldt dat ze voornamelijk handig zijn voor categoriale informatie. Dus bijvoorbeeld een andere kleur voor een ander land of voor een ander geslacht.

Al het voorgaande komt samen in een aantal richtlijnen. Wat maakt een rapport effectief in de informatieoverdracht die moet plaatsvinden? Wanneer gebruik je welke rapportvorm en met welke visualisatie?

9.7.3 Rapportrichtlijnen

De eerste cruciale vraag bij het maken van een rapport is de vraag welke boodschap overgebracht moet worden. Wat wil je laten zien aan de lezer? Wat kan er geleerd worden van de gegevens? Daarbij kun je onder andere denken aan:

- vergelijking van nominale waarden (nominal comparison);
- chronologische vergelijking (time series);
- positiebepaling (ranking);
- verhoudingen (part-to-whole);
- afwijking (deviation);
- verdeling (distribution);
- correlatie (correlation);
- geografische (geospatial) componenten.

Vergelijking van nominale waarden

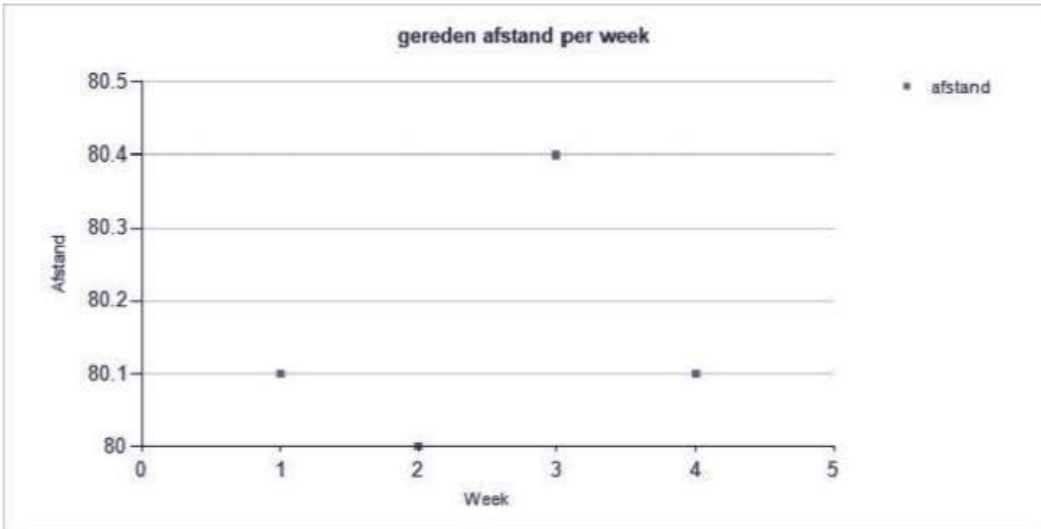
Bij het vergelijken van nominale waarden worden twee of meer numerieke grootheden in absolute zin met elkaar vergeleken. Een rapportage van omzet per maand, bijvoorbeeld. De afstand per week van figuur 9.9 is een ander voorbeeld.

De effectiefste manier om nominale waarden te vergelijken is het staafdiagram (bar chart). Er kunnen subtiele verschillen bestaan tussen een horizontaal en verticaal staafdiagram, maar in veel gevallen maakt dat niet veel uit. Liggende staven blijken iets effectiever dan staande, tenzij je een waarde uitzet tegen periodes, zoals de omzet per maand.

Een staafdiagram brengt eigenlijk twee zaken tegelijkertijd eenvoudig in beeld. Door getallen langs de as te zetten (of eventueel af te drukken binnen een staaf) ziet de gebruiker gemakkelijk de echte nominale waarde. Door het attribuut lengte van de staaf kan de gebruiker ook meteen intuïtief de verschillende getoonde nominale waarden ten opzichte van elkaar vergelijken. Welke is de grootste, welke de kleinste en wat zijn de verhoudingen?

Een staafdiagram kan ook gemakkelijk uitgebreid worden met extra categoriale informatie. In het voorbeeld van figuur 9.9 zouden bij elke week twee verschillend gekleurde staven kunnen staan: één voor de afstand gereden op de mountainbike en de ander voor de afstand gereden op de racefiets. Wil je voornamelijk het totaal per week laten zien, dan maak je een gestapeld staafdiagram (stacked bar chart). Is de verhouding tussen mountainbike en racefiets belangrijker, dan zet je de staven naast elkaar (clustered bar chart).

Naast het staafdiagram zou een diagram met punten ook goed werken. Punten vereisen geen gemeenschappelijke nullijn, waardoor je beter in staat bent om de verschillen te benadrukken. Bekijk nog eens figuur 9.8 en het als beter voorgestelde alternatief 9.9. Als je de verschillen van week tot week wilt benadrukken, lijkt 9.9 niet een heel overtuigend alternatief. Figuur 9.14 is misschien beter.



Figuur 9.14 Afstand per week

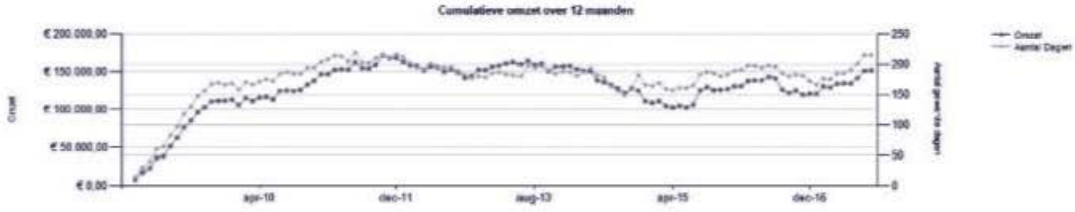
Omdat de punten allemaal gelijk zijn, bepaalt alleen hun relatieve positie de informatie. En voor relatieve positie maakt de schaal van de as niet uit. Een staafdiagram met het verschil ten opzichte van gemiddeld, met kleinere waarden getoond als staaf die naar beneden wijst, is ook een mooi alternatief.

Chronologische vergelijking

Bij een chronologische vergelijking zijn we geïnteresseerd in het verloop van een meetwaarde over de tijd gemeten. Meestal houdt dat in dat we de tijd op de horizontale as van een grafiek zetten en de meetwaarde waarin we inzicht willen hebben op de y-as. Vaak willen we weten of er een patroon is, bijvoorbeeld een opwaartse of neerwaartse trend of iets als seizoensinvloeden.

Een staafdiagram is zeker een optie. Voor een grafiek die omzet per maand toont, zetten we de twaalf maanden op de horizontale as met staven voor omzet. Eventueel kunnen we het staafdiagram nog stacked of clustered maken als we de omzet nog willen uitsplitsen naar bijvoorbeeld omzet per productcategorie. Een staafdiagram is echter niet de meest geschikte visualisatie als het gaat om het detecteren van een patroon. Een lijngrafiek is dan beter. Door de meetwaarden als punten op de coördinaten van de grafiek te zetten en vervolgens te verbinden, wordt de nadruk verlegd van de grootte van een staaf (en daarmee de nominale waarde) naar de onderlinge samenhang. Is er een stijgende trend? Is er een terugkerend patroon met bijvoorbeeld een dip in de zomermaanden?

In figuur 9.15 wordt een totaal van twaalf maanden getoond. Over de jaren heen is dit redelijk stabiel met een kleine dip rond begin 2015. Voorafgaand aan dat dipje is de verhouding tussen dagen en omzet anders dan in de rest van de beschreven periode.

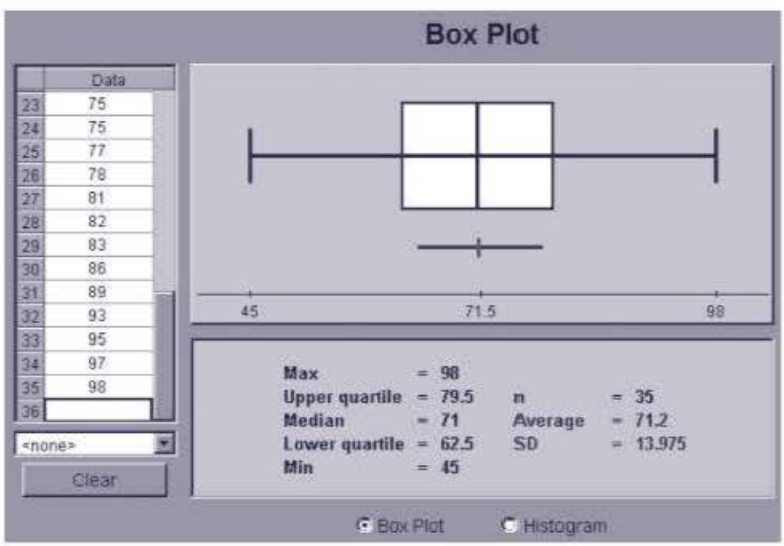


Figuur 9.15 Lijngrafiek

Bij een lijngrafiek is het van belang dat de periodes opeenvolgend zijn. Als er bijvoorbeeld maanden ontbreken en je daardoor met een lijn het meetpunt van januari rechtstreeks verbindt met dat van juli, kan er een heel verkeerd beeld ontstaan. Je ziet bijvoorbeeld een sterk stijgende lijn terwijl er eigenlijk sprake is van een licht oplopende trend. Of misschien is juli wel een uitschieter in een verder stabiele (horizontale) lijn en wordt die uitschieter geïnterpreteerd als een oplopende trend die er helemaal niet is. Enkelvoudige punten, niet verbonden door een lijn, zijn dan beter.

Als je inzichtelijk wilt maken hoe de verdeling van data is over de tijd, kun je een serie van boxplots maken. Als je bijvoorbeeld wilt weten hoe salarissen zich in Nederland hebben ontwikkeld over de afgelopen jaren, vertelt een lijngrafiek met daarin het modaal salaris niet het hele verhaal. Je wilt ook inzichtelijk krijgen of en hoeveel het verschil tussen arm en rijk is toegenomen. Een boxplot kan dit laten zien.

Figuur 9.16 legt de boxplot uit. De helft van alle waarden ligt in de box, het witte kader in het midden. De uiteinden van de lijn geven respectievelijk de kleinste en grootste waarde aan. En de verticale lijn in het kader is de mediaan. Voor de mediaan geldt dat evenveel waarden groter zijn als dat er waarden kleiner zijn.



Figuur 9.16 Boxplot

Zet je in een grafiek tien boxplotdiagrammen naast elkaar waarbij elk boxplot de salarissen in Nederland weergeeft in een specifiek jaar, dan kun je zien hoe salarissen zich ontwikkelen over de tijd. Als elke volgende boxplot iets hoger staat, betekent dat dat de salarissen in hun geheel stijgen. Als de boxplots identiek zijn, blijft de verdeling gelijk. Als de mediaan opschuift naar links binnen het kader, liggen er heel veel salarissen aan de onderkant dicht bij elkaar en is de spreiding (het verschil) groter bij hogere salarissen. Er worden dus vaker lagere salarissen betaald dan hoge. Dit soort informatie kan belangrijk zijn voor economen en politici.

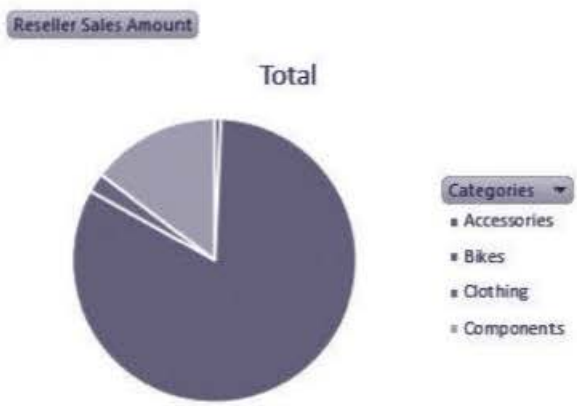
Positiebepaling

Bij positiebepaling wil je weten hoe verschillende zaken scoren ten opzichte van elkaar. Wat zijn mijn tien best verkopende producten? Welke vijf verkopers halen de hoogste omzet binnen?

Eigenlijk is dat grotendeels een nominale vergelijking. De nominale waarde bepaalt immers wat beter of slechter scoort en daarmee de positie (eerste, tweede, derde, ...). Het staafdiagram is dan ook weer de beste keuze. Alleen sorteer je het staafdiagram op de meetwaarde die goed en slecht, en dus de volgorde, bepaalt en niet op de categoriale waarde die de verschillende staven vertegenwoordigen.

Figuur 9.10 is een goed voorbeeld hiervan. De grafiek had de verticale as kunnen sorteren op alfabet. Dat is handig als je snel een specifiek land wilt zoeken en de bijbehorende meetwaarde. Wil je echter gemakkelijk zien wat de landen zijn waar ik het meest, respectievelijk het minst heb gefietst, dan is sortering op afstand beter.

Verhoudingen



Figuur 9.17 Pie chart

In figuur 9.17 zie je een pie chart (taartdiagram of cirkeldiagram). Pie charts, hoewel vaak gebruikt om de onderlinge verhoudingen te tonen van een meetwaarde uitgesplitst naar een categoriale kolom, worden over het algemeen afgeraden. Probeer maar eens de vraag te beantwoorden hoeveel de categorie 'Bikes' groter is in omzet dan de categorie 'Components'. Een pie chart oogt

leuk, maar het is heel moeilijk om daar goede informatie uit te halen. Met twee categorieën valt het misschien nog mee, maar met drie kunnen significante verschillen al onzichtbaar worden.
Een dergelijke vraag wordt een stuk beter beantwoord door een staafdiagram waar je niet de nominale waarde van (in dit voorbeeld) omzet toont, maar het percentage van elke categorie als de nominale waarde gebruikt. Het nadeel is dat een staafdiagram niet intuïtief wordt geïnterpreteerd als verhoudingen ten opzichte van het geheel, waardoor ook op deze manier de echte boodschap verloren gaat. Een goede titel aan de grafiek geven zou daar een beetje bij kunnen helpen.
Een andere truc kan zijn de staven tegen elkaar aan te zetten in plaats van er een beetje witruimte tussen te laten. Door ze tegen elkaar aan te zetten zijn mensen meer geneigd ze te interpreteren als onderdeel van één geheel.
Gestapelde staafdiagrammen zijn ook nog een mogelijkheid. Ze hebben helaas al vrij snel dezelfde tekortkoming als pie charts.

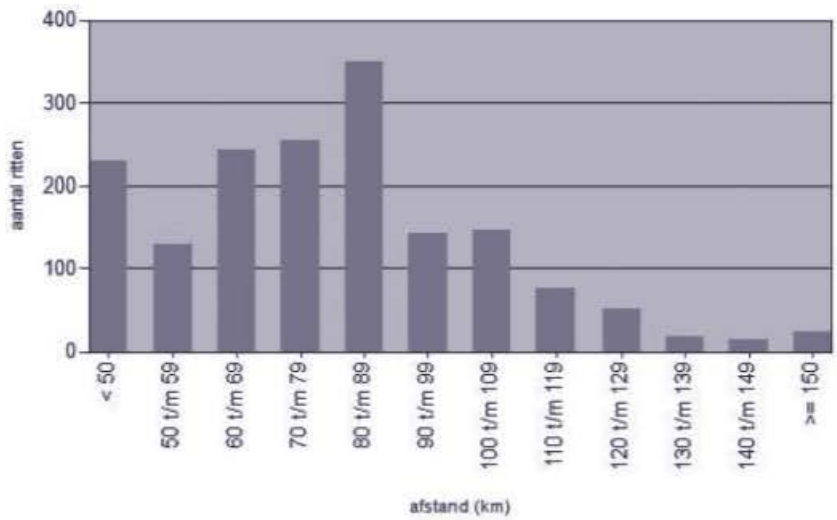
Afwijking

Met afwijking bedoelen we hier bijvoorbeeld het inzichtelijk maken hoeveel een gerealiseerde waarde verschilt van de vooraf gestelde doelstelling of van dezelfde waarde in het voorgaande jaar.
Een mooie manier om dat inzichtelijk te maken kan een staafdiagram zijn met de horizontale as in het midden in plaats van onderaan de grafiek. Je berekent dan eerst het verschil tussen de echte waarde en de doelstelling en zet dat verschil uit in het staafdiagram. Negatieve verschillen worden balken die naar beneden wijzen, positieve verschillen zijn balken die naar boven wijzen. Per balk wordt een andere categoriale waarde weergegeven.
Vaak zal er ook een chronologische component in deze vraag zitten: hoe verandert het verschil tussen de waarde en de doelstelling over de tijd? Je kunt nu een lijngrafiek gebruiken met weer een horizontale as in het midden. Deze stelt de doelstelling voor en representeert 100%. Als de echte waarde precies op deze lijn zit, heb je precies je doelstelling gehaald. Alles wat je erboven zit is wat je meer gerealiseerd hebt en onder de lijn is wat je minder hebt gehaald. De getoonde meetwaarde wordt dus uitgezet als een percentage van de doelstelling.
Afwijking ligt dicht aan tegen KPI's. Bij KPI's is de vraag vaak gereduceerd van 'hoeveel beter of slechter presteer ik dan verwacht?' tot 'presteer ik beter of slechter dan verwacht?'. Daarvoor volstaat een enkelvoudig icoontje, een smiley of een stoplicht. Dit soort indicators oogt vaak mooi en door positie en of kleur slim te kiezen, kun je de aandacht naar de KPI trekken. Let wel op dat je dat per rapport maar een of twee keer kunt doen. Een rapport met honderd smileys die schreeuwen om aandacht, zal amper succesvol informatie overdragen aan de lezer.

Verdeling

Een veelgebruikte visualisatie voor dataverdeling (distribution) is de boxplot. Die is al besproken bij de chronologische verwerking.

Een boxplot kan als nadeel hebben dat mensen niet weten hoe ze hem moeten lezen. Termen als mediaan en 25e percentiel zijn bij veel mensen niet bekend. Dan gaat de informatie in een boxplot al snel verloren. Voor statistici is de boxplot een heel mooi middel om snel belangrijke informatie over een verdeling te krijgen.
Waarschijnlijk vaker gebruikt omdat het makkelijker te interpreteren is, is het histogram. Een histogram is een soort staafdiagram. Op de horizontale as staat echter niet een categoriale variabele zoals maand of productcategorie, maar een numerieke meetwaarde. Dit is de meetwaarde waarvan je de gegevensverdeling wil weten. Hoe vaak komt een bepaalde meetwaarde eigenlijk voor? Komen alle waarden ongeveer even vaak voor (normaalverdeling) of is bijvoorbeeld de onderkant begrensd (minimumloon) terwijl de bovenkant dat niet is (graaiende managers) en is er dus sprake van skew? Om dit goed te laten zien wordt de meetwaarde verdeeld in een aantal ranges (intervallen) en elke staaf toont het aantal meetwaarden dat geteld is in dat ene interval.



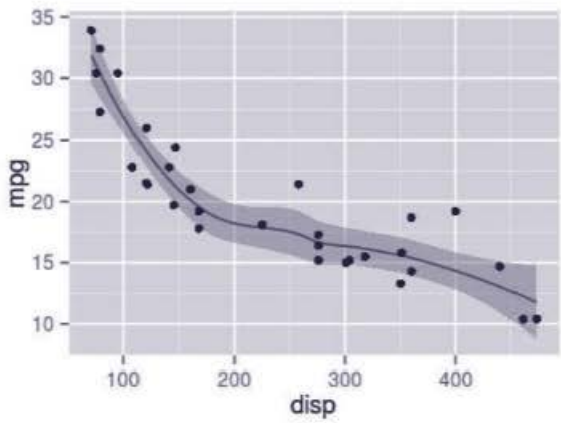
Figuur 9.18 Histogram

Omdat een histogram erg lijkt op een staafdiagram zijn mensen geneigd om de absolute aantallen die de staven vertegenwoordigen te gaan bekijken. Meestal gaat het niet zozeer om absolute aantallen, maar om de onderlinge verhoudingen. Een mooi voorbeeld is een internetwinkel die wil weten hoeveel dagen na de order de producten bij de klant zijn. Eén gemiddelde (gemiddeld na twee werkdagen) zegt dan heel weinig. Hoeveel orders er na een, twee of drie dagen zijn geleverd, wat een histogram mooi toont, is eigenlijk niet wat ik wil weten. Een frequentiepolygoon (frequency polygon) laat goed zien wat de verdeling is van hoeveel procent van de orders na hoeveel dagen geleverd wordt. Het aantal dagen staat op de horizontale as, het percentage orders dat na zoveel dagen geleverd wordt op de verticale as, en een lijn laat zien hoeveel procent van de orders wordt geleverd op de hoeveelste dag na de order. Dat kan ook nog cumulatief waarbij je de lijn laat oplopen tot 100%.

Correlatie

Correlatie zegt iets over een afhankelijkheid tussen twee variabelen. In de lijst met rapporten die we hier bespreken, gaan we uit van een afhankelijkheid van twee numerieke, nominale variabelen en niet zozeer tussen een nominale en een categoriale waarde.

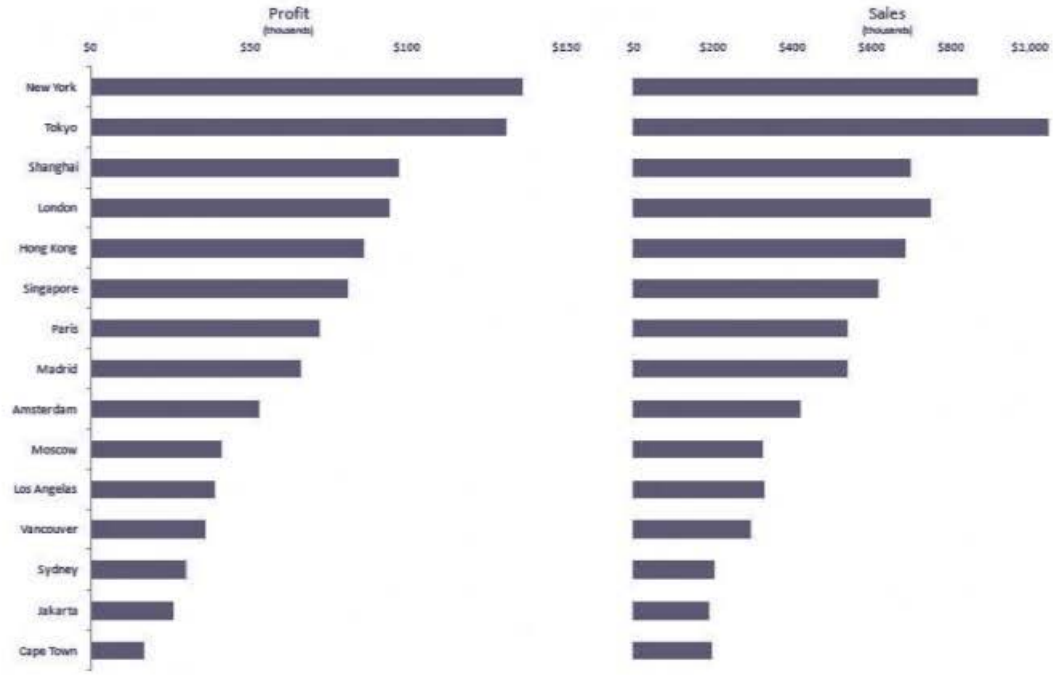
Het spreidingsdiagram (scatterplot) is de meest gebruikte grafiekvorm voor correlatie. In figuur 9.19 zie je een voorbeeld. Op de horizontale as staat de inhoud van een automotor (displacement) en op de verticale as hoeveel Amerikaanse mijlen de auto kan rijden op één gallon (MPG). Elke punt op het diagram stelt een auto voor. De positie van de punt komt overeen met de specificaties van die auto.



Figuur 9.19 Scatterplot

Het spreidingsdiagram maakt snel inzichtelijk dat auto's met een groter motorvolume minder zuinig rijden dan auto's met een kleiner motorvolume. In het voorbeeld is er nog een 'best-fit'-lijn toegevoegd aan de plot. Deze lijn geeft zo goed mogelijk het verband aan tussen beide variabelen. Daaraan zien we dat het verband sterker is bij wat kleinere motoren en minder sterk wordt bij grotere motoren.

Een alternatief is de zogenaamde table lens. Daarin zet je beide variabelen in hun eigen staafdiagram, vaak met horizontale balken. Het linker staafdiagram sorteer je op de nominale waarde van de variabele die je toont. Het rechter staafdiagram sorteer je op de nominale waarde van het eerste staafdiagram. De tweede toont nu of er wel of niet sprake is van een correlatie. Zie figuur 9.20 voor een voorbeeld. In dit voorbeeld zien we dat winst en omzet gecorreleerd zijn.



Figuur 9.20 Table lens

Geografische componenten

Als laatste is het goed om stil te staan bij informatie met een geografische component erin. Stel dat je omzet per land wil laten zien. Of het gereden aantal kilometers zoals in de figuren 9.10 tot en met 9.13. Het lastige is dat de preattentieve attributen niet werken. Een staafdiagram werkt niet omdat die een gemeenschappelijke nul-as nodig heeft. 2D-positie werkt niet omdat die bepaald wordt door de kaart, de geografische component.

In figuren 9.12 en 9.13 wordt dat opgelost met de tree map of de gesorteerde bar chart. Die laten helaas weer niet goed zien in welk werelddeel er het meest is gefietst. Het is dus een beetje schipperen. Maar de boodschap die overgebracht moet worden is weer de leidraad voor de te maken keuze. Wil je voornamelijk landen onderling vergelijken, dan is het gesorteerde staafdiagram een goede kandidaat. Wil je laten zien waar ter wereld er veel gereden is of juist waar de lacunes zitten, is de landkaart juist heel geschikt. Grootte van objecten is meestal makkelijker te vergelijken dan kleurintensiteit, wat suggereert dat de kaart van figuur 9.11 licht de voorkeur krijgt boven die van 9.10.

De kaart is wel weer relatief geschikt als je meer meetwaarden moet tonen. Je kunt immers landen inkleuren op basis van één gegeven en ook nog per land een cirkel plaatsen met andere grootte gebaseerd op een tweede meetwaarde (en eventueel zelfs de cirkel een andere kleur geven op basis van een derde meetwaarde).

Conclusie

De belangrijkste conclusie die je uit deze paragraaf moet trekken, is dat je goed moet weten welke boodschap je met een rapport wilt overbrengen. In welke categorie valt de vraag die je wilt beantwoorden? Als dat duidelijk is, volgt de keuze voor een specifieke grafiekvorm grotendeels vanzelf. Je eigen specifieke dataset kan nog bepalend zijn voor de laatste keuze. Uiteraard is ook de specifieke rapportagetool die je gebruikt van invloed op de keuze welke visualisatie je gebruikt.

Een wat meer impliciete conclusie is dat je eigenlijk moet weten wie je rapport gaat lezen. Een boxplot is het duidelijkste voorbeeld van de genoemde grafiekvormen waarmee je moet oppassen. Lang niet iedereen begrijpt de informatie die een boxplot weergeeft. Maar ook de getoonde tree map is niet voor iedereen vanzelfsprekend. Wil je dat je boodschap overkomt, dan moet je een visualisatie gebruiken die effectief is voor de doelgroep van het rapport.

Naast de hier genoemde grafiekvormen bestaan er nog vele andere mogelijkheden om informatie over te brengen. Het woord infographics beschrijft in zijn algemeenheid dat we proberen informatie over te brengen op een grafische manier. Een wordcloud is bijvoorbeeld tegenwoordig populair. Met een wordcloud probeer je inzicht te geven in waar een tekst over gaat. Het belangrijkste woord maak je groot, minder belangrijke woorden voor de tekst worden kleiner. Denk bij het gebruik van infographics aan attentief versus preattentief. Weet goed wat je wilt vertellen en probeer in te schatten hoe goed jouw graphic in staat is om die boodschap over te brengen.

9.7.4 Tabel versus grafiek

Misschien wel de huidige trend volgend – en volgens het adagium 'een plaatje zegt meer dan duizend woorden' – hebben we een hele rij van grafiekvormen en hun toepassingen besproken. Daarbij zijn de meest gebruikte rapportvormen, de tabel en de matrix (draaitabel) nog niet eens aan bod gekomen.

De (draai)tabel is altijd het overwegen waard vanwege de eenvoud en de bijbehorende kleine kans op foutieve interpretatie. Iedereen snapt hoe hij/zij een tabel moet lezen. Dat geldt voor een heleboel visualisaties niet. Het lezen van de (draai) tabel is wel een attentieve inspanning en vraagt dus meer van de gebruiker.

Tabellen en draaitabellen zijn primair bedoeld voor het opzoeken van enkelvoudige getallen. Op basis van categoriale informatie, bijvoorbeeld land en jaartal, zoek je gemakkelijk een nominale waarde, bijvoorbeeld omzet, op. Die waarde mag best een geaggregeerde waarde zijn.

Heb je echter meer waarden, bijvoorbeeld de omzet van de laatste tien jaren, en wil je een patroon zien, dan is een grafiekvorm beter geschikt. In dit geval een lijngrafiek omdat het een chronologische vergelijking betreft.

Naast dit alles hebben tabellen en draaitabellen nog enkele mogelijke voordelen:

- precisie;
- verschillende eenheden;
- verschillende aggregatieniveaus.

Precisie

Als je een bedrag tot op de cent nauwkeurig wilt of moet weten, is een grafiek vaak niet handig. In sommige grafieken kun je tekstuele labels aan je grafiek toevoegen, maar dat maakt een grafiek niet altijd leesbaarder. Zonder beschrijvend label laat een staafdiagram heel goed in orde van grootte een waarde zien, maar niet op enkele decimalen nauwkeurig. En dat is soms wel een eis.

Verschillende eenheden

Eerder hebben we positiebepaling (ranking) besproken en gezegd dat een staafdiagram een goede optie is. Maar wat nu als je van de tien beste producten wilt weten hoeveel stuks je hebt verkocht én de gerealiseerde omzet én de behaalde marge als percentage? Aantal, omzet en marge behandelen als categoriale waarden en drie balkjes maken is onzinnig. In een staafdiagram is er een gemeenschappelijke schaal en daarmee een gemeenschappelijke eenheid waarin de meetwaarden worden weergegeven.
In een tabel kun je gemakkelijk vier kolommen maken. De eerste kolom toont de naam van een product en de overige drie kolommen tonen de meetwaarden. Elke meetwaarde kan in zijn eigen eenheid getoond worden zonder dat er verwarring ontstaat of verkeerde suggesties gewekt worden.

Verschillende aggregatieniveaus

Met enige regelmaat is het handig om in één oogopslag zowel gedetailleerde informatie als geaggregeerde informatie te zien. Je zou bijvoorbeeld een omzetrapportage over een jaar kunnen bekijken. Met een staafdiagram toon je gemakkelijk de omzet per maand, maar moet de gebruiker zelf de omzet per jaar uitrekenen. Met alleen de jaaromzet zie je niet of elke maand ongeveer evenveel heeft bijgedragen of dat er wellicht sprake is van een toenemende of afnemende trend. Met een tabel kun je zowel de maandomzetten alsook in een totaalregel de aggregatie tot jaaromzet laten zien.

Conclusie

Een plaatje zegt meer dan duizend woorden. Tegelijkertijd hebben we vaak behoefte aan detailinformatie die zich het beste laat vertalen naar tabellen. Per rapport moet je bedenken wat de effectiefste methode is om je informatie te tonen. Het antwoord hangt af van wie je gebruiker is en welke boodschap je wilt overbrengen.

9.8 Tot slot

In dit laatste hoofdstuk heb je gelezen over analytics en de zichtbare aspecten van Business Intelligence. Terugdenkend aan hoofdstuk 2 gaat dit hoofdstuk over het verrichten van Business Intelligence waar de voorgaande hoofdstukken meer gingen over het inrichten van Business Intelligence. Hoofdstuk 2 zelf ging over het richten van Business Intelligence.
Het front-end is wat gebruikers zien en waar ze mee werken. Daarbij is de vraag wie die gebruikers nou eigenlijk zijn en wat hun behoeften zijn, het eerste wat je je moet afvragen.
Om self-service BI-gebruikers goed te kunnen bedienen, dat wil zeggen van zinvolle en betrouwbare informatie te voorzien, moeten de beschikbare gegevens meestal nog worden bewerkt. In de meest structurele oplossing houdt dat in dat er een datawarehouse komt waarin alle relevante gegevens worden samengebracht. Eventueel wordt er een Master Data-initiatief gestart om eenduidigheid te creëren over de primaire grootheden binnen de organisatie. Een ETL-proces zorgt ervoor dat het datawarehouse geladen wordt met kwalitatief zo goed mogelijke gegevens en in de vorm die de functionele eisen voorschrijven. Door middel van (OLAP-)modellen kunnen we de informatie nu ontsluiten voor de organisatie. Goede front-ends geven gebruikers inzicht (intelligence) in hun business.

Op de website bij dit boek kun je terecht voor de oefenvragen bij hoofdstuk 9. 

Register

Symbolen

3 V's 154

A

accumulating snapshot 137
actieve dataset 183
additief feit 136
afwijking 279
aggregaties 229
agile Business Intelligence 46
alfanumeriek datatype 174
alternate key 126
ambitie 52
ambitieniveau 52
Analytics 238
appliance 184
architect 68
associaties 147
attentieve verwerking 270
attentive processing 270
attribuut 117
auditing 202

B

balanced scorecard 64, 263
bandbreedte 184
bar chart 274
batch layer 223
berekende kolommen 219
beschikbaarheid datawarehouse 179
Big Data 33
  definitie 154
bottom-up 243
boxplot 277
bronsleutel 126
Business Intelligence Competence Center (BICC) 42
Business Intelligence Life Cycle 43
Business Intelligence Maturity Matrix 51

C

chronologische vergelijking 276
cirkeldiagram 278
classification 247
cluster 211
clustered bar chart 275
clustered index 89
clustering 248
compliancy 196
compressie 177
concurrency 96
concurrent gebruikers 178
confirmatory analyse 242
conformed dimension 131
Corporate Information Factory (CIF) 142
correlatie 281
CPU-capaciteit 182
Customer Data Integration 194

D

dashboard 264
Data Analytics 238
data cleansing 214
Data Dictionary 192
data governance 195
datakwaliteit 213
Data Lake 27, 160
data latency 200
datamart 29, 109
datamining 31, 244
data scientist 259
datashadowsystems 39
data steward 69, 198
datatype 173
Data Vault 145
datawarehouse 27, 83
  grootte van 177
  magic quadrant 170
datumdimensie 118
  bouwen van 121
DBA'ers 68
DBMS 96
Decision Support Systems (DSS) 20
degenerate dimensie 134
Descriptive Analytics 239
detailniveau 113
  bepalen van 122
Diagnostic Analytics 239
dimensie 117
  bepalen van 115

dimensiematrix 131
dimensietabel 111
dimensioneel modelleren 109, 111
discretie 251
doelgroep 61
DOLAP 233
draaitabel 226, 283
drill-down 257
drill-through 257
dubbele gegevens 98

E

enterprise structure data 192
ethiek 249
ETL 189
  implementeren 199
  mappingtabel 209
ETL-proces 28
executive sponsor 69
exploratory analyse 243
explorers 62, 257

F

farmers 61, 254
feiten, soorten van 136
feitentabel 110
  soorten van 137
Filled Map 272
foutieve gegevens 101
frequentiepolygoon 280
front-end 237
functioneel ontwerp 171

G

Gartner magic quadrant 170
gebroken boekjaren 120
gebruikersaantal 178
gedegenereerde dimensie 134
geheugencapaciteit 183
geografische componenten 282
gesloten vraag 22
gestructureerde gegevens 156
grain 113
grain statement 115

H

Hadoop 212
hiërarchie 230
histogram 280
HOLAP 233
hubs 146
human datawarehouses 39

I

identiteit 252
inconsistente gegevens 101
index 177
indexstrategie 89
indicator 62
informatieanalist 68
informatieanalyse 22, 73, 74, 77
informatiebehoefte 60
Inmon, Bill 139
  kritiek op 143
intelligente organisatie 52
interview 22
IT-shadowsystems 39

J

junkdimensie 133

K

Kimball, Ralph 111
  kritiek op 139
klant 193
kolomnamen 218
kritieke prestatie-indicatoren (KPI's) 62, 260
kubus 30, 224
kwaliteitsproblemen 97

L

latency 200
lijngrafiek 277
links 147
logisch ontwerp 171

M

machine learning 32, 245
mapping diagram 207
Master Data 193
Master Data hub 195
Master Data Management 190, 194
matrix 283
Maturity Matrix 54
Maximum Consumption Rate (MCR) 182
MDX 228
members 225
metadata 192

miners 62, 258
mission statement 260
MOLAP 232
Multi Dimensional eXpressions (MDX) 228
multidimensionele database 30, 224

N

natuurlijke hiërarchie 230
niet-additief feit 136
nonclustered index 90
normaliseren 112
  dimensioneel 109
  van database 86
noSQL 34
numeriek datatype 173

O

ODS 29
OLAP-kubus 224
OLAP-workload 87
OLTP-workload 85
ongestructureerde gegevens 157
ontbrekende gegevens 99
open vraag 22
Operational Data Store (ODS) 29
operationeel niveau 55
opslagcapaciteit 184
organisatiestructuur 53
outlier detection 248

P

page size 175
parameters 267
performance 220
  features 186
  problemen 85
pie chart 273, 278
platgeslagen tabel 125
positiebepaling 278
preattentieve verwerking 270
preattentive processing 270
Predictive Analytics 32, 240
Prescriptive Analytics 241
primary key 122
privacy 251
product owner 68

Q

query
  performance 221
  schrijven van 92
  simpele, gemiddelde en moeilijke 178

R

rapport 266
  richtlijnen 275
rapportagedatabase 29, 91
RDBMS 33
record
  aantal per tabel 172
  gemiddelde lengte 173
redundantie 86
referenced architectures 185
reference data 192
regression 248
Relational DataBase Management System (RDBMS) 33
roadmap 55
ROLAP 233
row-based security 268

S

satellites 147
scale-out 155
scale-up 154
scatterplot 271, 281
schema-on-read 156
schema-on-write 156
schijfruimte 184
scope
  beperken van 73
  van een project 57
scorecard 262
scrum 75
security 268
self-service BI 21
semantisch model 30, 217
semi-additief feit 137
semi-gestructureerde gegevens 156
serving layer 223
signal-to-noiseratio 159
slice and dice 117
Slowly Changing Dimension (SCD) 105, 127

SMART 59, 260
smart key 122
snowflake 131
solid state disks (SSD) 184
source key 126
speed layer 223
spreadmarts 39
spreidingsdiagram 281
SPSS 258
SQL 209
staafdiagram 274, 275
stacked bar chart 275
staging database 29, 203
stermodel 109
  tekortkomingen 217
Storage Area Network (SAN) 94, 184
strategie 54
strategie van een organisatie 260
strategische niveau 54
strategy map 263
supervised learning 245
surrogate keys 126

T

taartdiagram 273, 278
tabel 284
table lens 281
tactisch niveau 55
technisch ontwerp 171
timing 24
top-down 242
to slice and dice 117
tourists 61, 255
transacties 147
transaction activity data 193
transaction audit data 193
transaction structure data 193
transparantie 251
tree map 273
Twitter 33
  datawarehouse 33

U

Unified Dimensional Model (UDM) 232
unknown member 134
unsupervised learning 246

V

value 159
variety 155
velocity 157, 158
veracity 158
verdeling 279
vergelijking
  chronologische 276
  van nominale waarden 275
verhoudingen 278
volume 154
volwassenheidsmodellen 38

W

warehouse keys 126
weeknummer 120
wildgroei 267
workload 178

Literatuurlijst

Van Beek, Daan (2020) *De intelligente, datagedreven organisatie* (7e ed.) Passioned publishers.

Computerworld.nl (z.d.) *Wat is hadoop.* https://www.computerworld.nl/big-data/75185-wat-is-hadoop.

Gartner, Inc (2021) *Analytics and Business Intelligence (ABI).* https://www.gartner.com/en/information-technology/glossary/business-intelligence-bi.

Gartner, Inc (2021) *Big Data.* www.gartner.com/it-glossary/big-data.

Den Hamer, Pieter (2004) *Business Intelligence* (2de ed.) Academic Service.

Healey, Christopher G. (2012) *Perception in Visualization.* https://www.csc2.ncsu.edu/faculty/healey/PP/.

Howson, Cindy, Duncan, Alan D. (2015) *ITScore Overview for BI and Analytics.* Gartner.https://www.gartner.com/en/documents/3136418/itscore-overview-for-bi-and-analytics.

Inmon, Bill (2007) https://web.archive.org/web/20160116082130/http://www.inmoncif.com/home/.

Kimball, Ralph, Ross, Margy (2013) *The Data Warehouse Toolkit* (3e ed.) John Wiley & Sons Inc.

Linstedt, Dan (2015) *Data Vault Basics.* https://danlinstedt.com/solutions-2/data-vault-basics/.

Mitchell, Tom M. (1997) *Machine Learning.* McGraw-Hill International Editions.

Norton, David, Kaplan, Robert (1996) *The balanced scoreboard.* Harvard Business Review Press.

Power, Dan (z.d.) *Master Data Management.* www.information-management.com.

Routledge, Richard (2005) *Law of large numbers – statistics.* Britannica. https://www.britannica.com/science/law-of-large-numbers.

TDWI (z.d.) *The Data Warehouse Institute.* https://tdwi.org.

Vitt, Elizabeth, Luckevich, Michael, Misner, Stacia (2002) *Business Intelligence – Making better decisions faster.* Microsoft press.

Wikipedia (2021) *Business intelligence.* https://nl.wikipedia.org/wiki/Business_intelligence.

Illustratieverantwoording

Hoofdstuk 1
Figuren: ter Braake. Peter
Hoofdstukopener: SFIO CRACHO/Shutterstock

Hoofdstuk 2
Figuur 2.1, 2.2: ter Braake, Peter
Figuur 2.3: geïnspireerd op: van Beek, Daan, *Crash course 'De intelligente, datagedreven organisatie'*, geraadpleegd op 13 oktober '21: https://www.passionned.nl/crash-course-de-intelligente-organisatie/
Figuur 2.4: Hamer, P. den (2010) *Business intelligence* (2e druk). Den Haag: Academic Service.
Figuur 2.5: Hamer, P. den (2010) *Business Intelligence* (2e druk). Den Haag: Academic Service.
Figuur 2.6: Linstedt, Dan, *Data vault basics*, geraadpleegd op 13 oktober '21: https://danlinstedt.com/solutions-2/data-vault-basics/
Figuur 2.7: R. Kaplan, D. Norton (1996) *The balanced Scoreboard*, Harvard Business School.
Figuur 2.8: ter Braake, Peter
Figuur 2.9: ter Braake, Peter
Hoofdstukopener: Rawpixel.com/Shutterstock

Hoofdstuk 3
Figuren: ter Braake, Peter
Hoofdstukopener: StudioPortoSabbia/Shutterstock

Hoofdstuk 4
Figuur 4.1-4.12: ter Braake, Peter
Figuur 4.13: Linstedt, Dan, *Data vault basics*, geraadpleegd op 13 oktober '21: https://danlinstedt.com/solutions-2/data-vault-basics/
Hoofdstukopener: Juice Flair/Shutterstock

Hoofdstuk 5
Figuur 5.1: Das, Joydeep (2018) *Introduction to Azure Data Lake*, geraadpleegd op 13 oktober '21: https://dzone.com/articles/introduction-to-azure-data-lake
Figuur 5.2-5.4: ter Braake, Peter
Hoofdstukopener: kubais/Shutterstock

Hoofdstuk 6
Figuur 6.1: Gartner inc.
Figuur 6.2-6.3: ter Braake, Peter
Figuur 6.4: Microsoft Official Course: *Implementing a Datawarehouse*
Hoofdstukopener: Chepko Danil Vitalevich/Shutterstock

Hoofdstuk 7
Figuur 7.1: Malcolm Chisholm, b-eye.network.com
Figuur 7.2-7.3: https://www.quotemaster.org/data+management
Figuur 7.4: Longman, Cliff (2008) *Ten Tips for Master Data Management Success*, geraadpleegd op 13 oktober '21: https://tdwi.org/Articles/2008/06/25/Ten-Tips-for-Master-Data-Management-Success.aspx?Page=2
Figuur 7.5-7.10: ter Braake, Peter
Figuur 7.11: Microsoft Official Course: *Implementing a Datawarehouse*
Figuur 7.12: ter Braake, Peter
Figuur 7.13: ter Braake, Peter
Hoofdstukopener: Ugorova Irina/Shutterstock

Hoofdstuk 8
Figuur 8.1: Asanka, Dinesh (2020) *OLAP Cubes in SQL Server*, geraadpleegd op 13 oktober '21: https://www.sqlshack.com/olap-cubes-in-sql-server/
Figuur 8.2-8.3: ter Braake, Peter
Hoofdstukopener: Rawpixel.com/Shutterstock

Hoofdstuk 9
Figuur 9.1: Gartner inc.
Figuur 9.2: ter Braake, Peter
Figuur 9.3: Feldbauer, Roman (2020) *Nearest Neighbors Classification*, geraadpleegd op 13 oktober '21: https://scikit-hubness.readthedocs.io/en/latest/documentation/auto_examples/plot_classification.html
Figuur 9.4: Sewaqu, Example of simple linear regression, which has one independent variable, geraadpleegd op 13 oktober '21: https://commons.wikimedia.org/wiki/File:Linear_regression.svg (public domain)
Figuur 9.5: ter Braake, Peter
Figuur 9.6 Strategy map template, geraadpleegd op 13 oktober '21: https://www.valuebasedmanagement.net/methods_strategy_maps_strategic_communication.html
Figuur 9.7-9.15, 9.17-9.19 Ter Braake, Peter
Hoofdstukopener: tadamichi/Shutterstock

(Voor deze editie is getracht alle afbeeldingen van een bron te voorzien. Mocht u nog aanvullingen hebben, dan horen wij het graag.)

Jarenlang hebben organisaties gegevens verzameld in databases. Nu moeten zij de stap maken naar zinvol gebruik van die gegevens: Business Intelligence. Goed toegepast kan Business Intelligence een organisatie kosteneffectiever laten werken. Of een voorsprong geven op concurrenten.

Leerboek Business Intelligence beantwoordt de volgende vragen:

- Wat is Business Intelligence, waarom zou je het toepassen en wat zijn de doelen?
- Wat is een datawarehouse en waar heb je dat voor nodig?
- Hoe vul je een datawarehouse met gegevens en waar moet je dan rekening mee houden?
- Wie zijn de gebruikers en hoe werken ze ermee?

Daarbij komen ook ETL, Analytics en het front-end uitgebreid aan de orde. Verder wordt dieper ingegaan op (Advanced) Analytics: van (het belang van) visualisaties tot en met toepassingen van Data Science en Machine Learning. Uiteraard kan de rol van de cloud hierbij niet onbelicht blijven. Om de relatie met de praktijk te leggen worden diverse voorbeelden uitgewerkt.

Deze derde druk is geactualiseerd en aangevuld. In deze editie is meer aandacht voor bedrijfskundige aspecten en zijn meer voorbeelden en opdrachten toegevoegd.

Via **www.boomstudent.nl** heb je toegang tot de online-omgeving van dit boek met meerkeuzevragen en feedback, uitgewerkte voorbeelden en de daarin gebruikte bestanden en scripts. Ook zijn hier het online boek en extra paragrafen te raadplegen.

Leerboek Business Intelligence is geschreven voor studenten van bedrijfskundige studies, maar ook voor sommige informaticarichtingen. Het boek richt zich zowel op technische als niet-technische lezers en is daarom ook geschikt voor professionals in het bedrijfsleven die een beter begrip willen krijgen van Business Intelligence in de breedste zin.

Peter ter Braake is zelfstandig SQL Server en Business Intelligence docent/consultant. Hij is auteur van de boeken *Handboek Power BI* en *Database modelleren*.

9 789024 441662

www.boomstudent.nl
www.boomhogeronderwijs.nl